西安电子科技大学
XIDIAN UNIVERSITY

2025数据库系统能力赛培训

5-查询处理和优化

李辉　2025.6

西安电子科技大学计算机科学与技术学院

hli@xidian.edu.cn

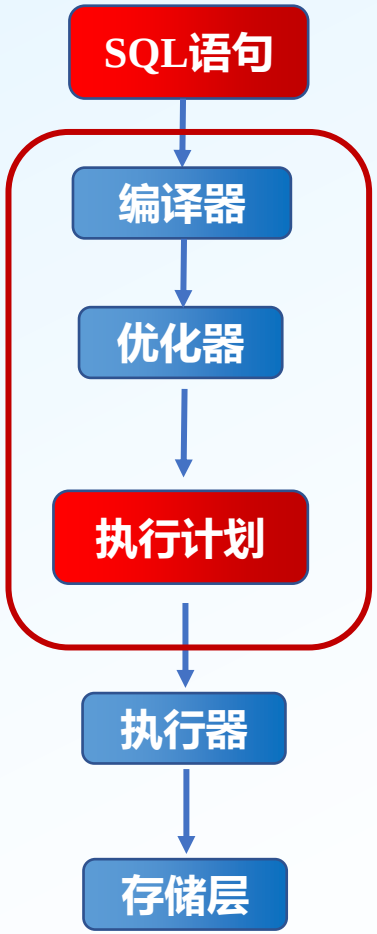
https://lihuixidian.github.io

https://dbedu.xidian.edu.cn

存储和索引




```
SELECT   *
         FROM   Student
WHERE    StudentID = '95003';
```



- 词法分析
- 语法分析
- 语义检查
- 授权检查

- join算子
- sort算子
- scan算子
- …

读取page 4

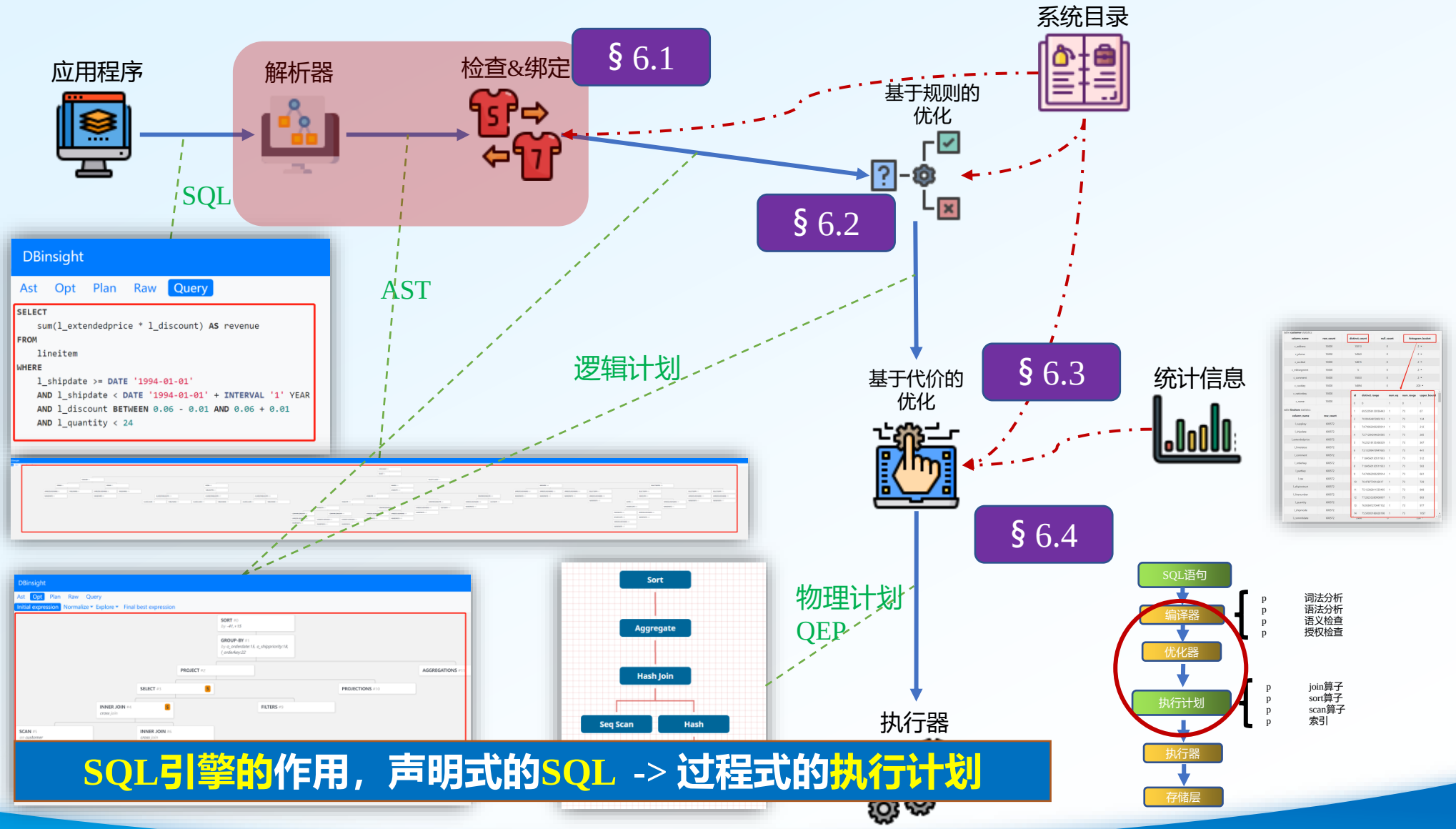
95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
…	…	…	…	…

Page4

存储引擎 ⟷ 持久性存储

查询处理 pipeline

DBEDU *All examples are provided by our DBEDU platform (Dbinsight & LANTERN subsystems) Visit dbedu.xidian.edu.cn and have a hand-on experience yourself!



本章内容

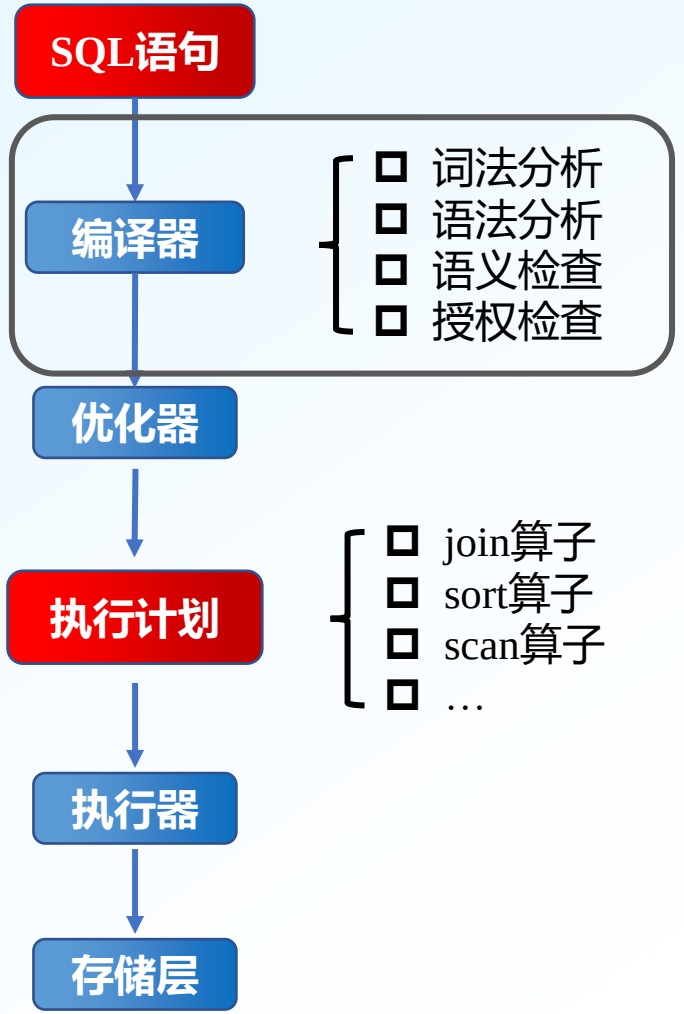


一、查询解析

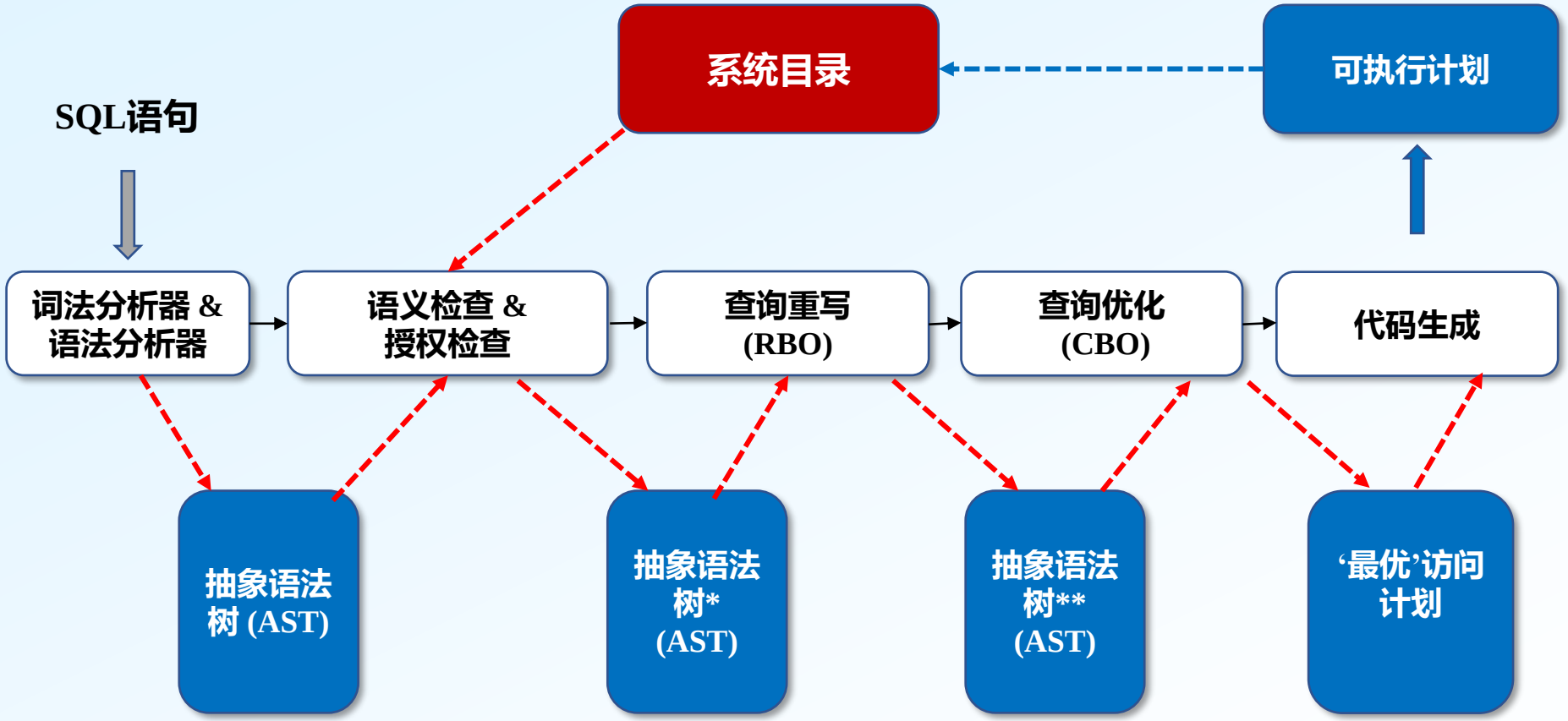
二、逻辑重写

三、物理算子

四、查询优化



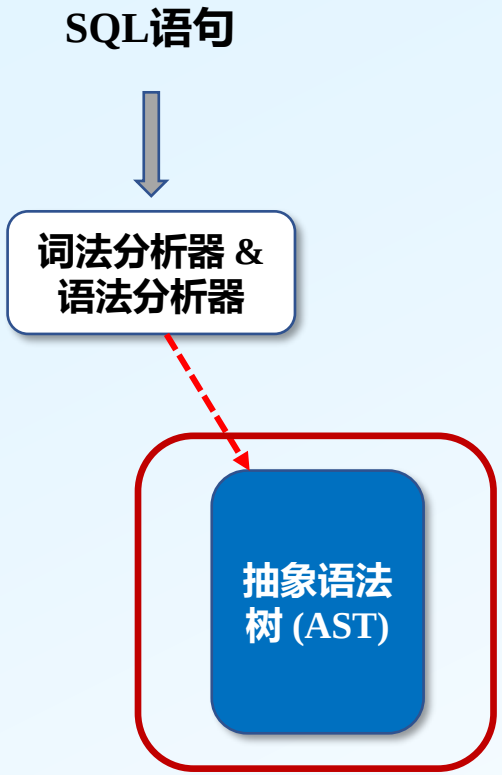
整体流程



* 带语义信息的AST

** 表示查询重写后的AST

抽象语法树 Abstract Syntax Tree



词法分析器 & 语法分析器


```
SELECT   id,  quantity
FROM   order
WHERE   id = 1024;
```

(正则表达式：例如Lex)

词法分析器

SELECT id , quantity ... WHERE id = 1024

(上下文无关语法，例如Yacc)

语法分析器

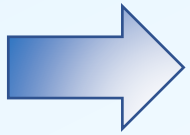


词法分析器

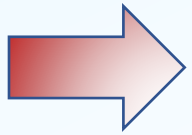
- **自动生成的词法分析器(如Lex)：**
  - 使用正则表达式来定义元语言；
  - Lex将正则表达式作为输入，生成对应的词法分析器。
- **自动生成的语法分析器 (如Yacc)：**
  - 使用元语言定义SQL的语法规则；
  - Yacc将语法规则作为输入，生成对应的语法分析器。该语法分析器实际上是一个下推自动机(pushdown automata)，或带栈自动机(automata with a stack)。
- **语义处理的关键操作：**
  - 移进操作(将字符标记放入栈中)；
  - 归约操作(用非终结符代替栈的顶部元素)。

词法分析器示例

SELECT id, quantity
FROM order
WHERE id = 1024;



词法分析器



字符标记流

“SELECT”

“ID: id”

“SEP: ,”

“ID: quantity”

“FROM”

“ID: order”

“WHERE”

“ID: id”

“OP: =”

“INT: 1024”

“SEP: ;”

词法分析器示例

- **SQL语言词法规则示例（正则表达式）**
  - **ID　[a-z][a-z0-9]+**
  - **Keyword　select | from | where**
  - **SEP　, | ;**
  - **OP　= | + | <**
  - **INT　[0-9]+**

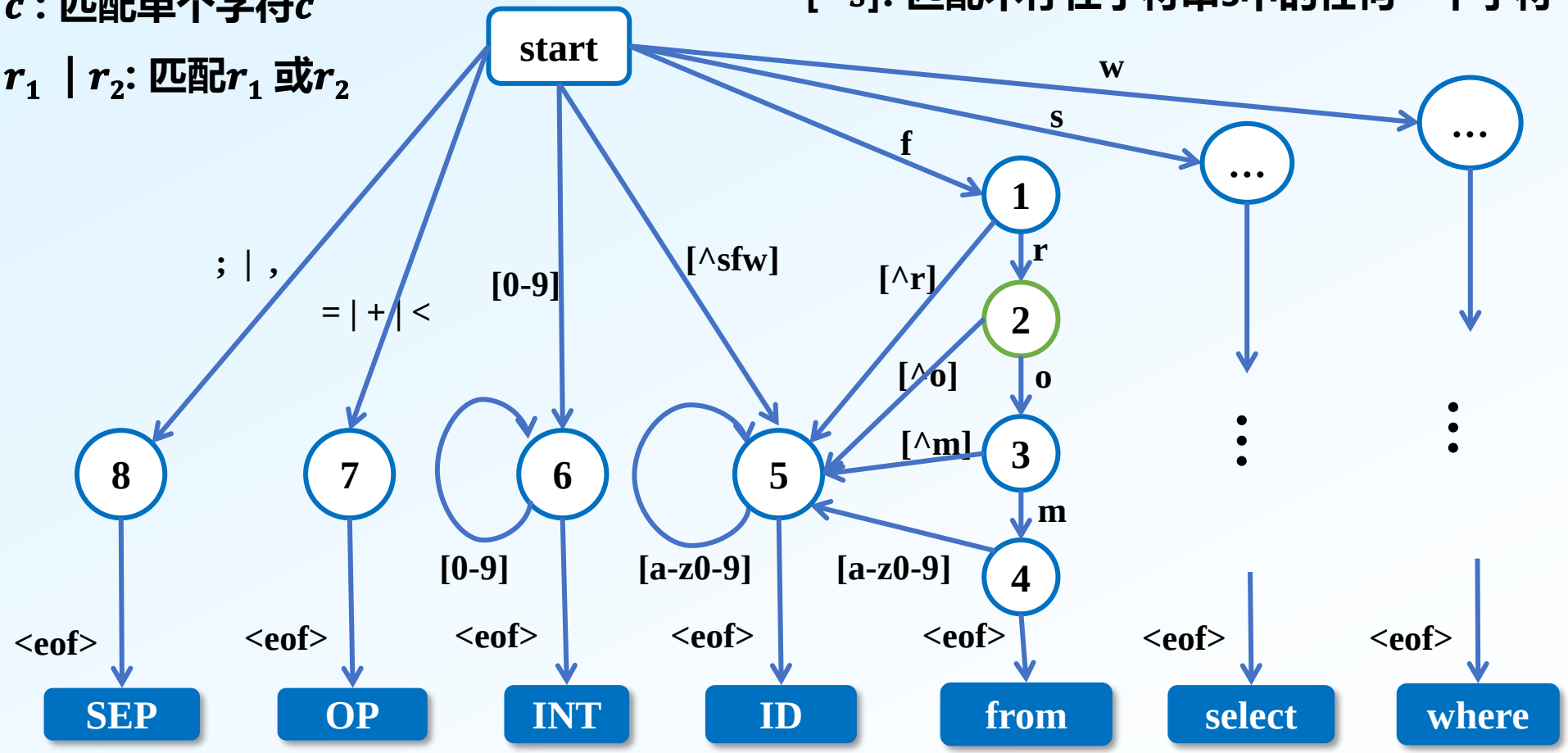
词法分析器示例

$[s]$: 匹配字符串s中的任何一个字符

$[\hat{}s]$: 匹配不存在字符串s中的任何一个字符

c：匹配单个字符c

$r_1 \mid r_2$: 匹配r_1 或r_2



词法分析器示例

字符标记流

“SELECT”

“ID: id”

“SEP: ,”

“ID: quantity”

“FROM”

“ID: order”

“WHERE”

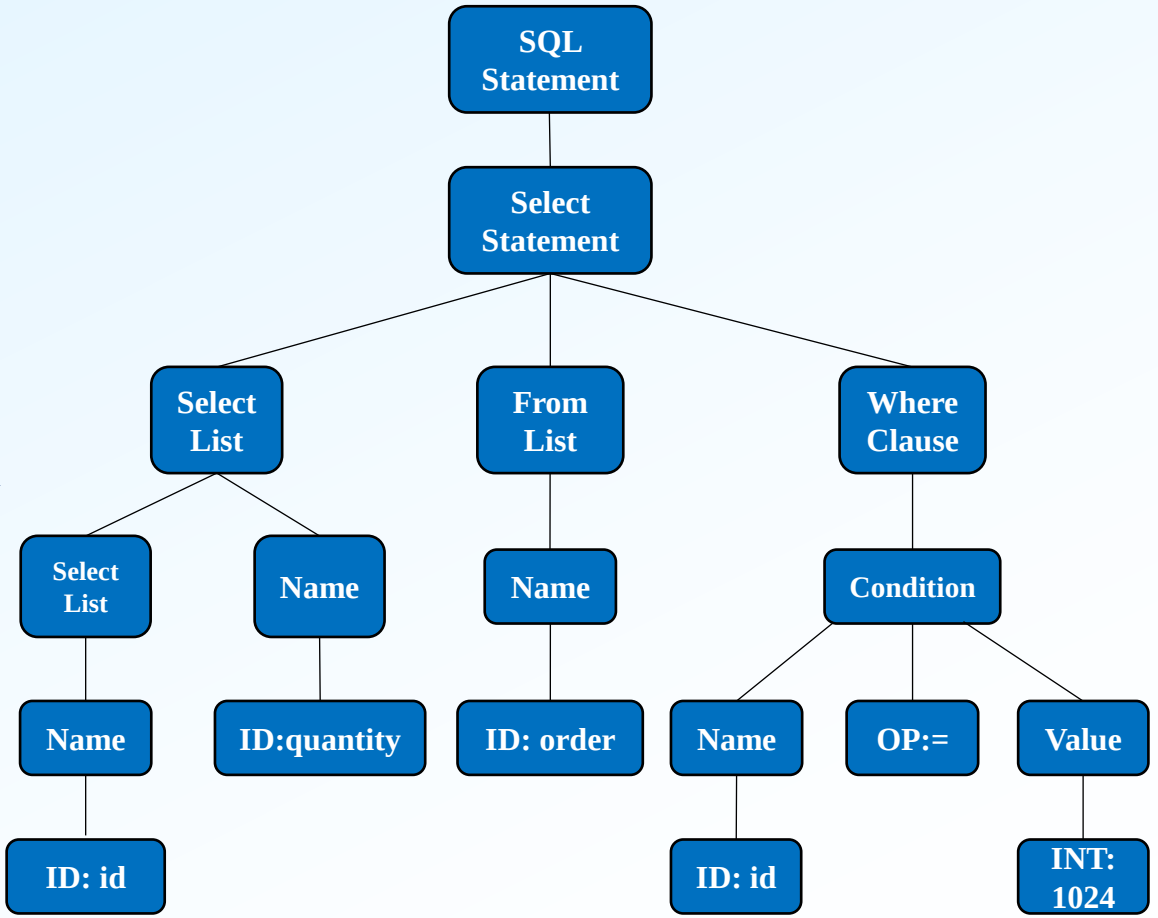
“ID: id”

“OP: =”

“INT: 1024”

“SEP: ;”

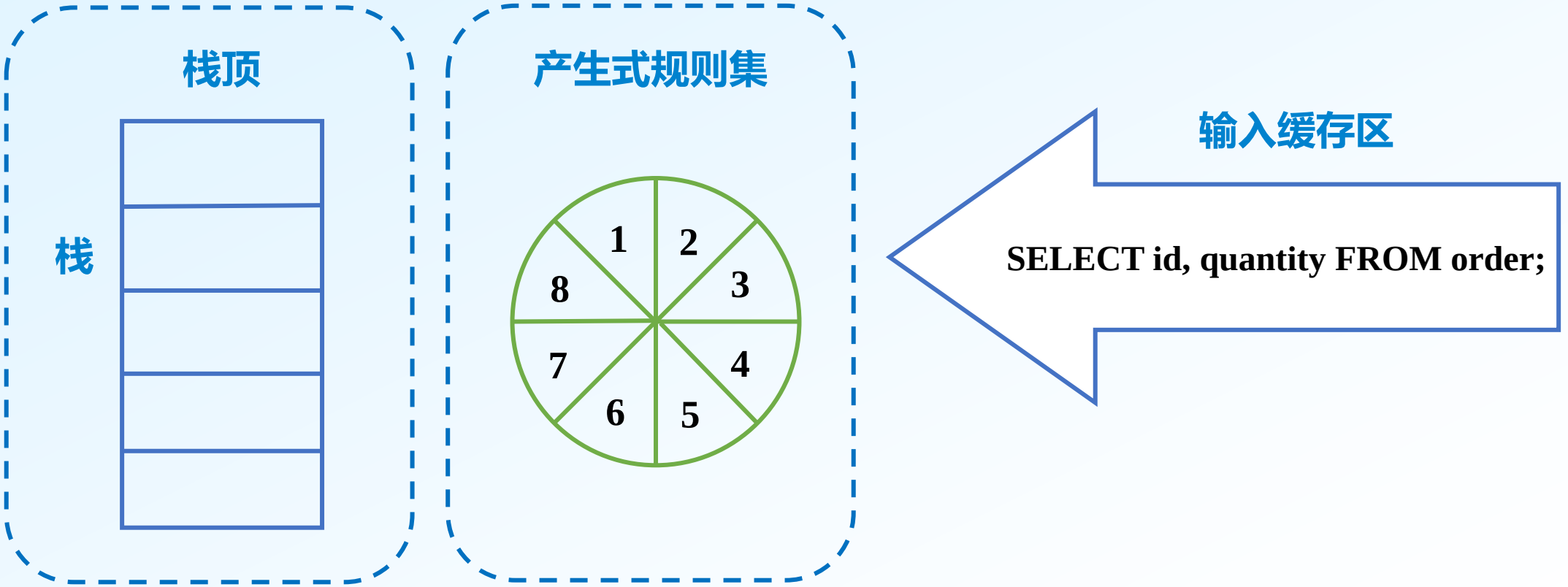
词法分析器



词法分析器示例

- **SQL语言的上下文无关语法示例：** **<栈顶New(AST父节点)> := <栈顶Old(AST子节点)>**
  - **Rule 1：** **<SQL Statement> ::= <Select Statement> | <Insert Statement> | <Update Statement>**
  - **Rule 2：** **<SelectStatement> ::= SELECT <SelectList> FROM <FromList> WHERE <WhereClause>';'**
  - **Rule 3：** **<SelectList> ::= <SelectList>, <Name> | <Name> | '*'**
  - **Rule 4：** **<Name> ::= <ID> | <ID>.<ID>**
  - **Rule 5：** **<FromList> ::= <FromList>,<Name> | <Name>**
  - **Rule 6：** **<WhereClause> ::= <Condition>**
  - **Rule 7：** **<Condition> ::= <Name> <OP> <Value> | <Condition> AND <Condition>**
  - **Rule 8：** **<Value> ::= <INT> | <FLOAT> | <DOUBLE>**

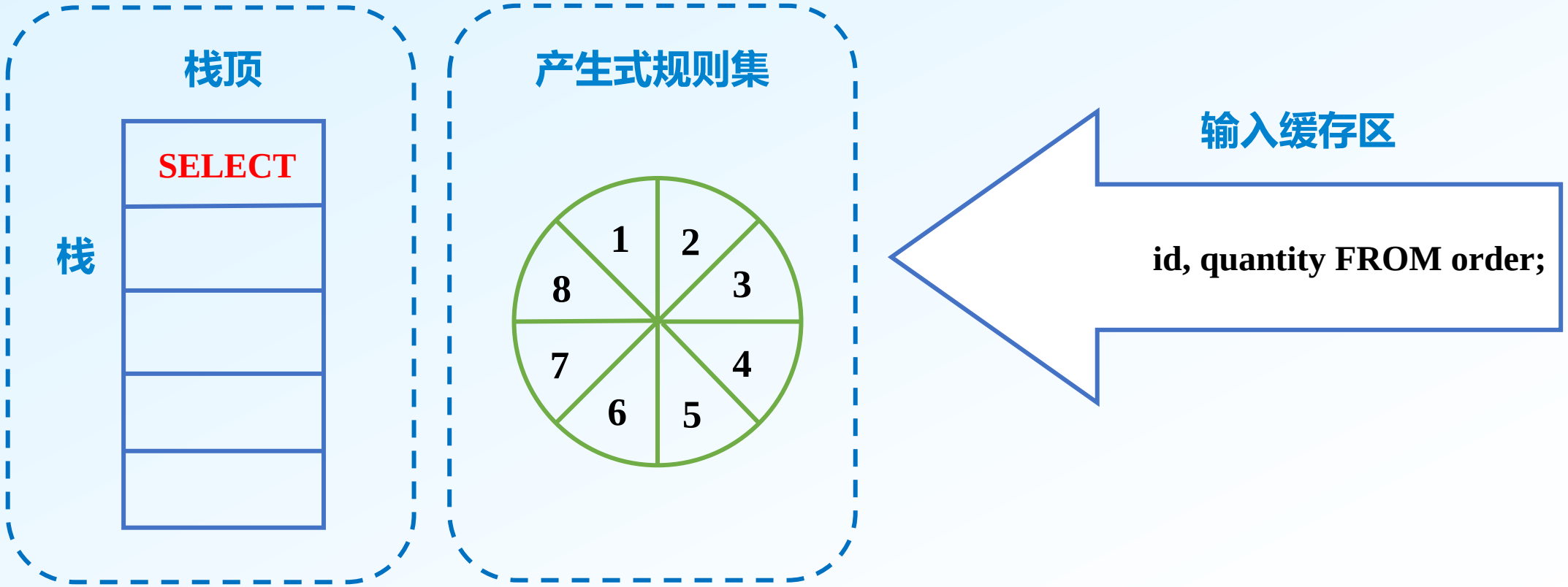
移进和规约（shift & reduce）



语法分析器图示

移进和规约（shift & reduce）

- **移进：将字符标记放入栈中**



语法分析器图示

移进和规约（shift & reduce）

- **规约：用非终结符代替栈的顶部元素**

id —归约（规则4）→ <Name>　　<Name> —归约（规则3）→ <SelectList>



■ **字符标记流：** “SELECT” “ID:id” “SEP: ,” “ID:quantity” “FROM” “ID:order” “WHERE” “ID:id” “OP: =” “INT:1024”“SEP: ;”

step	stack	look ahead	action
1	empty	SELECT	移进

■ **SQL语言的上下文无关语法示例：**

Rule 1： <SQL Statement> ::= <Select Statement> | <Insert Statement> | <Update Statement>

Rule 2： <SelectStatement> ::= SELECT <SelectList> FROM <FromList> WHERE <WhereClause>';'

Rule 3： <SelectList> ::= <SelectList>, <Name> | <Name> | '*'

Rule 4： <Name> ::= <ID> | <ID>.<ID>

Rule 5： <FromList> ::= <FromList>,<Name> | <Name>

Rule 6： <WhereClause> ::= <Condition>

Rule 7： <Condition> ::= <Name> <OP> <Value> | <Condition> AND <Condition>

Rule 8： <Value> ::= <INT> | <FLOAT> | <DOUBLE>

■ **字符标记流：** “SELECT” “ID:id” “SEP: ,” “ID:quantity” “FROM” “ID:order” “WHERE” “ID:id” “OP: =” “INT:1024”“SEP: ;”

step	stack	look ahead	action
1	empty	SELECT	移进
2	SELECT	ID:id	移进

■ **SQL语言的上下文无关语法示例：**

Rule 1： <SQL Statement> ::= <Select Statement> | <Insert Statement> | <Update Statement>

Rule 2： <SelectStatement> ::= SELECT <SelectList> FROM <FromList> WHERE <WhereClause>';'

Rule 3： <SelectList> ::= <SelectList>, <Name> | <Name> | '*'

Rule 4： <Name> ::= <ID> | <ID>.<ID>

Rule 5： <FromList> ::= <FromList>,<Name> | <Name>

Rule 6： <WhereClause> ::= <Condition>

Rule 7： <Condition> ::= <Name> <OP> <Value> | <Condition> AND <Condition>

Rule 8： <Value> ::= <INT> | <FLOAT> | <DOUBLE>

■ **字符标记流：** “SELECT” “ID:id” “SEP: ,” “ID:quantity” “FROM” “ID:order” “WHERE” “ID:id” “OP: =” “INT:1024”“SEP: ;”

step	stack	look ahead	action
1	empty	SELECT	移进
2	SELECT	ID:id	移进
3	SELECT ID	SEP: ,	归约(Rule 4)

ID: id

■ **SQL语言的上下文无关语法示例：**

Rule 1： <SQL Statement> ::= <Select Statement> | <Insert Statement> | <Update Statement>

Rule 2： <SelectStatement> ::= SELECT <SelectList> FROM <FromList> WHERE <WhereClause>';'

Rule 3： <SelectList> ::= <SelectList>, <Name> | <Name> | '*'

Rule 4： <Name> ::= <ID> | <ID>.<ID>

Rule 5： <FromList> ::= <FromList>,<Name> | <Name>

Rule 6： <WhereClause> ::= <Condition>

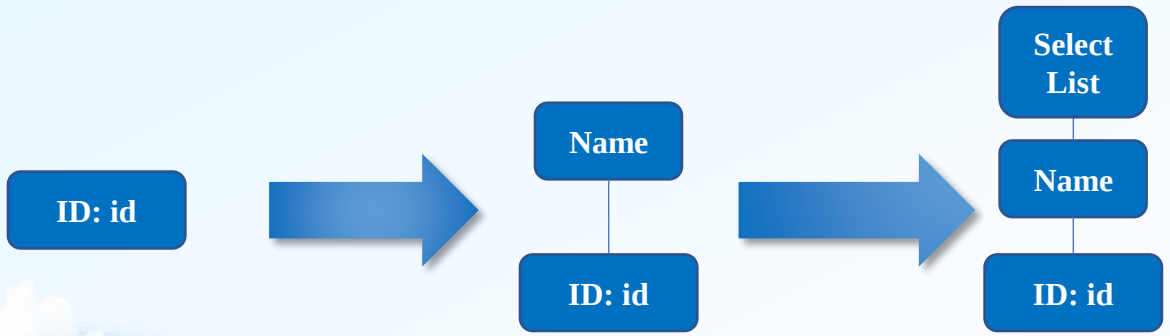
Rule 7： <Condition> ::= <Name> <OP> <Value> | <Condition> AND <Condition>

Rule 8： <Value> ::= <INT> | <FLOAT> | <DOUBLE>

词法分析器 & 语法分析器

■ **字符标记流：** “SELECT” “ID:id” “SEP: ,” “ID:quantity” “FROM” “ID:order” “WHERE” “ID:id” “OP: =” “INT:1024”“SEP: ;”

step	stack	look ahead	action
1	empty	SELECT	移进
2	SELECT	ID:id	移进
3	SELECT ID	SEP: ,	归约(Rule 4)
4	SELECT <Name>	SEP: ,	归约(Rule 3)



■ **SQL语言的上下文无关语法示例：**

Rule 1： <SQL Statement> ::= <Select Statement> | <Insert Statement> | <Update Statement>

Rule 2： <SelectStatement> ::= SELECT <SelectList> FROM <FromList> WHERE <WhereClause>';'

Rule 3： <SelectList> ::= <SelectList>, <Name> | <Name> | '*'

Rule 4： <Name> ::= <ID> | <ID>.<ID>

Rule 5： <FromList> ::= <FromList>,<Name> | <Name>

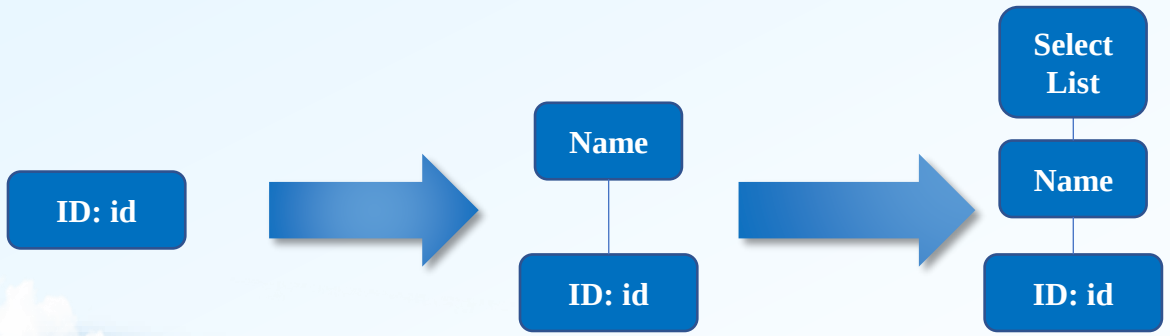
Rule 6： <WhereClause> ::= <Condition>

Rule 7： <Condition> ::= <Name> <OP> <Value> | <Condition> AND <Condition>

Rule 8： <Value> ::= <INT> | <FLOAT> | <DOUBLE>

■ **字符标记流：** “SELECT” “ID:id” “SEP: ,” “ID:quantity” “FROM” “ID:order” “WHERE” “ID:id” “OP: =” “INT:1024”“SEP: ;”

step	stack	look ahead	action
1	empty	SELECT	移进
2	SELECT	ID:id	移进
3	SELECT ID	SEP: ,	归约(Rule 4)
4	SELECT <Name>	SEP: ,	归约(Rule 3)
5	SELECT <SelectList>	SEP: ,	移进



■ **SQL语言的上下文无关语法示例：**

Rule 1： <SQL Statement> ::= <Select Statement> | <Insert Statement> | <Update Statement>

Rule 2： <SelectStatement> ::= SELECT <SelectList> FROM <FromList> WHERE <WhereClause>';'

Rule 3： <SelectList> ::= <SelectList>, <Name> | <Name> | '*'

Rule 4： <Name> ::= <ID> | <ID>.<ID>

Rule 5： <FromList> ::= <FromList>,<Name> | <Name>

Rule 6： <WhereClause> ::= <Condition>

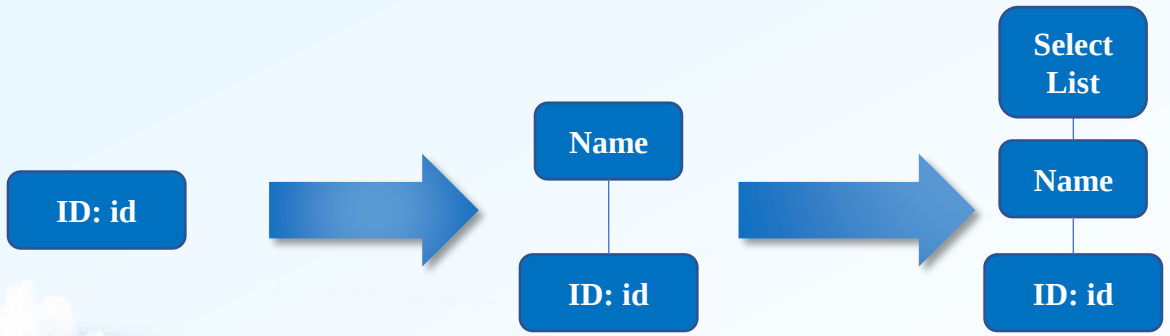
Rule 7： <Condition> ::= <Name> <OP> <Value> | <Condition> AND <Condition>

Rule 8： <Value> ::= <INT> | <FLOAT> | <DOUBLE>

词法分析器 & 语法分析器

■ **字符标记流：** “SELECT” “ID:id” “SEP: ,” “ID:quantity” “FROM” “ID:order” “WHERE” “ID:id” “OP: =” “INT:1024”“SEP: ;”

step	stack	look ahead	action
1	empty	SELECT	移进
2	SELECT	ID:id	移进
3	SELECT ID	SEP: ,	归约(Rule 4)
4	SELECT <Name>	SEP: ,	归约(Rule 3)
5	SELECT <SelectList>	SEP: ,	移进
6	SELECT <SelectList>,	ID:quantity	移进



■ **SQL语言的上下文无关语法示例：**

Rule 1： <SQL Statement> ::= <Select Statement> | <Insert Statement> | <Update Statement>

Rule 2： <SelectStatement> ::= SELECT <SelectList> FROM <FromList> WHERE <WhereClause>';'

Rule 3： <SelectList> ::= <SelectList>, <Name> | <Name> | '*'

Rule 4： <Name> ::= <ID> | <ID>.<ID>

Rule 5： <FromList> ::= <FromList>,<Name> | <Name>

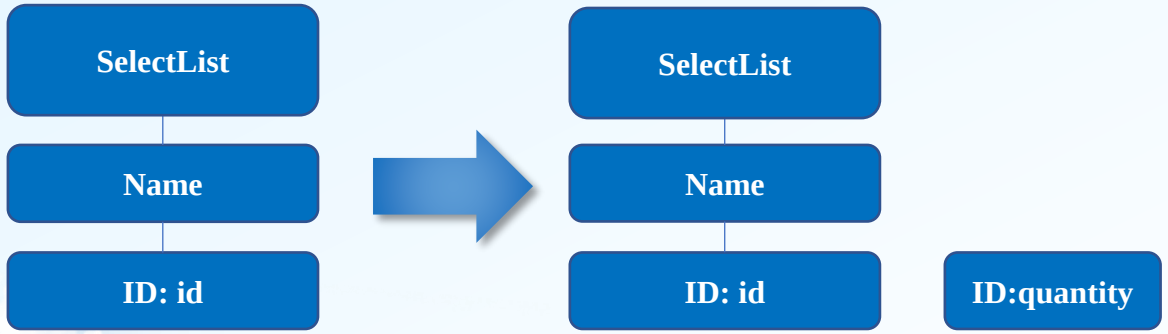
Rule 6： <WhereClause> ::= <Condition>

Rule 7： <Condition> ::= <Name> <OP> <Value> | <Condition> AND <Condition>

Rule 8： <Value> ::= <INT> | <FLOAT> | <DOUBLE>

■ **字符标记流：** “SELECT” “ID:id” “SEP: ,” “ID:quantity” “FROM” “ID:order” “WHERE” “ID:id” “OP: =” “INT:1024”“SEP: ;”

step	stack	look ahead	action
6	SELECT <SelectList>,	ID:quantity	移进
7	SELECT <SelectList>,ID	FROM	归约(Rule 4)



■ **SQL语言的上下文无关语法示例：**

Rule 1： <SQL Statement> ::= <Select Statement> | <Insert Statement> | <Update Statement>

Rule 2： <SelectStatement> ::= SELECT <SelectList> FROM <FromList> WHERE <WhereClause>';'

Rule 3： <SelectList> ::= <SelectList>, <Name> | <Name> | '*'

Rule 4： <Name> ::= <ID> | <ID>.<ID>

Rule 5： <FromList> ::= <FromList>,<Name> | <Name>

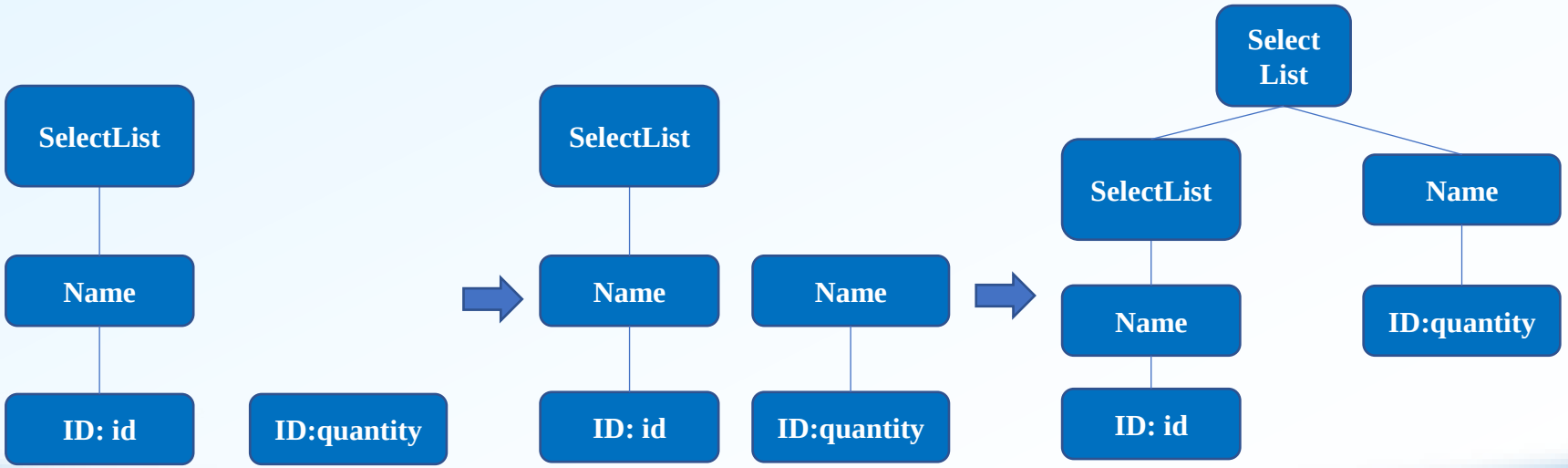
Rule 6： <WhereClause> ::= <Condition>

Rule 7： <Condition> ::= <Name> <OP> <Value> | <Condition> AND <Condition>

Rule 8： <Value> ::= <INT> | <FLOAT> | <DOUBLE>

- **字符标记流：** “SELECT” “ID:id” “SEP: ,” “ID:quantity” “FROM” “ID:order” “WHERE” “ID:id” “OP: =” “INT:1024”“SEP: ;”

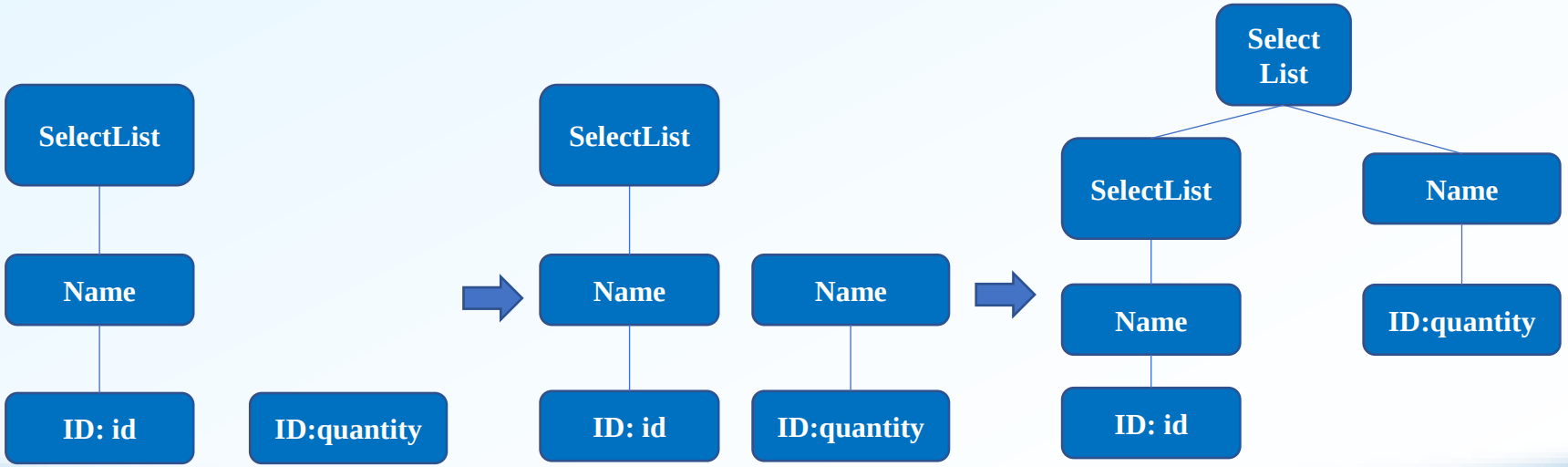
step	stack	look ahead	action
6	SELECT <SelectList>,	ID:quantity	移进
7	SELECT <SelectList>,ID	FROM	归约(Rule 4)
8	SELECT <SelectList>, <Name>	FROM	归约(Rule 3)



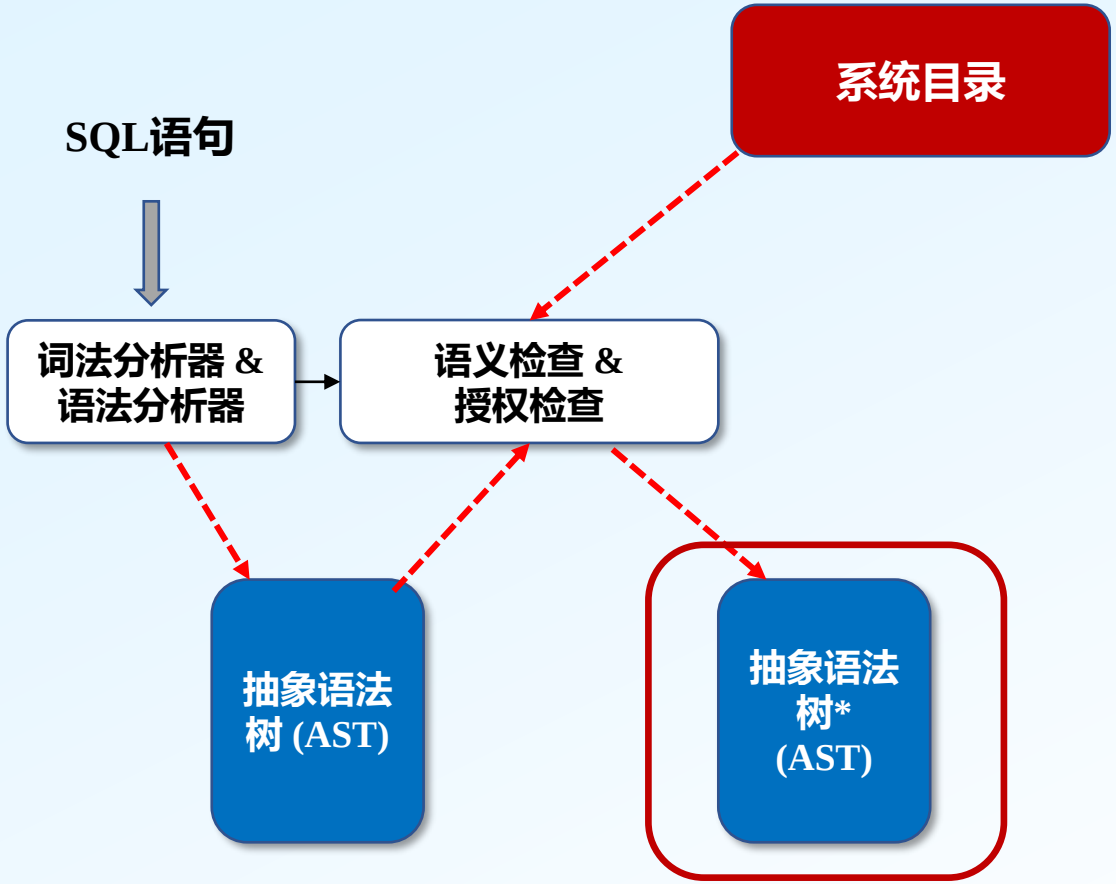
词法分析器 & 语法分析器

- **字符标记流：** “SELECT” “ID:id” “SEP: ,” “ID:quantity” “FROM” “ID:order” “WHERE” “ID:id” “OP: =” “INT:1024”“SEP: ;”

step	stack	look ahead	action
6	SELECT <SelectList>,	ID:quantity	移进
7	SELECT <SelectList>,ID	FROM	归约(Rule 4)
8	SELECT <SelectList>, <Name>	FROM	归约(Rule 3)
9	SELECT <SelectList>	FROM	shift



抽象语法树*



* 带语义信息的AST

** 表示查询重写后的AST

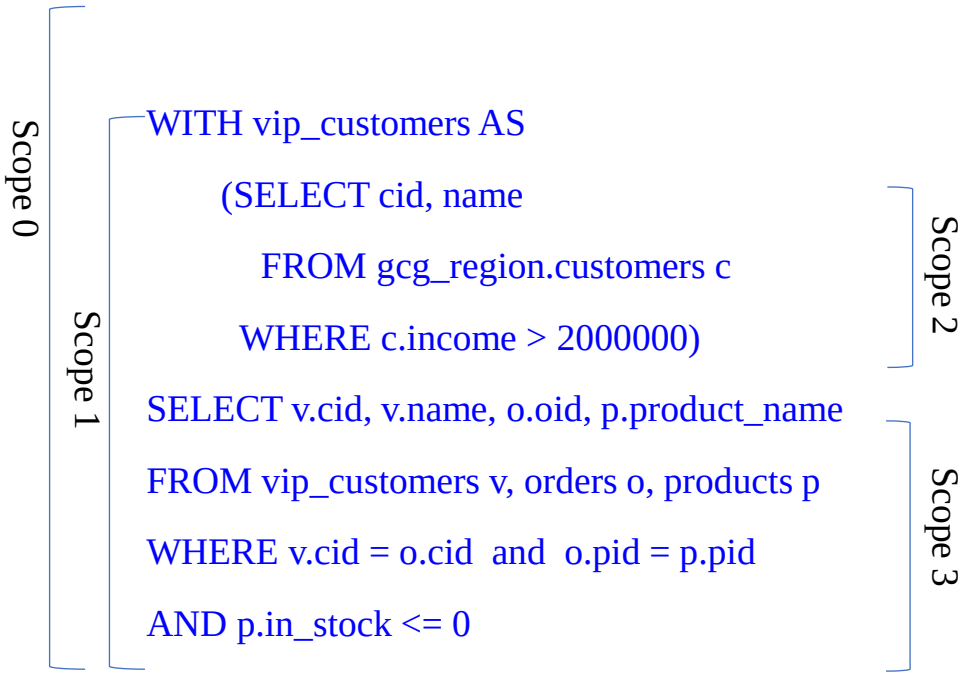
符号表和名称作用域

- **系统目录表:**

  表名、视图名、列名、函数名、类型名等。

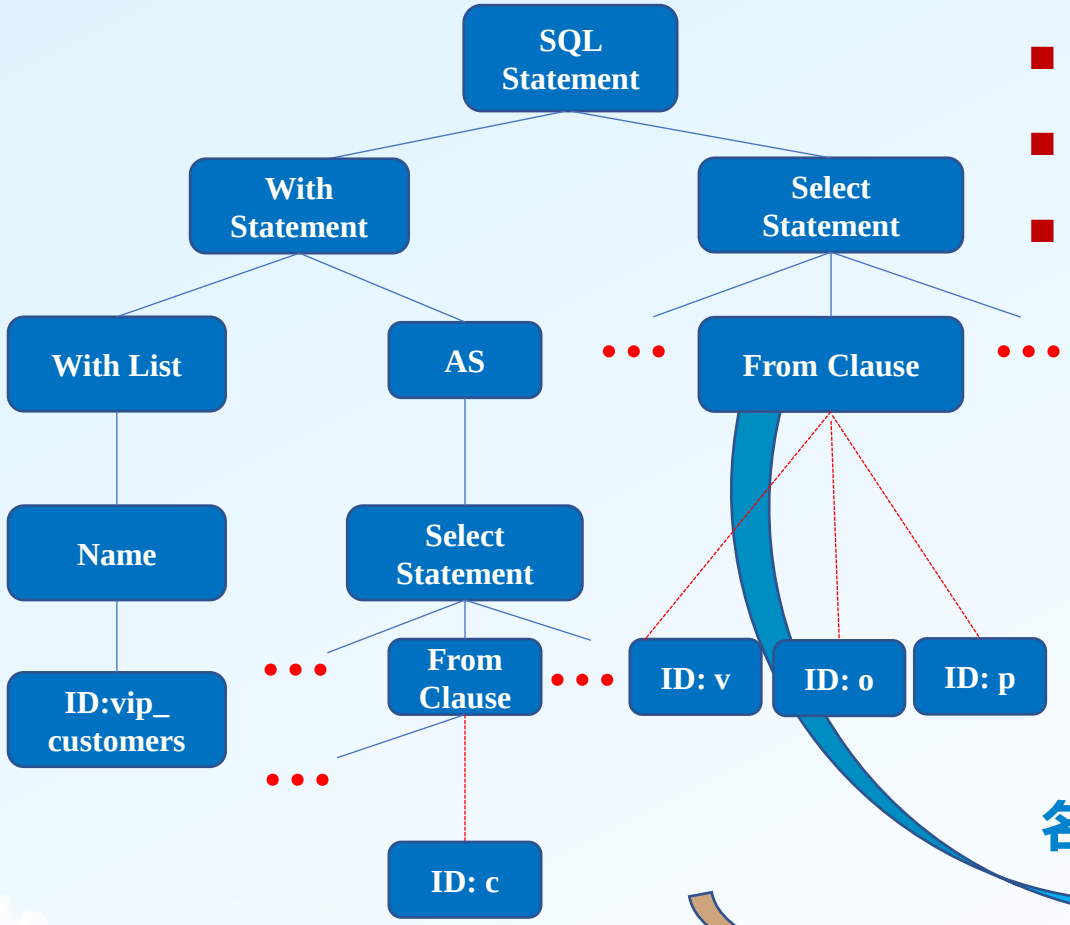
- **在(嵌套的)查询块中定义的名称:**

  表表达式(WITH子句)，表别名(FROM子句)，等等。

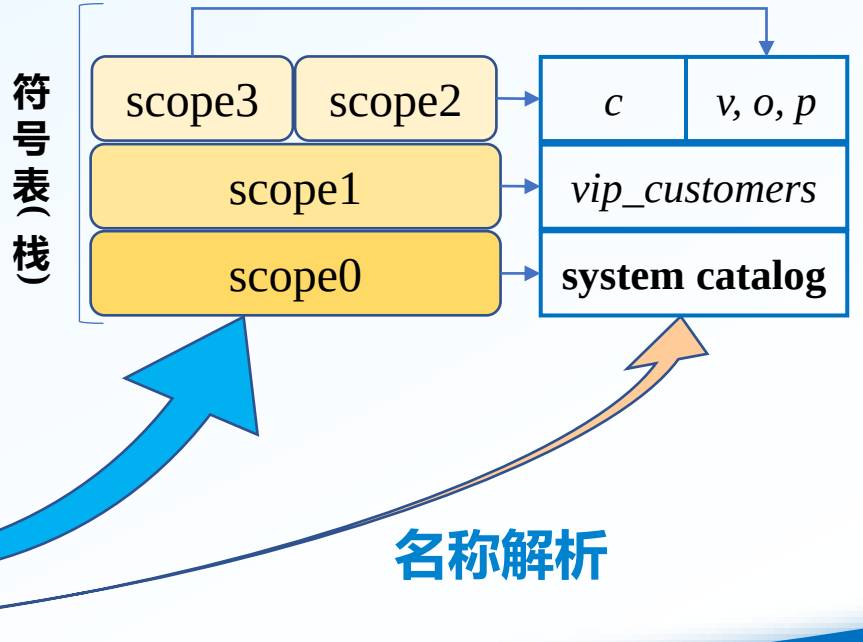


- **名称作用域的层次结构:**
  - **Scope 0: System catalog**
  - **Scope**
    - **vip_customers**
    - **Scope 2: c**
    - **Scope 3: v, o, p**

- **WITH** vip_customers AS (SELECT cid, name FROM gcg_region.customers c WHERE c.income > 2000000)

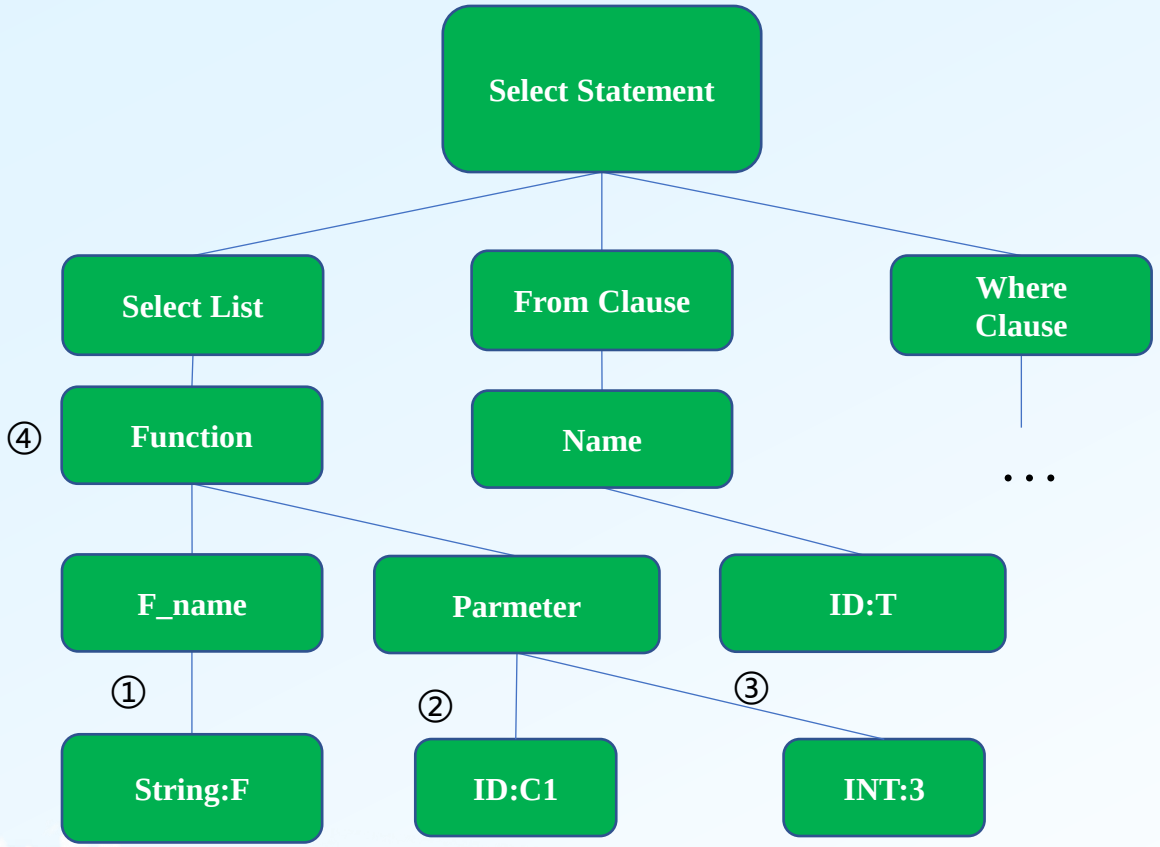


- **SELECT** v.cid, v.name, o.oid, p.product_name
- **FROM** vip_customers v, orders o, products p
- **WHERE** v.cid = o.cid and o.pid = p.pid and p.in_stock <= 0



语义检查示例

- 语义检查是一个自底向上不断交替进行**名称解析、类型解析、函数解析、类型匹配并判断正误**的过程。
- 以“SELECT F(c1,3) FROM T WHERE c2>10;”为例：



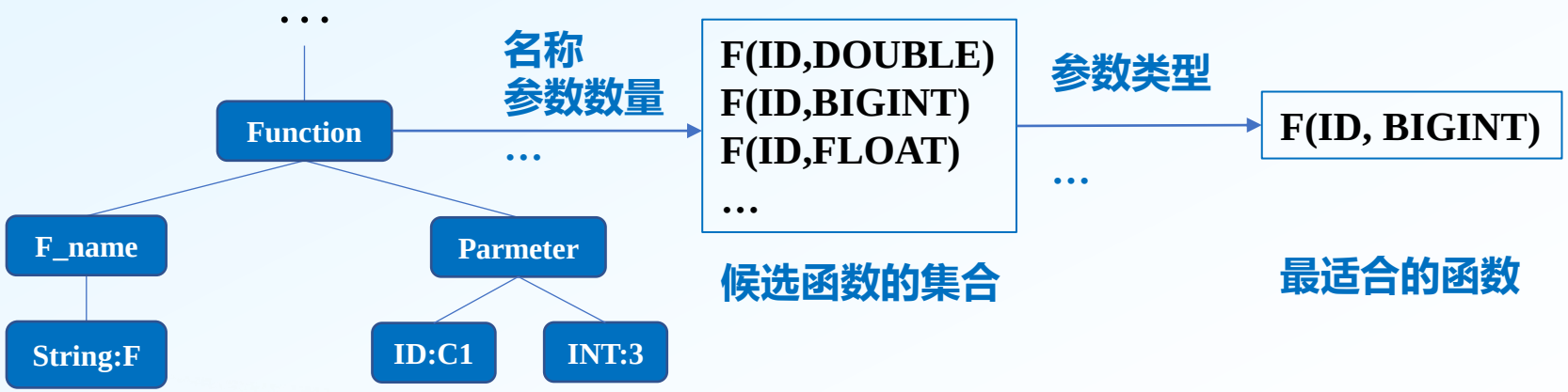
- **名称解析，**例如①函数名F和②列名c1；
- **类型解析，**例如确定②列c1的数据类型；
- **函数解析，**例如④函数F(c1,3)。

语义检查示例

- **名称解析**
  - 表名，列名，视图名，变量名，等等；
  - 函数名；
- **类型解析**
  - 识别特定对象（如列名）的数据类型；
- **函数解析**
  - 函数名解析
  - （参数）类型匹配
- **类型匹配**
  - 类型转换；
  - 类型提升。

函数解析

- 函数名的解析遵循与表名的解析相同的规则。函数解析包含以下两个步骤:
  - 确定候选函数的集合:
    - 函数的非限定名称;
    - 指定的参数数量;
    - 对函数实例的执行权限，即授权ID。
  - 从候选函数集中选择与该函数具有最佳类型匹配的函数:
    - 与函数调用中参数的数据类型比较;
    - 确定是否可以将输入参数隐式转换为所需类型;
    - 返回结果数据类型与调用函数的上下文是否兼容;
    - 当前路径中的模式顺序会影响函数的选择顺序。



类型匹配

- **类型匹配是函数解析的一部分，包括以下内容：**
  - **类型转换：由一种数据类型转换为另一种数据类型。**
    - 例如：从C字符串(string)转换到SQL整数类型(INTEGER)。
  - **类型提升：不同的数据类型中存在相容的数据类型组，可以将数据按顺序提升至下一种类型；相同类型的数据属性(长度)不同，可以通过类型提升具有相同的属性。**
    - 例如：SMALLINT->INTEGER->BIGINT->DECIMAL…
    - 例如：CHAR[8]->CHAR[10]。
  - **注：SQL中可以通过用户定义的函数进行强制类型转换。**
    - 例如：使用CAST函数将整数转换为布尔类型：CAST(0 AS BOOL)。

授权检查

- **授权ID；**
- **对象：表，视图，函数等；**
- **操作：SELECT, INSERT, DELETE, UPDATE, CREATE, DROP, ALTER, 等等；**
- **特权：是/否；**
- **高级功能：**
  - **受信任上下文；**
  - **细粒度的访问控制(行级权限、列屏蔽等)。**

授权ID，对象，操作

- CockroachDB会为执行的每条语句检查用户的权限。
- 如果user对一条语句没有足够的权限， CockroachDB将返回错误。



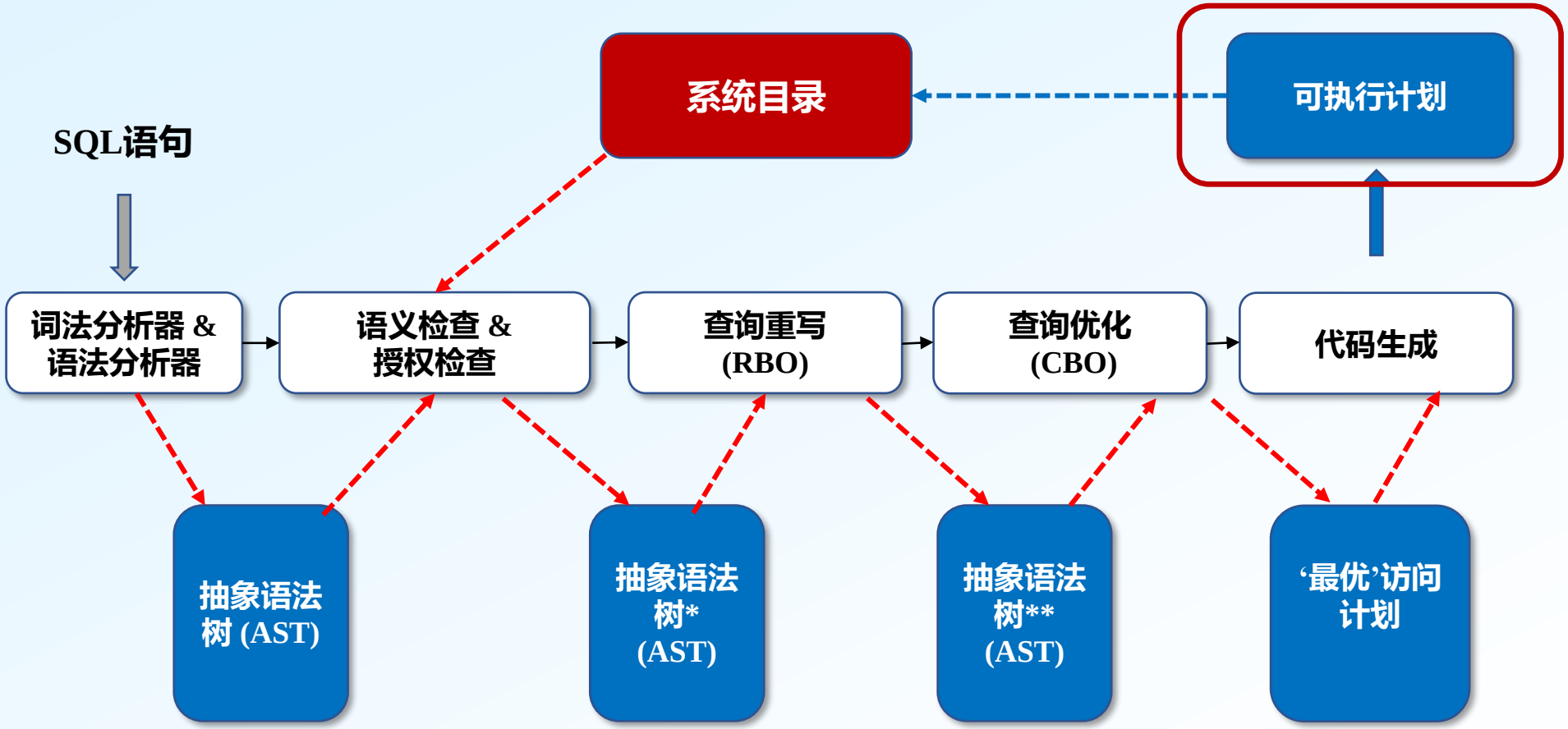

```
GRANT INSERT, DELETE, UPDATE, SELECT ON vehicles TO app_user;
```

```
database_name | schema_name | table_name | grantee | privilege_type
+---------------+-------------+------------+----------+----------------+
movr | public | vehicles | app_user | DELETE
movr | public | vehicles | app_user | INSERT
movr | public | vehicles | app_user | SELECT
movr | public | vehicles | app_user | UPDATE
(4 rows)
```

高级功能

- **在IBM DB2中还支持以下高级功能(CockroachDB暂不支持):**
  - **受信任上下文：为数据库和外部实体(如应用程序服务器)之间的连接定义信任关系。信任关系基于以下属性:**
    - 系统授权ID:表示建立数据库连接的用户;
    - IP地址(或域名):表示建立数据库连接的主机;
    - 数据流加密:表示数据库服务器和数据库客户机之间的数据通信的加密设置(如果有的话)。
  - **细粒度访问控制（FGAC）：在行级、列级或同时在行级和列级控制对表的访问。FGAC可以用来补充表特权模型。**

可执行计划



* 带语义信息的AST

** 表示查询重写后的AST

本章内容

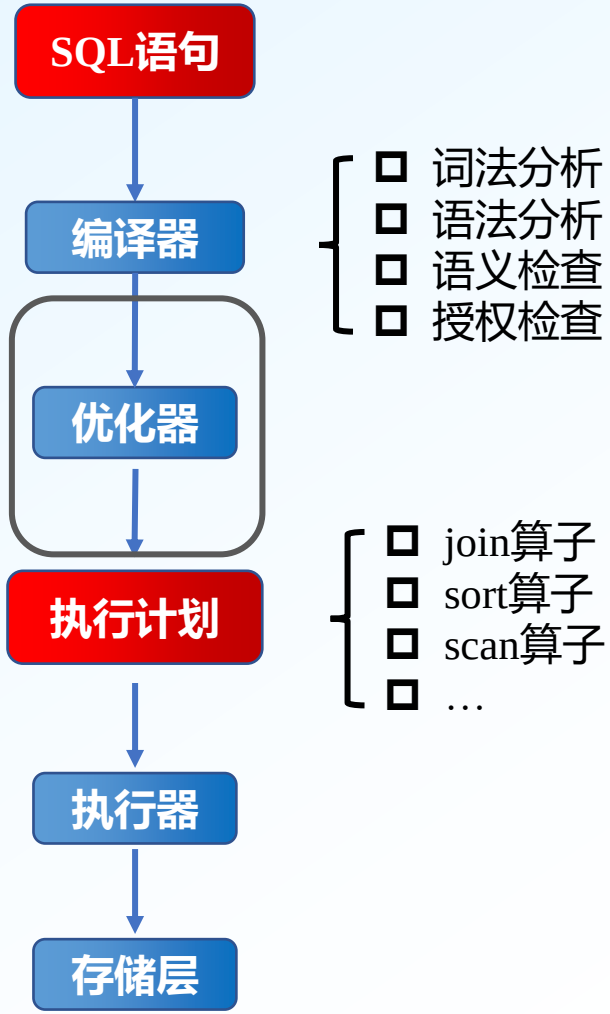


一、查询解析

二、逻辑重写

三、物理算子

四、查询优化



RBO & CBO

基于规则的优化（Rule-based Optimization, RBO）

- 查询的逻辑重写，以尽可能减少不合理的开销
- 基于（关系代数的等价变换）规则
- 调整操作顺序
- 生成：由逻辑运算符组成的逻辑计划（树）
  - 逻辑运算符：关系代数的算子（π，σ，⋈，...）
- 与关系实例无关

本节内容

基于代价的优化（Cost-based Optimization, CBO）

- 查询的物理优化
- 使用代价模型，对操作进行代价估计
- 衡量所有可能的执行方式，选择执行代价最小的
- 生成：由物理操作符组成的物理执行计划（树）
  - 物理操作符：

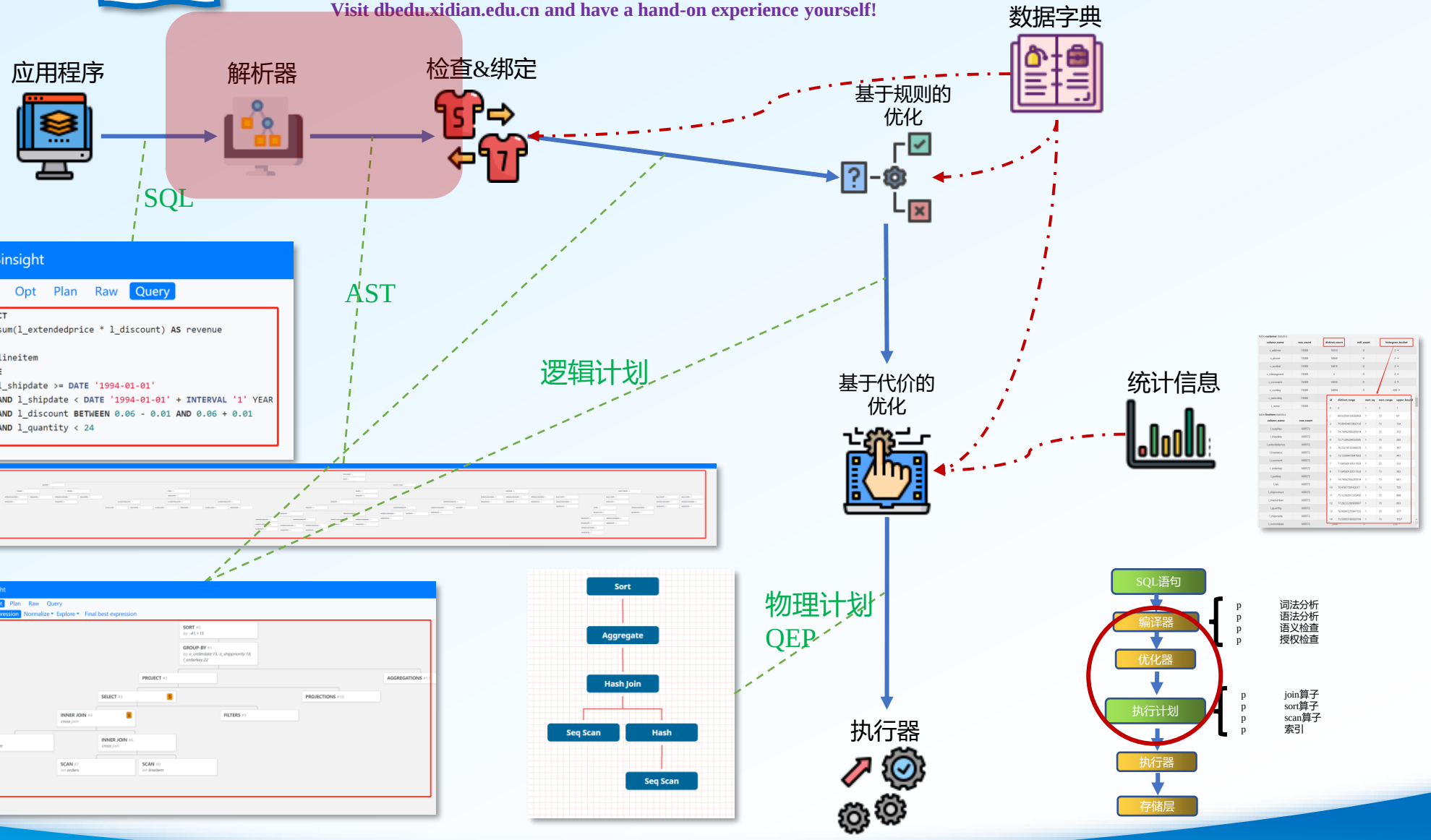
    关系代数算子的具体物理实现 （1 -> n映射）
- 与关系实例相关
  - 需要使用实例的统计信息

下节内容

SQL引擎pipeline

DBEDU

*All examples are provided by our DBEDU platform (Dbinsight & LANTERN subsystems)
Visit dbedu.xidian.edu.cn and have a hand-on experience yourself!

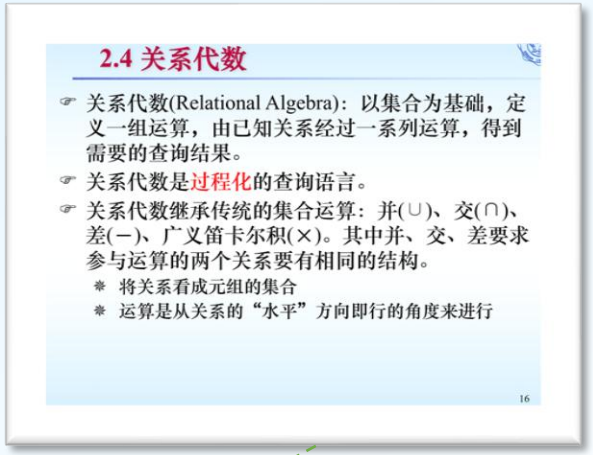


SQL -> 关系代数RA

- **SQL是一种声明式语言而不是过程式语言！**
  - **用户指定数据访问和计算的意图 - 是什么(what)；**
  - **不用指定访问数据的算法 - 不是怎么做(not how)；**

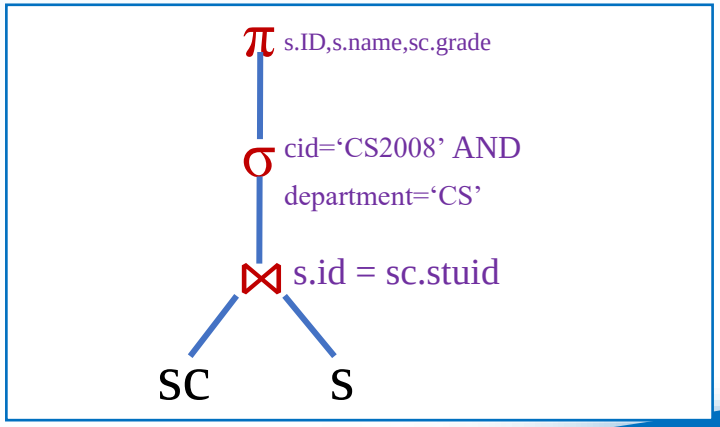







```
SELECT s.ID,s.name,sc.grade
from student s, studentcourse sc
where sc.cid= 'CS2008'
        and s.id = sc.stuid
        and s.department='CS';
```

$$\pi_{s.ID,s.name,sc.grade}(\sigma_{cid='CS2008' \wedge department='CS'}(SC \bowtie S))$$



关系代数的等价变换


```
SELECT B,D
FROM R,S
WHERE R.A = "c"  AND
S.E = 2 AND R.C=S.C
```

R

A	B	C
a	1	10
b	1	20
c	2	10
d	2	35
e	3	45

S

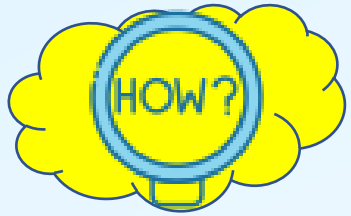
C	D	E
10	x	2
20	y	2
30	z	2
40	x	1
50	y	3

结果

B	D
2	x

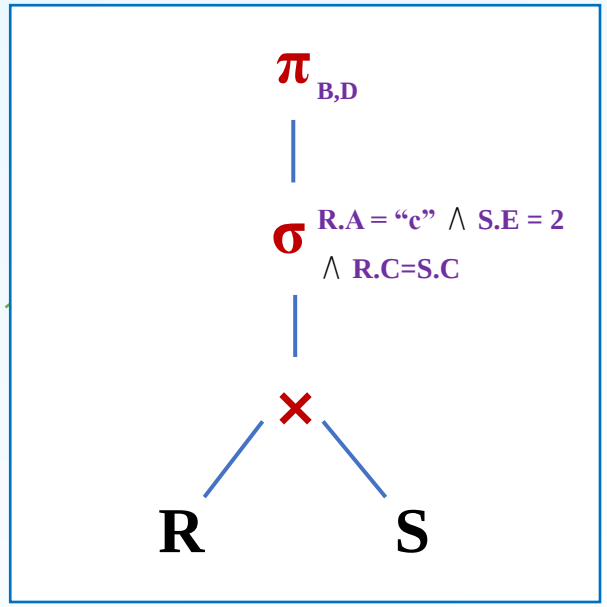
执行策略1





如何执行？？？

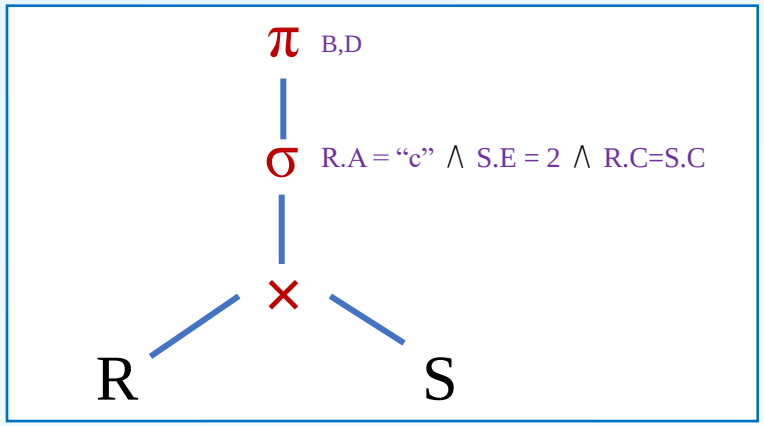
```
SELECT B,D
FROM R,S
WHERE R.A = “c” AND  S.E = 2 AND
R.C=S.C;
```



$$\pi_{B,D}[\sigma_{R.A=“c” \wedge S.E=2 \wedge R.C=S.C}(R \times S)]$$

执行策略1


```
SELECT B,D
FROM R,S
WHERE R.A = "c"  AND
S.E = 2 AND R.C=S.C
```

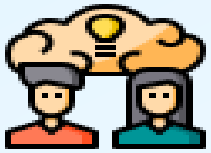


R × S

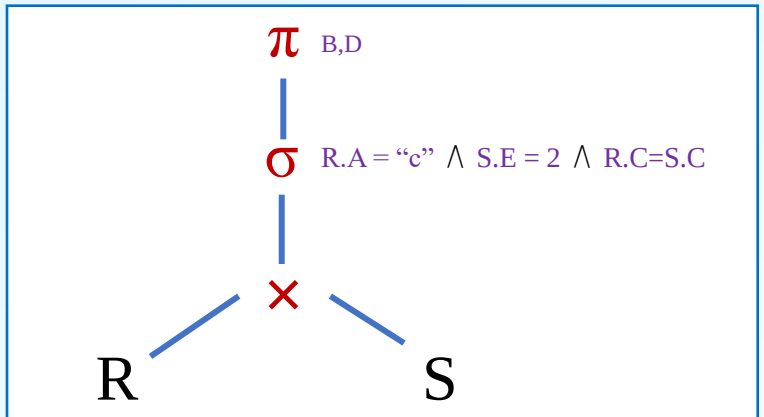
R.A	R.B	R.C	S.C	S.D	S.E
a	1	10	10	x	2
a	1	10	20	y	2
.					
.					
c	2	10	10	x	2
.					
.					

执行策略2


```
SELECT B,D
FROM R,S
WHERE R.A = "c"  AND
S.E = 2 AND R.C=S.C
```

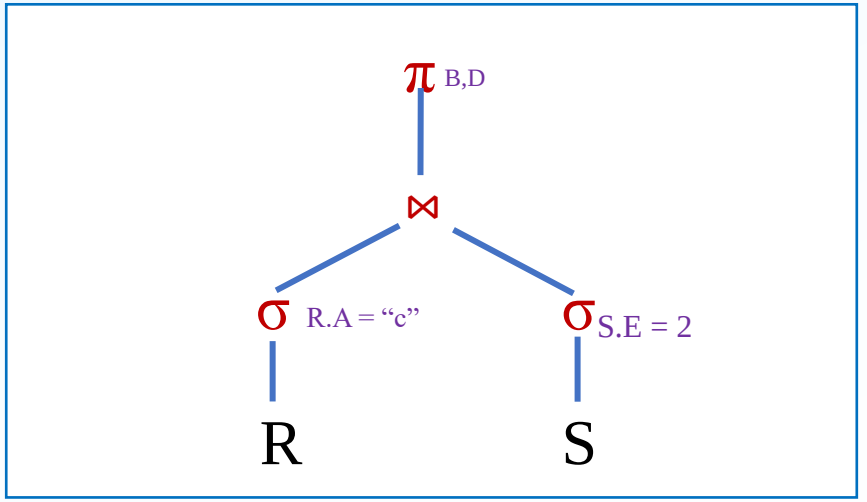


关系代数的等价变换



$\pi_{B,D}[\sigma_{R.A="c" \wedge S.E=2 \wedge R.C=S.C}(R \times S)]$

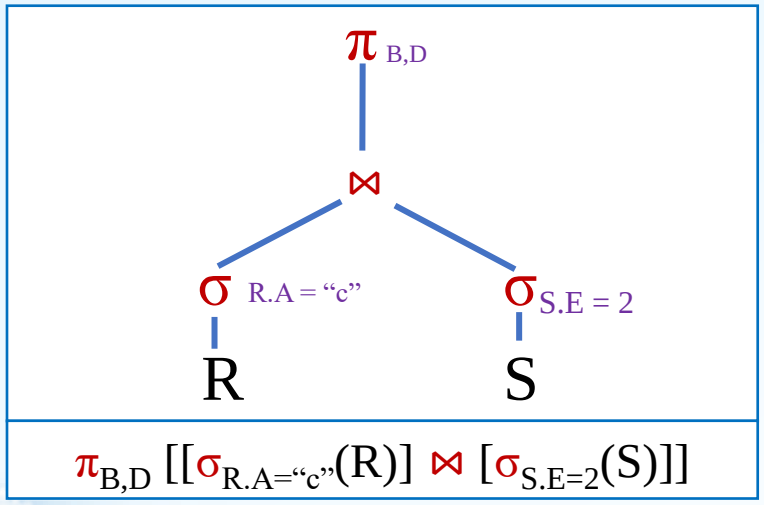




$\pi_{B,D}[[\sigma_{R.A="c"}(R)] \bowtie [\sigma_{S.E=2}(S)]]$

执行策略2


```
SELECT B,D
FROM R,S
WHERE R.A = "c"  AND
S.E = 2 AND R.C=S.C
```



$\pi_{B,D}\,[[\sigma_{R.A="c"}(R)] \bowtie [\sigma_{S.E=2}(S)]]$

R

A	B	C
a	1	10
b	1	20
c	2	10
d	2	35
e	3	45

s (R)

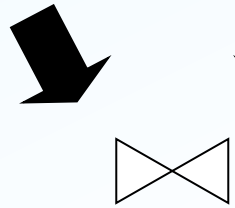
A	B	C
c	2	10

s(S)

C	D	E
10	x	2
20	y	2
30	z	2

S

C	D	E
10	x	2
20	y	2
30	z	2
40	x	1
50	y	3



关系代数的等价变换规则

- 策略1 -> 策略2
  - 等价变换
  - 操作数（涉及的行数）大幅下降
- 等价变换规则
  - 选择
    - 级联 $\sigma_{c1\wedge\ldots\wedge cn}(R) \equiv \sigma_{c1}(\ldots \sigma_{cn}(R))$
    - 交换律 $\sigma_{c1}(\sigma_{c2}(R)) \equiv \sigma_{c2}(\sigma_{c1}(R))$
  - 投影
    - 级联 $\pi_{a1}(R) \equiv \pi_{a1}(\ldots(\pi_{an}(R)))$
- 等价变换规则
  - 连接
    - 结合律 $R \bowtie (S \bowtie T) \equiv (R \bowtie S) \bowtie T$
    - 交换律 $(R \bowtie S) \equiv (S \bowtie R)$
  - 其他规则
    - 投影选择可交换（谓词条件仅涉及投影列）
    - 笛卡尔积->连接
    - 选择条件分配律
    - ...

基于规则的优化RBO

- **基于关系代数的等价变换规则**
  - 初始逻辑计划 -> （最优）逻辑计划
  - 提高最优（执行）计划被选择到的概率
  - 实例无关
- **RBO通用规则**
  - 复合谓词拆分
  - 谓词下推
  - 笛卡尔积->连接
  - 投影下推

复合谓词拆分

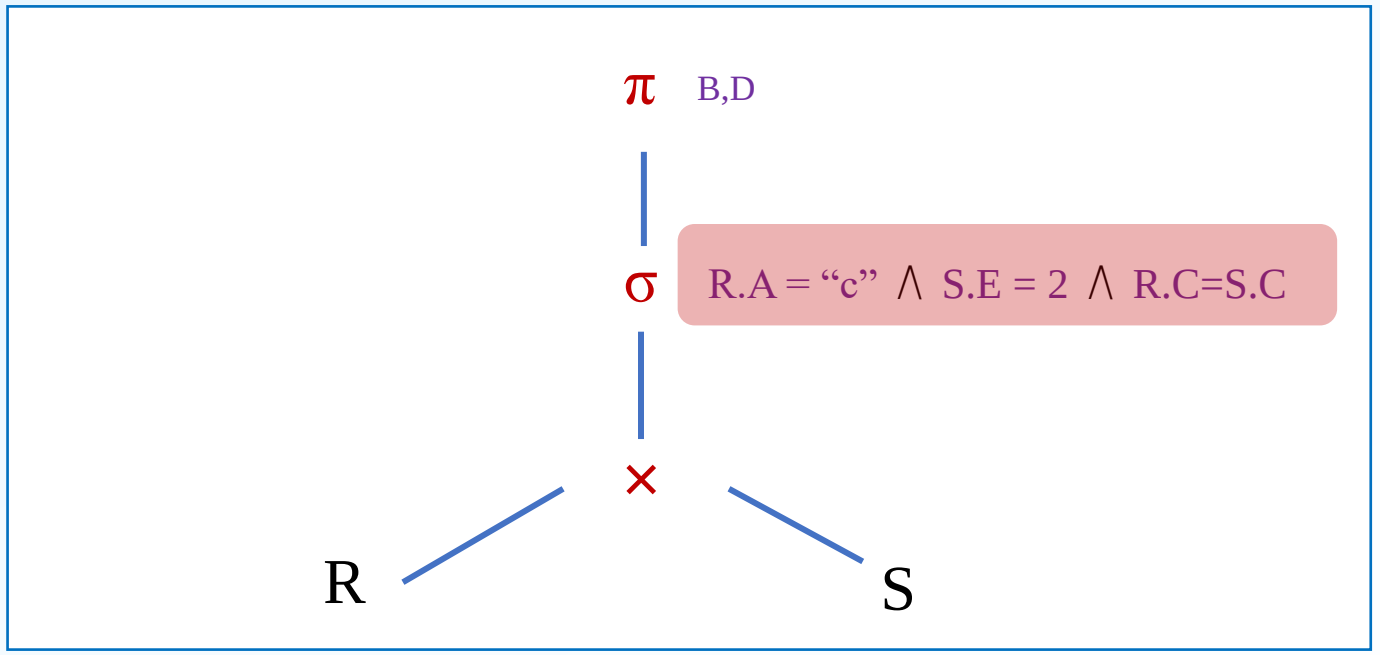
SELECT B,D

FROM R,S

WHERE R.A = “c”

AND S.E = 2

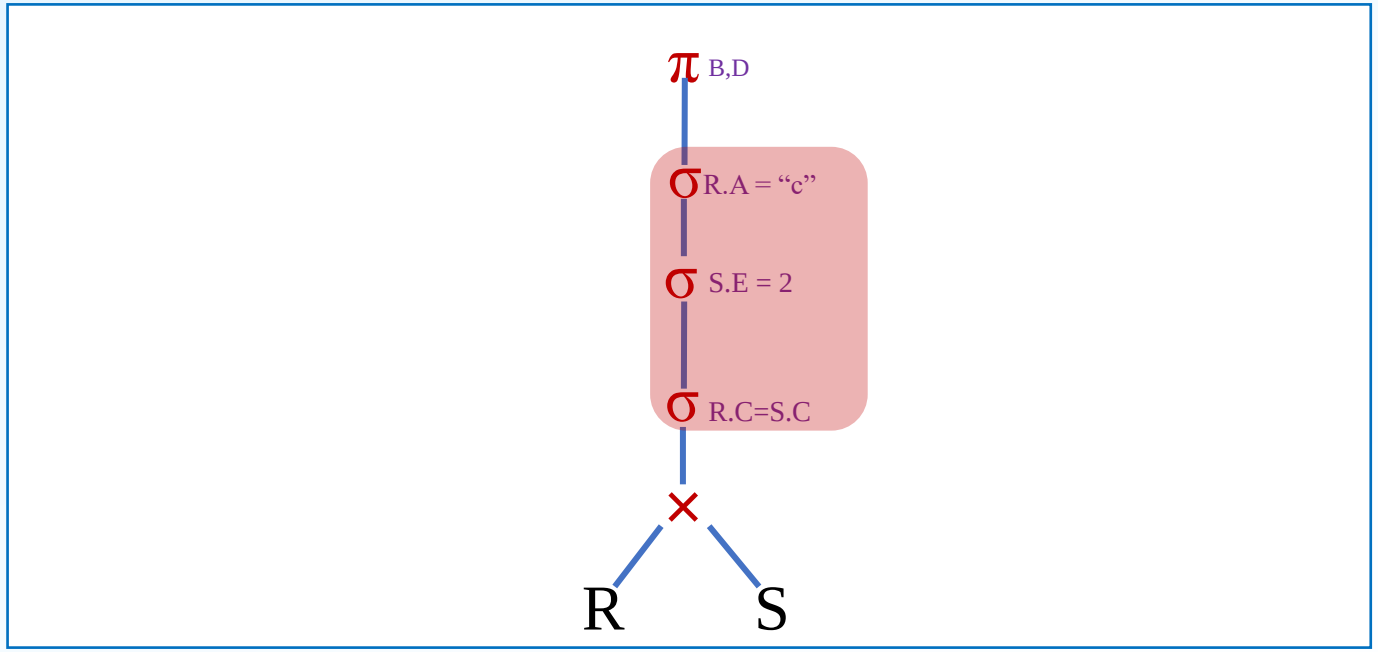
AND R.C=S.C



$$\pi_{B,D}[\sigma_{R.A=\text{“c”} \wedge S.E=2 \wedge R.C=S.C}(R \times S)]$$

复合谓词拆分

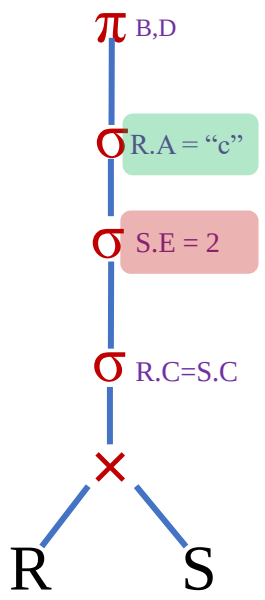

```
SELECT B,D
FROM R,S
WHERE R.A = “c”
AND   S.E = 2
AND R.C=S.C
```



$$\pi_{B,D}[\sigma_{R.A=\text{“c”}}(\sigma_{S.E=2}(\sigma_{R.C=S.C}(R\times S)))]$$

谓词下推


```
SELECT B,D
FROM R,S
WHERE R.A = “c”
AND  S.E = 2
AND R.C=S.C
```



$$\pi_{B,D}[\sigma_{R.A=“c”}(\sigma_{S.E=2}(\sigma_{R.C=S.C}(R\times S)))]$$

谓词下推

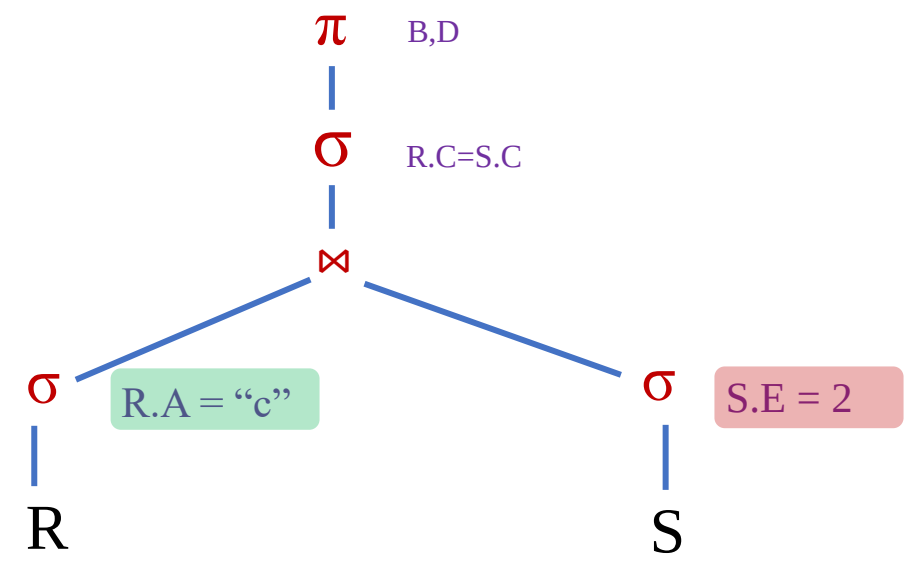
SELECT B,D

FROM R,S

WHERE R.A = “c”

AND S.E = 2

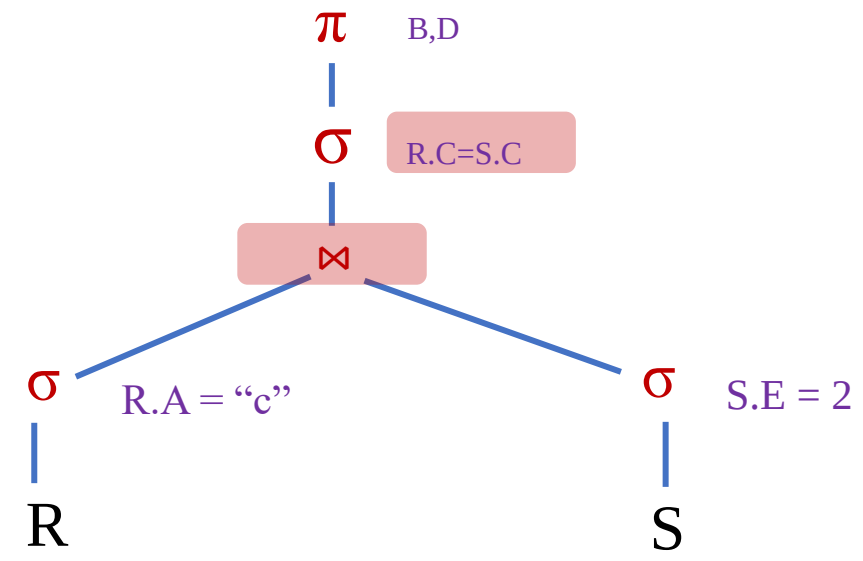
AND R.C=S.C



$$\pi_{B,D}[\sigma_{R.C = S.C}(\sigma_{R.A=“c”}(R) \times \sigma_{S.E=2}(S))]$$

笛卡尔积->连接

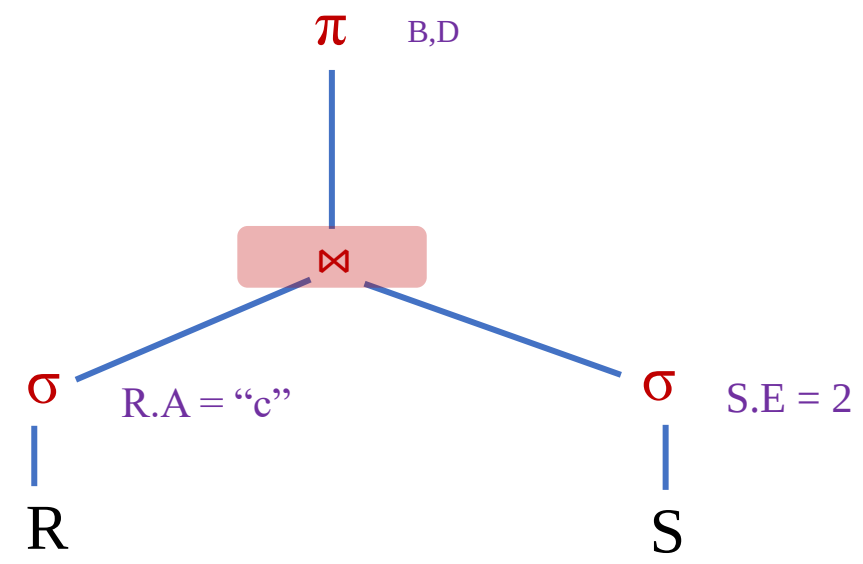

```
SELECT B,D
FROM R,S
WHERE R.A = “c”
AND   S.E = 2
AND R.C=S.C
```



$$\pi_{B,D}[\sigma_{R.C=S.C}(\sigma_{R.A=“c”}(R) \times \sigma_{S.E=2}(S))]$$

笛卡尔积->连接


```
SELECT B,D
FROM R,S
WHERE R.A = “c”
AND   S.E = 2
AND R.C=S.C
```



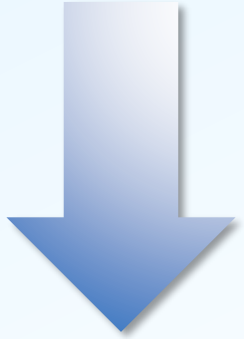
$$\pi_{B,D}\,[[\sigma_{R.A=\text{“c”}}(R)] \bowtie [\sigma_{S.E=2}(S)]]$$

其他的重写规则：嵌套查询

- 嵌套查询

```
SELECT R.B
FROM R
WHERE R.A = “c” AND EXISTS (SELECT * FROM S WHERE S.E = 2 AND  R.C=S.C);
```

- 类似视图的处理方式
  - 重写(为连接查询)【 ref., 视图消解】
  - 实体化临时表【 ref., 实体化视图】



```
SELECT R.B
FROM R,S
WHERE R.A = “c” AND  S.E = 2 AND  R.C=S.C;
```

其他的重写规则：嵌套查询

- **嵌套查询**

```
SELECT R.B
FROM R
WHERE R.A = "c" AND EXISTS ( SELECT * FROM S WHERE S.E = 2 AND  R.C=S.C );
```

- **类似视图的处理方式**
  - **重写(为连接查询)【 ref., 视图消解】**
  - **实体化临时表【 ref., 实体化视图】**

SELECT S.C FROM S WHERE S.E = 2;



```
SELECT R.B
FROM R,S
WHERE R.A = "c" AND  S.E = 2 AND  R.C=S.C;
```

其他的重写规则：表达式重写

- **查询语句中的表达式**
  - 永真（假）式预计算、连接移除、投影忽略…
  - 实例无关
  - 通过规则库匹配实现

SELECT R.B FROM R WHERE 1=1;

↓

SELECT R.B FROM R;

SELECT R1.B FROM R AS R1 JOIN R AS R2 ON R1.A=R2.A;

↓

SELECT R1.B FROM R;

SELECT B FROM R WHERE A BETWEEN 3 AND 20 OR A BETWEEN 10 AND 40;

↓

SELECT B FROM R WHERE A BETWEEN 3 AND 40;

SIA: Optimizing Queries using Learned Predicates. SIGMOD Conference 2021 [1]

- 尽可能将过滤下推->predicate的“无中生有”技术
  - SELECT * FROM A, B WHERE A.id = B.id AND A.val + 10 > B.val + 20 AND B.val + 10 > 20

↓

SELECT * FROM A, B WHERE A.id = B.id AND A.val + 10 > B.val + 20 AND B.val + 10 > 20 AND A.val>20;

[1] Qi Zhou, Joy Arulraj, Shamkant B. Navathe, William Harris, Jinpeng Wu: SIA: Optimizing Queries using Learned Predicates. SIGMOD Conference 2021: 2169-2181

小结

基于规则的优化（Rule-based Optimization, RBO）

- 查询的逻辑重写，以尽可能减少不合理的开销
- 基于（关系代数的等价变换）规则
- 调整操作顺序
- 生成：由逻辑运算符组成的逻辑计划（树）
  - 逻辑运算符：关系代数的算子（π，σ，⋈，...）
- 与关系实例无关

本节内容

基于代价的优化（Cost-based Optimization, CBO）

- 查询的物理优化
- 使用代价模型，对操作进行代价估计
- 衡量所有可能的执行方式，选择执行代价最小的
- 生成：由物理操作符组成的物理执行计划（树）
  - 物理操作符：

    关系代数算子的具体物理实现 （1 -> n映射）
- 与关系实例相关
  - 需要使用实例的统计信息

下节内容

本章内容

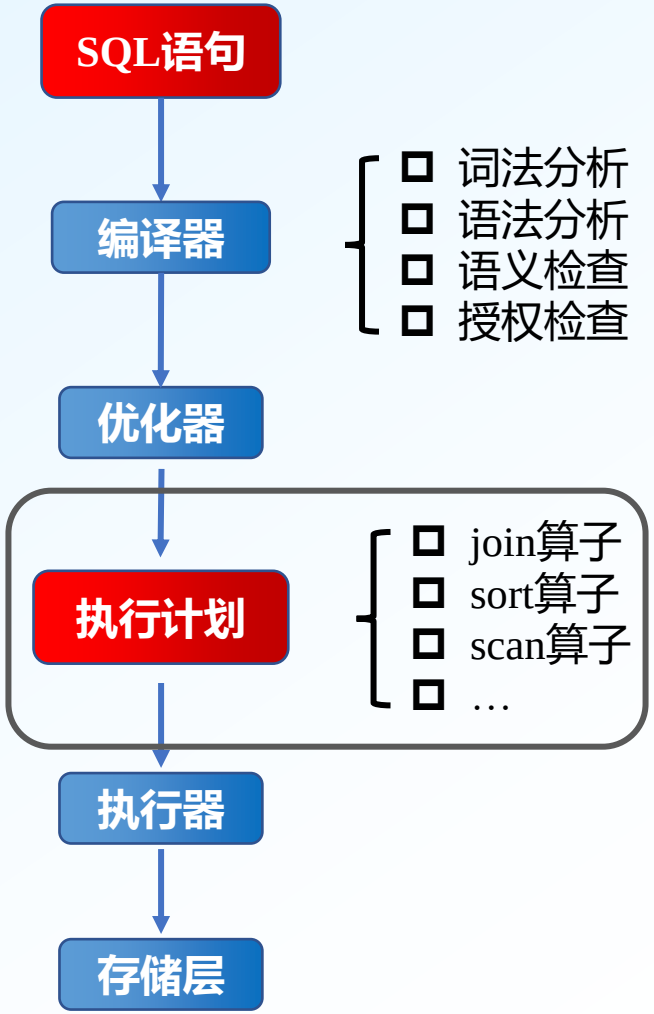


一、查询解析

二、逻辑重写

三、物理算子

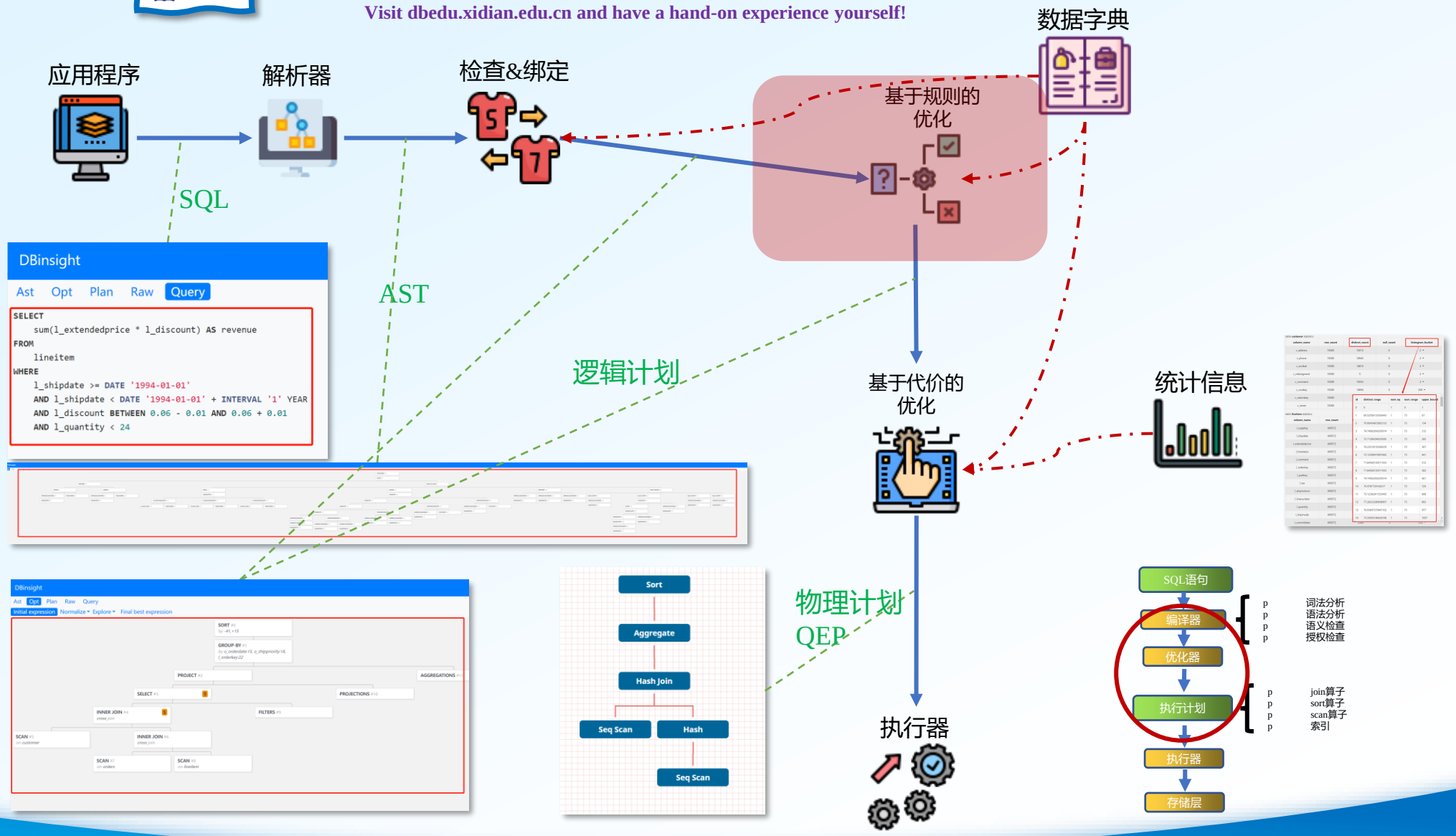
四、查询优化



SQL引擎pipeline

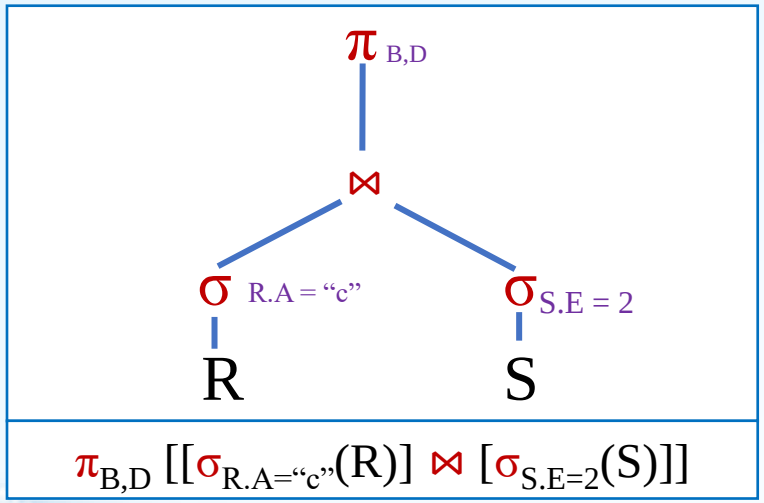
西安电子科技大学 XIDIAN UNIVERSITY

DBEDU *All examples are provided by our DBEDU platform (Dbinsight & LANTERN subsystems)
Visit dbedu.xidian.edu.cn and have a hand-on experience yourself!

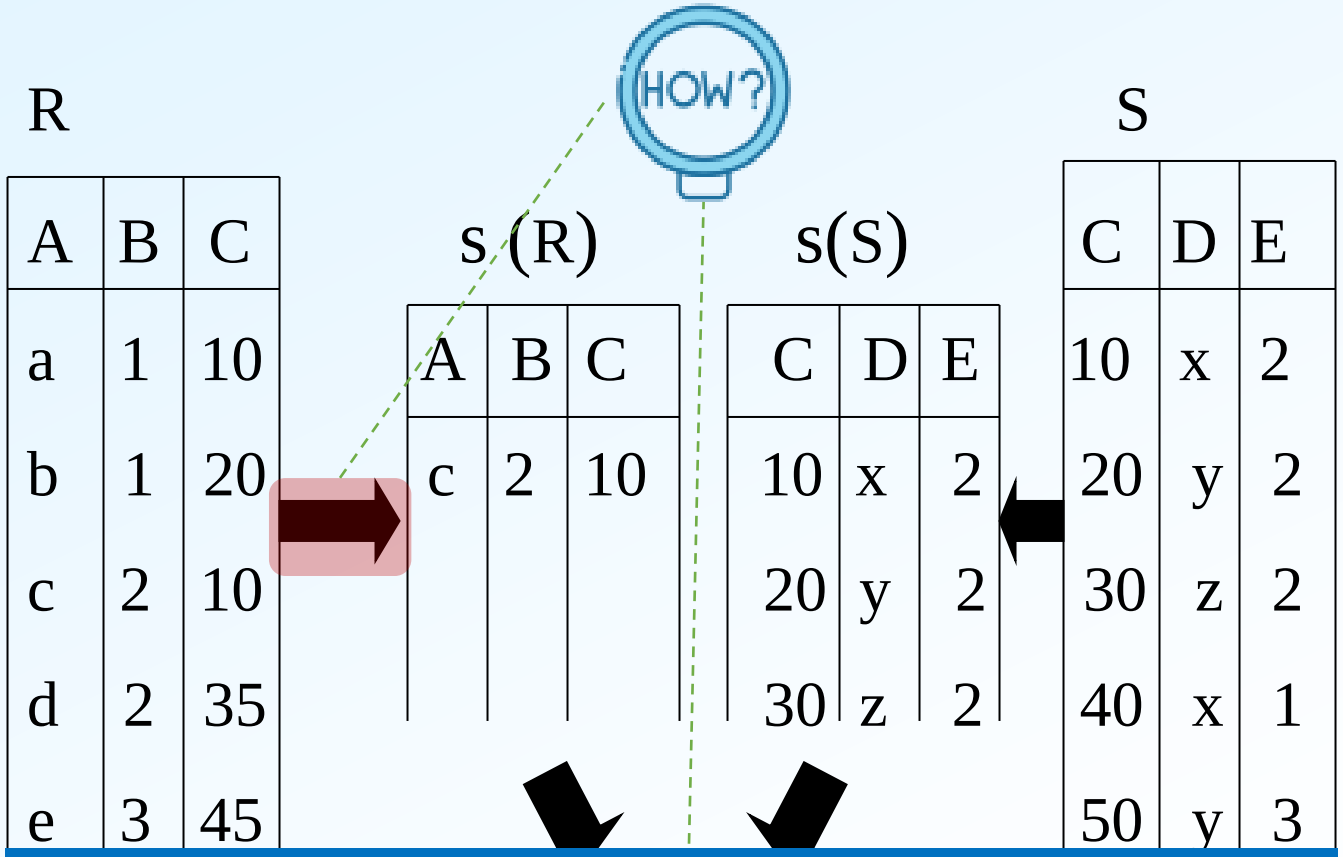


关系代数的局限性


```
SELECT B,D
FROM R,S
WHERE R.A = "c"  AND
S.E = 2 AND R.C=S.C
```



$\pi_{B,D}\,[[\sigma_{R.A=\text{“c”}}(R)] \bowtie [\sigma_{S.E=2}(S)]]$



逻辑重写（关系代数等价变换）作用

声明式的SQL - > 查询执行的逻辑顺序

Unanswered Question：每个逻辑算子如何实现?

RBO & CBO

基于规则的优化（Rule-based Optimization, RBO）

- 查询的逻辑重写，以尽可能减少不合理的开销
- 基于（关系代数的等价变换）规则
- 调整操作顺序
- 生成：由逻辑运算符组成的逻辑计划（树）
  - 逻辑运算符：关系代数的算子（π，σ，⋈，...）
- 与关系实例无关

上节内容

基于代价的优化（Cost-based Optimization, CBO）

- 查询的物理优化
- 使用代价模型，对操作进行代价估计
- 衡量所有可能的执行方式，选择执行代价最小的
- 生成：由物理操作符组成的物理执行计划（树）
  - 物理操作符：

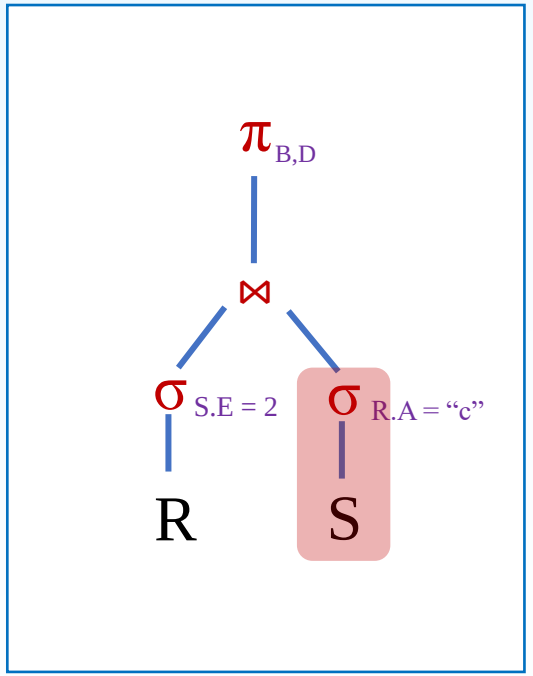
    关系代数算子的具体物理实现 （1 -> n映射）
- 与关系实例相关
  - 需要使用实例的统计信息

本节内容

关系运算符的物理实现

- **物理操作符**
  - 关系代数算子的具体物理实现
  - 1 -> n 映射
- **关系代数算子**
  - 选择σ
  - 投影π
  - 连接
    - 等值连接 ⋈
    - θ连接 ⋈θ
  - 排序τ
  - 聚集函数GγF(A) OR GGF(A)
  - 去重δ
  - 集合运算
  - …

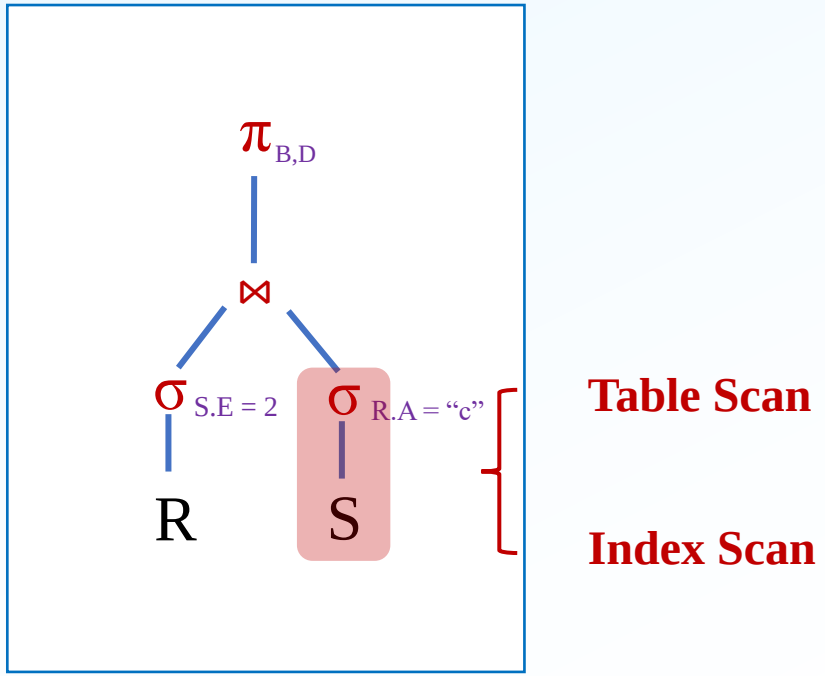




关系运算符的物理实现

- **物理操作符**
  - 关系代数算子的具体物理实现
  - 1 -> n 映射
- **关系代数算子**
  - 选择σ
  - 投影π
  - 连接
    - 等值连接 ⋈
    - θ连接 ⋈θ
  - 排序τ
  - 聚集函数GγF(A) OR GGF(A)
  - 去重δ
  - 集合运算
  - …

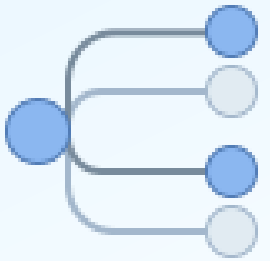




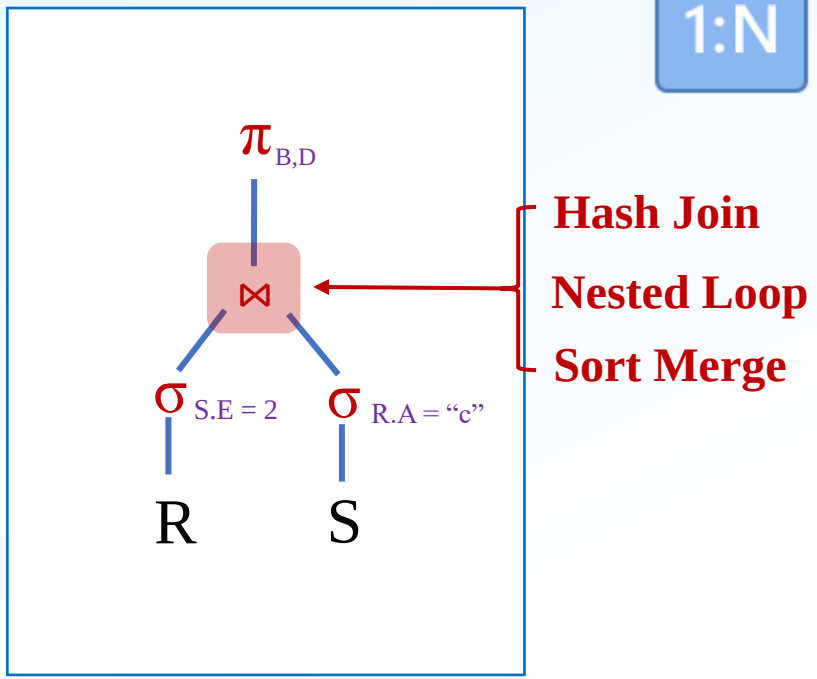
关系运算符的物理实现

- **物理操作符**
  - 关系代数算子的具体物理实现
  - 1 -> n 映射
- **关系代数算子**
  - 选择σ
  - 投影π
  - 连接
    - 等值连接 ⋈
    - θ连接 ⋈θ
  - 排序τ
  - 聚集函数GγF(A) OR GGF(A)
  - 去重δ
  - 集合运算
  - …



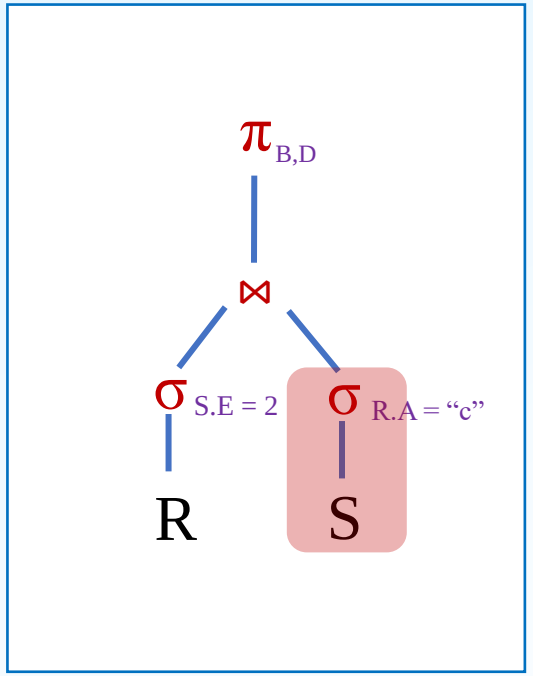


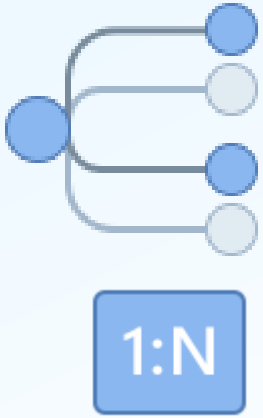
1:N



代价指标

- **物理操作符**
  - 关系代数算子的具体物理实现
  - 1 -> n 映射
    - 逻辑（关系）算子->物理操作符
    - 算法区别 -> 执行代价（cost）
- **代价指标**
  - 影响查询时间的因素
    - 磁盘访问　➢ CPU操作　➢ 网络通信
  - 简化讨论，本章以磁盘访问的次数（I/O）作为代价指标来比较不同的物理操作符
    - DBMS具体实现中会同时考察磁盘和CPU开销（参考: 课外阅读材料，西电智课）
    - 网络通信在分布式DBMS上不可忽略
    - 设B(R)为关系R的块（页）数，T(R)为关系R中的元组数





全表扫描（=谓词）

B(R)=2000，T(R)=100000，(80B/tuple)

- **全表扫描 Table Scan (a.k.a. Sequential Scan)**
  - 顺序扫描整个Table
  - 依次检查每个元组以衡量是否满足过滤谓词
  - B(R)（2000）次I/O操作
    - 当谓词是“=”且对应属性是码/唯一约束时平均B(R)/2
    - 命中即可结束
- **索引扫描 Index Scan**
  - 借助索引的扫描
    - 索引->Table
  - 过滤谓词必须是某个索引的搜索码

95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
…	…	…	…	…

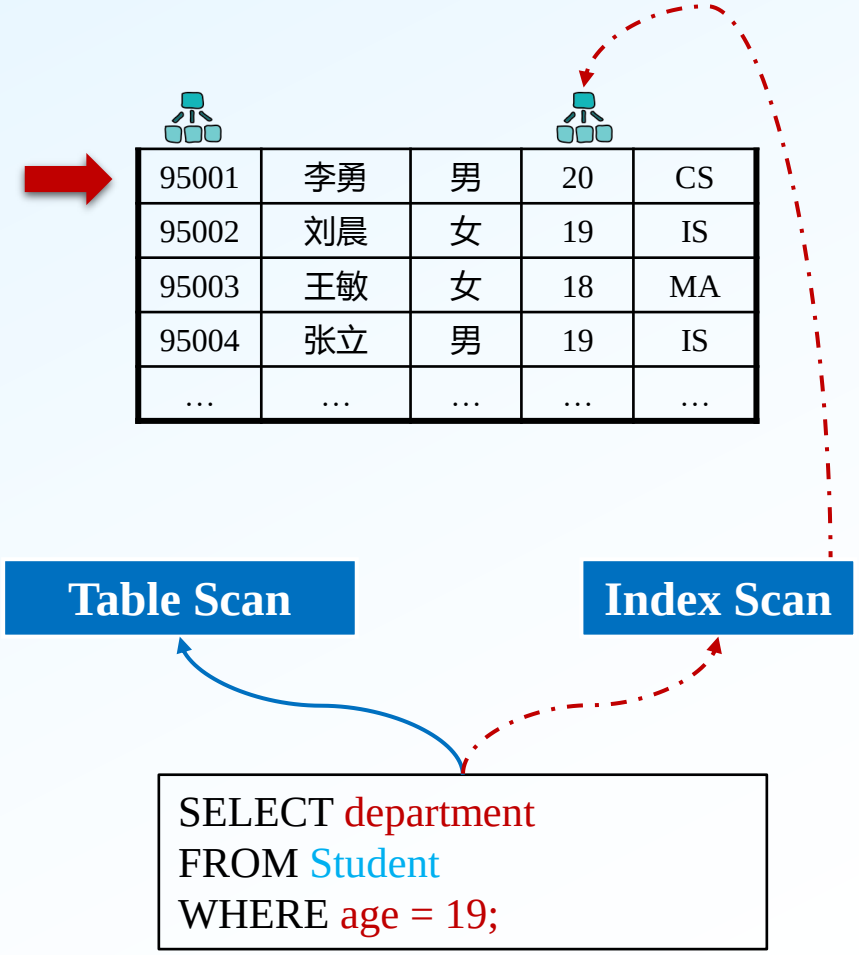
Table Scan

```
SELECT department
FROM Student
WHERE age = 19;
```

全表扫描（=谓词）

- **全表扫描 Table Scan (a.k.a. Sequential Scan)**
  - 顺序扫描整个Table
  - 依次检查每个元组以衡量是否满足过滤谓词
  - B(R)（2000）次I/O操作
    - 当谓词是“=”且对应属性是码/唯一约束时平均B(R)/2
    - 命中即可结束
- **索引扫描 Index Scan**
  - 借助索引的扫描
    - 索引->Table
  - 过滤谓词必须是某个索引的搜索码

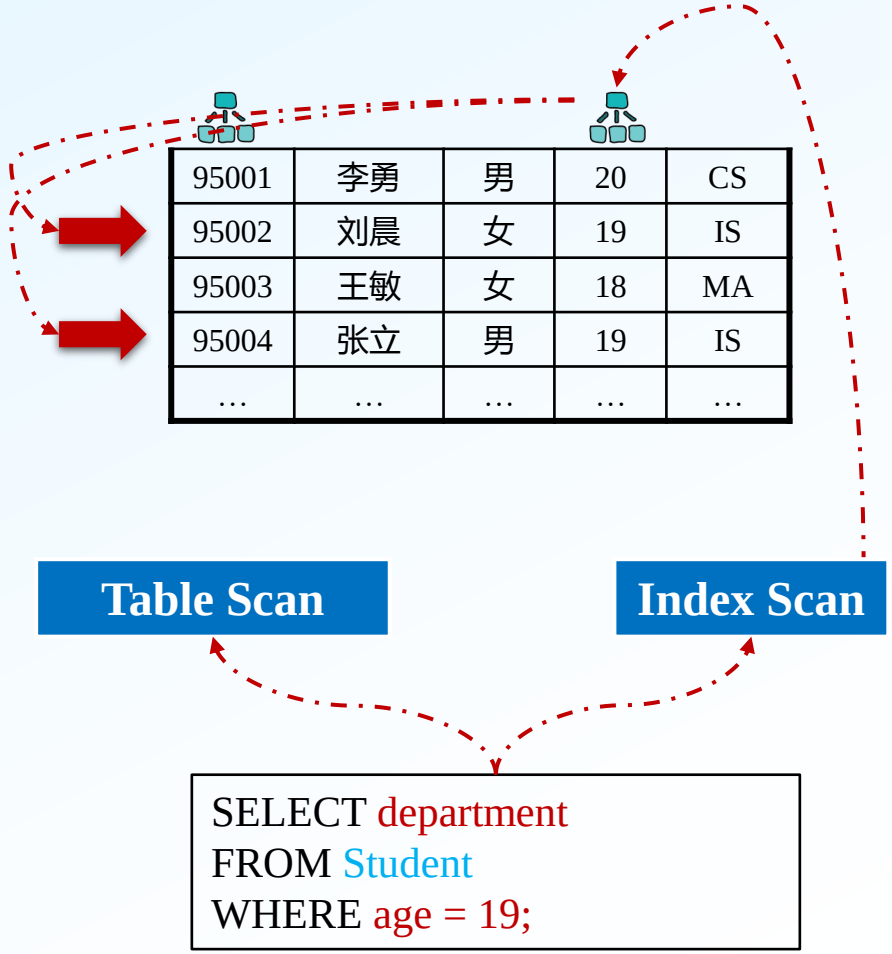
B(R)=2000，T(R)=100000，(80B/tuple)



全表扫描（=谓词）

- **全表扫描 Table Scan (a.k.a. Sequential Scan)**
  - 顺序扫描整个Table
  - 依次检查每个元组以衡量是否满足过滤谓词
  - B(R)（2000）次I/O操作
    - 当谓词是“=”且对应属性是码/唯一约束时平均B(R)/2
    - 命中即可结束
- **索引扫描 Index Scan**
  - 借助索引的扫描（索引->Table）
    - 从根页面开始，自上而下遍历索引
    - 找到第一个叶子页面
    - 标识页面中起始的索引键
    - 从左到右扫描索引
    - 对于每个索引键，使用记录的ID访问相应的表记录
    - 按顺序跳到下一个索引键并重复此过程
  - 过滤谓词必须是某个索引的搜索码

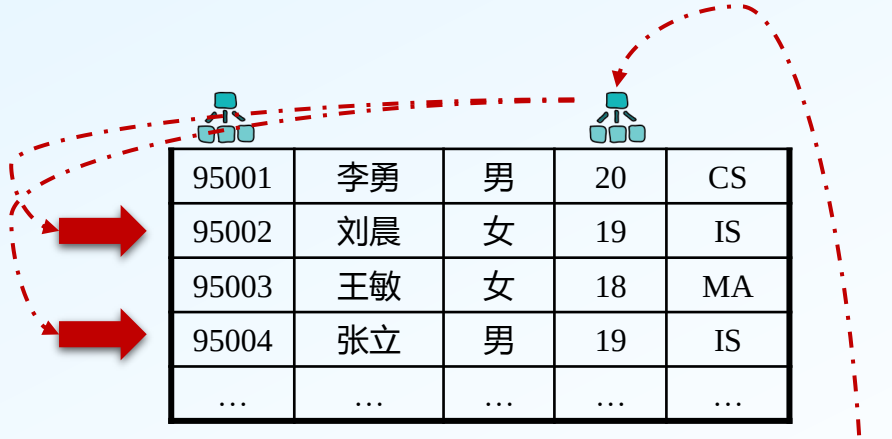
B(R)=2000，T(R)=100000，(80B/tuple)



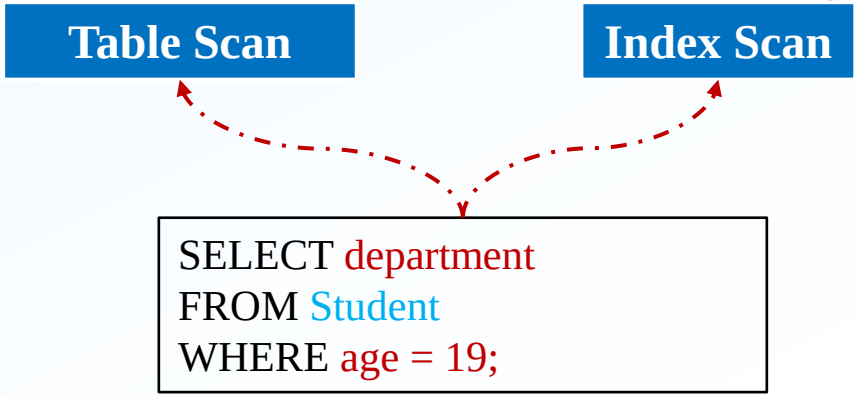
索引扫描（=谓词）

B(R)=2000，T(R)=100000，(80B/tuple)

- **全表扫描 Table Scan (a.k.a. Sequential Scan)**
  - 顺序扫描整个Table
  - 依次检查每个元组以衡量是否满足过滤谓词
  - B(R)（2000）次I/O操作
    - 当谓词是“=”且对应属性是码/唯一约束时平均B(R)/2
    - 命中即可结束
- **索引扫描 Index Scan**
  - 借助索引的扫描（索引->Table）
  - 过滤谓词必须是某个索引的搜索码
  - I/O操作数
    - 当谓词是主码/唯一约束时h+1
    - 否则，h+b
      - h: B+树深
      - b: 匹配的元组对应的块数（匹配的m个元组可能分布于m’块）



95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
…	…	…	…	…

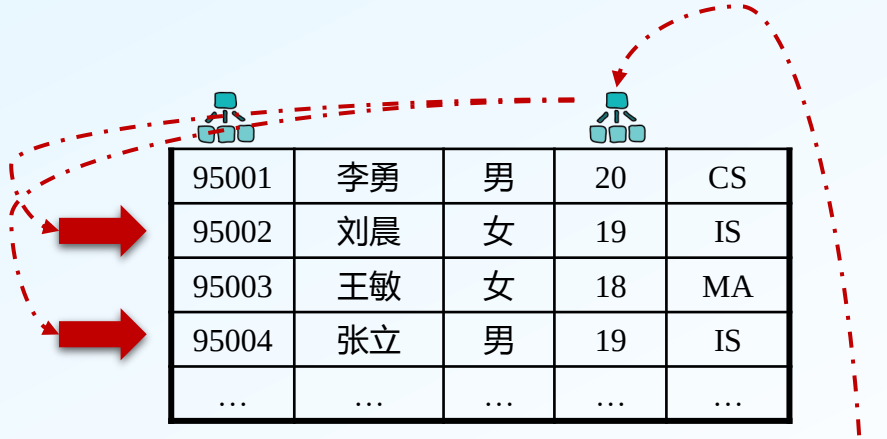


索引扫描（=谓词）

B(R)=2000，T(R)=100000，(80B/tuple)

- **全表扫描 Table Scan (a.k.a. Sequential Scan)**
  - 顺序扫描整个Table
  - 依次检查每个元组以衡量是否满足过滤谓词
  - B(R)（2000）次I/O操作
    - 当谓词是“=”且对应属性是码/唯一约束时平均B(R)/2
    - 命中即可结束
- **索引扫描 Index Scan**
  - 借助索引的扫描（索引->Table），设树深为h
  - 过滤谓词必须是某个索引的搜索码
  - I/O操作数？？？





95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
…	…	…	…	…

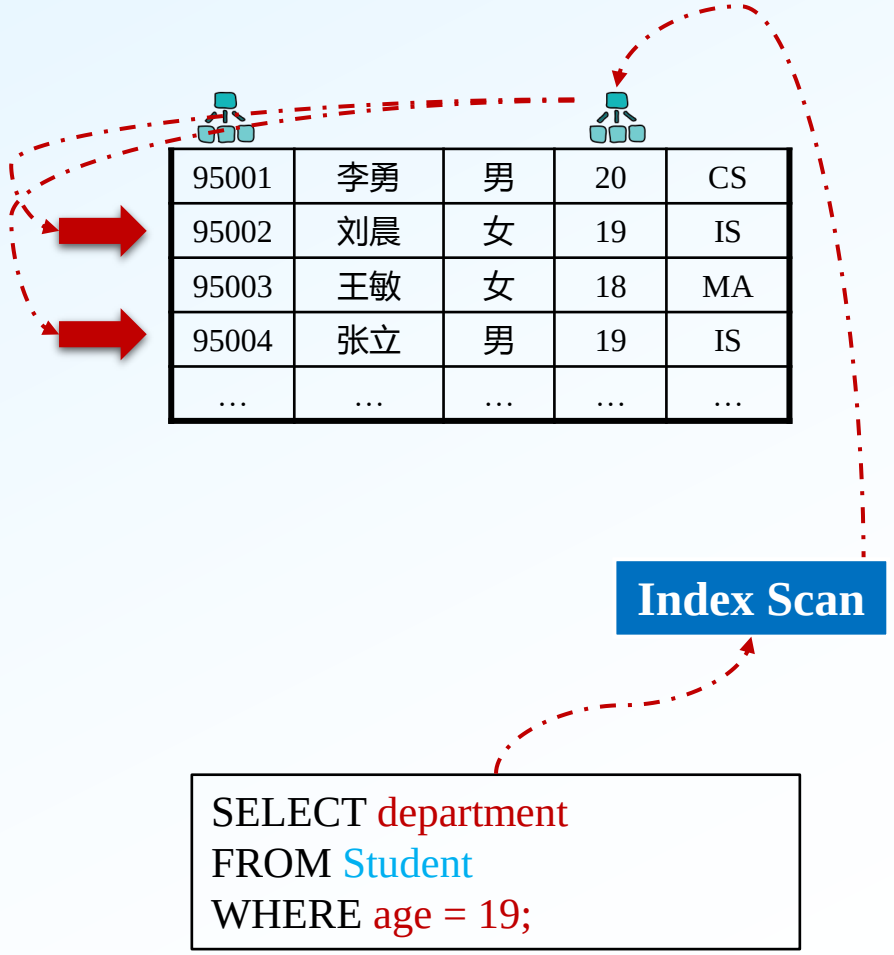
Table Scan　　Index Scan

```
SELECT department
FROM Student
WHERE age = 19;
```

索引扫描（=谓词）

- **索引扫描 Index Scan**
  - 过滤谓词必须是某个索引的搜索码
    - 聚簇索引
      - 元组按搜索码顺序存储在连续的块中
    - 非聚簇索引

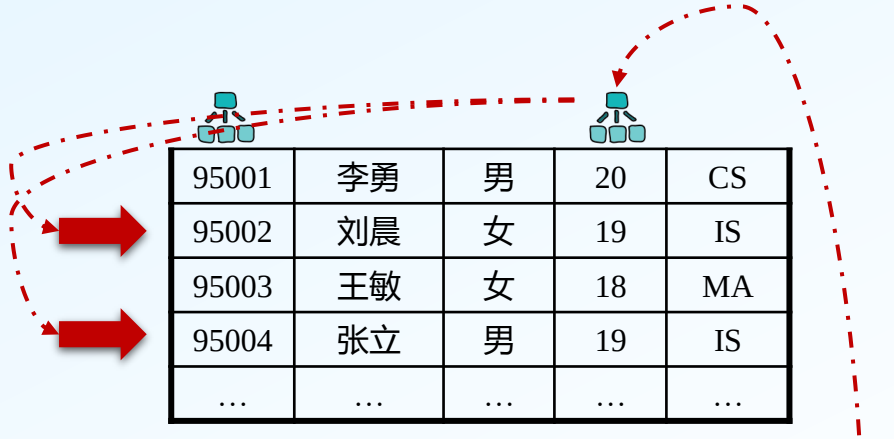
B(R)=2000，T(R)=100000，(80B/tuple)



索引扫描（=谓词、聚簇索引）

- **索引扫描 Index Scan**
  - 过滤谓词，聚簇索引
    - 当搜索码是码/唯一时（命中一个即结束）
      - h+1次I/O
    - 当搜索码不是码（不唯一）
      - 会命中多条记录
      - 由于是（面向当前搜索码的）聚簇索引，多条记录存储于连续块
      - h+b次I/O (b为该记录分布到的块数)
        - b能否计算（估计）？

B(R)=2000，T(R)=100000，(80B/tuple)



95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
…	…	…	…	…

Index Scan

```
SELECT department
FROM Student
WHERE age = 19;
```

索引扫描（=谓词、聚簇索引、非码）

- **h+b次I/O (b为该记录分布到的块数)**
  - **b能否计算（估计）？**
  - $b/B(R) = T(\sigma_{age=19}(R))/T(R)$

B(R)=2000，T(R)=100000，(80B/tuple，50tuple/block)

95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
…	…	…	…	…

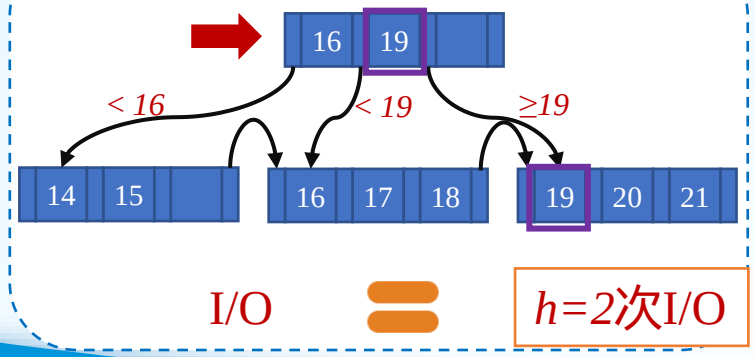
聚簇索引

```
SELECT department
FROM Student
WHERE age = 19;
```

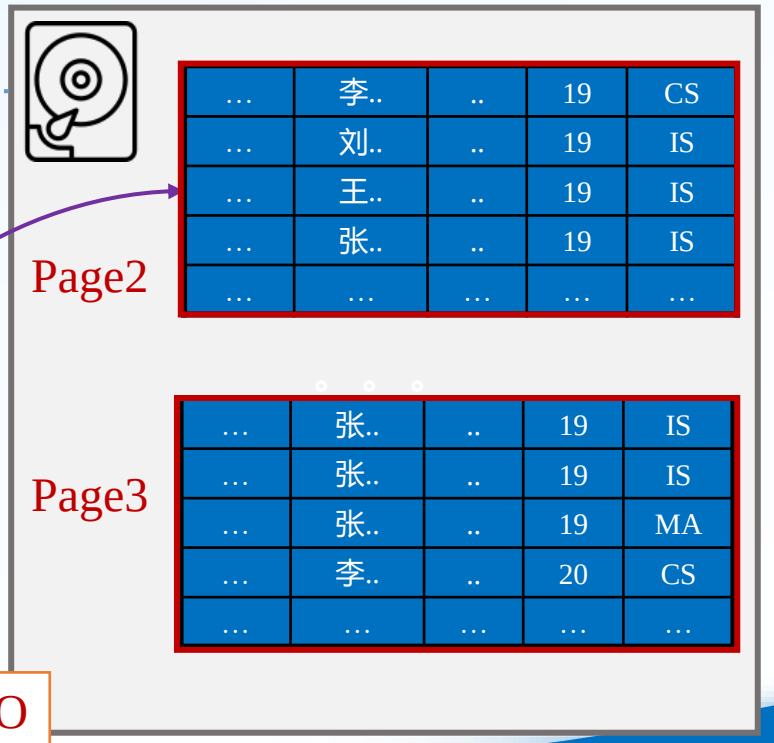
Index Scan



age



I/O = *h=2次I/O*



…	李..	..	19	CS
…	刘..	..	19	IS
…	王..	..	19	IS
…	张..	..	19	IS
…	…	…	…	…

…	张..	..	19	IS
…	张..	..	19	IS
…	张..	..	19	MA
…	李..	..	20	CS
…	…	…	…	…

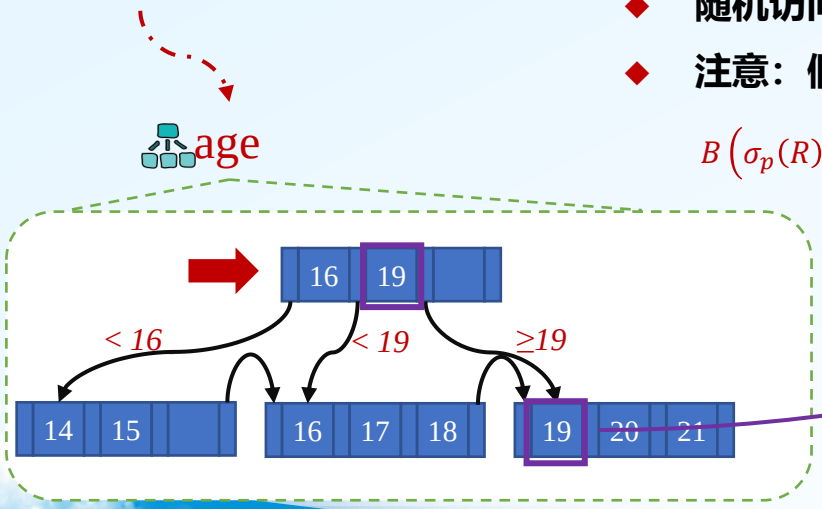
+ *b=2次I/O*

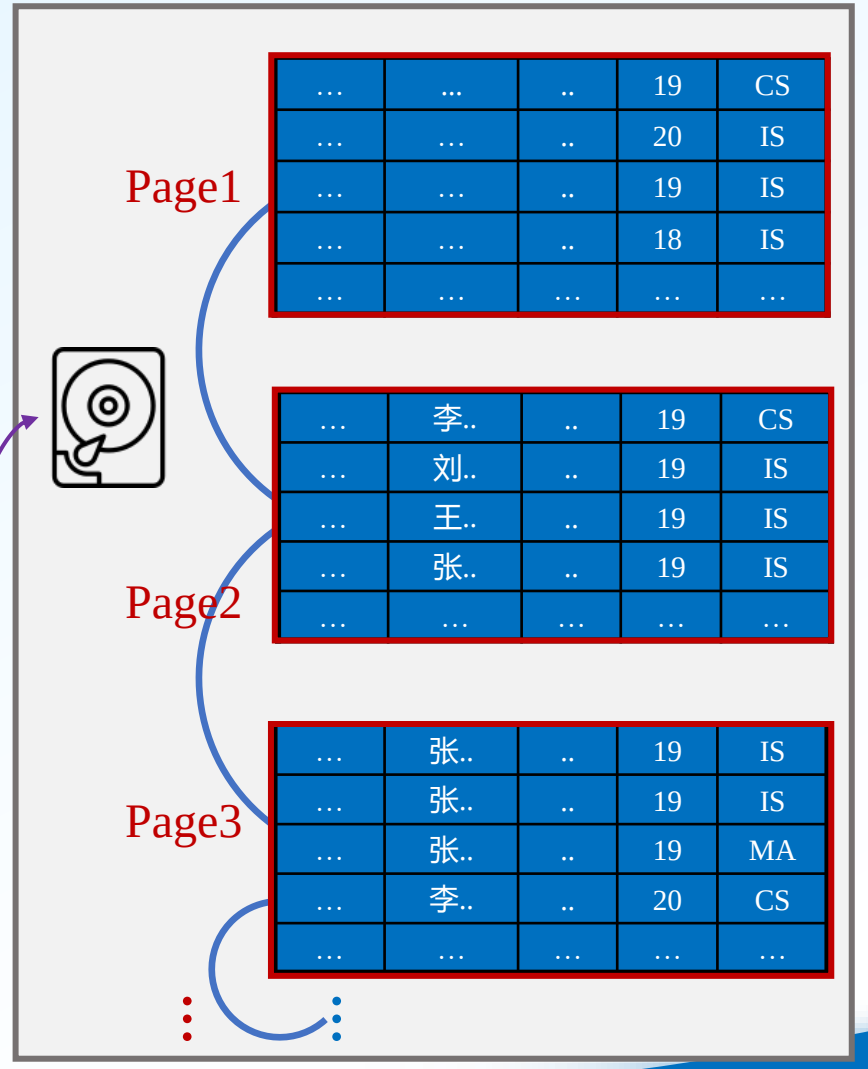
索引扫描（=谓词、聚簇索引）

- **索引扫描 Index Scan**
  - **过滤谓词必须是某个索引的搜索码**
    - **聚簇索引** $b = B(R) \times T(\sigma_{age=19}(R))/T(R)$
    - **非聚簇索引?**
      - **搜索码为码时（h+1次I/O）**
      - **区别仅在于（搜索码非码时）**
        - **定位到的记录分布于相对随机的**

          $b' = B\left(\sigma_p(R)\right)$**个磁盘块**
        - **随机访问 *b'* 块 vs. 顺序访问 *b* 块**
        - **注意：假设搜索码在各页面均匀随机分布，**

          $B\left(\sigma_p(R)\right) = \min\left\{T\left(\sigma_p(R)\right), B(R)\right\}$





索引扫描（比较谓词、聚簇索引）

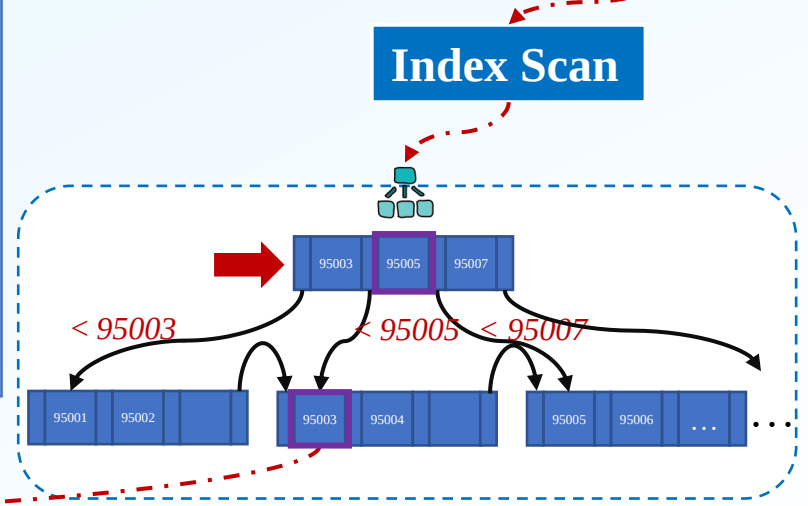
B(R)=2000，T(R)=100000，(80B/tuple，50tuple/block)

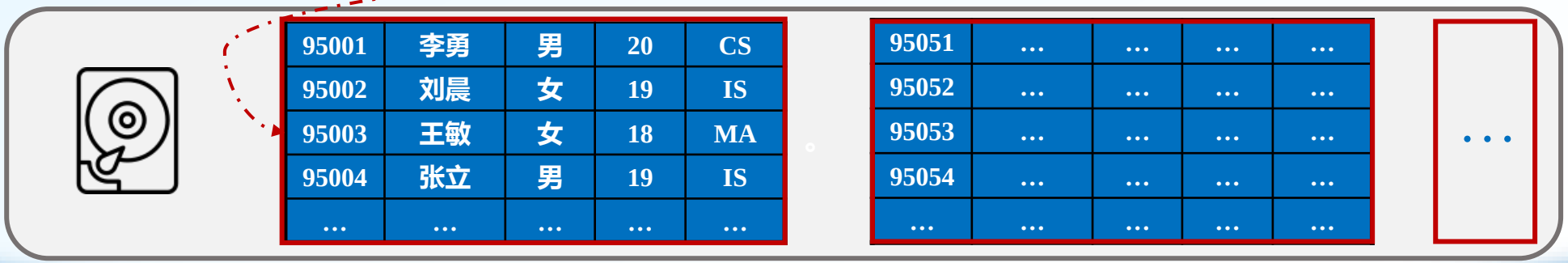
索引扫描 Index Scan

- 前面讨论的过滤谓词都是“=”
- 如果是比较谓词“<, >, ≤, ≥”？
  - 对于Table Scan仍然是B(R)（2000）次I/O操作
  - 对于Index Scan
    - 聚簇索引
      - ≥v：先定位到v对应的叶子(h次)，然后从其指向的数据块顺序扫描所有剩余块(b块)
      - 设满足条件的记录数为：$T(\sigma_{A\geq v}(R))$
      - 则 $b/B(R)= T(\sigma_{A\geq v}(R))/T(R)$
      - 合计I/O次数：$h+b=h+B(R)*T(\sigma_{A\geq v}(R))/T(R)$

谓词A≥v的选择率(Selectivity)

```
SELECT department
FROM Student
WHERE Stuid ≥ 95003;
```





95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
…	…	…	…	…

95051	…	…	…	…
95052	…	…	…	…
95053	…	…	…	…
95054	…	…	…	…
…	…	…	…	…

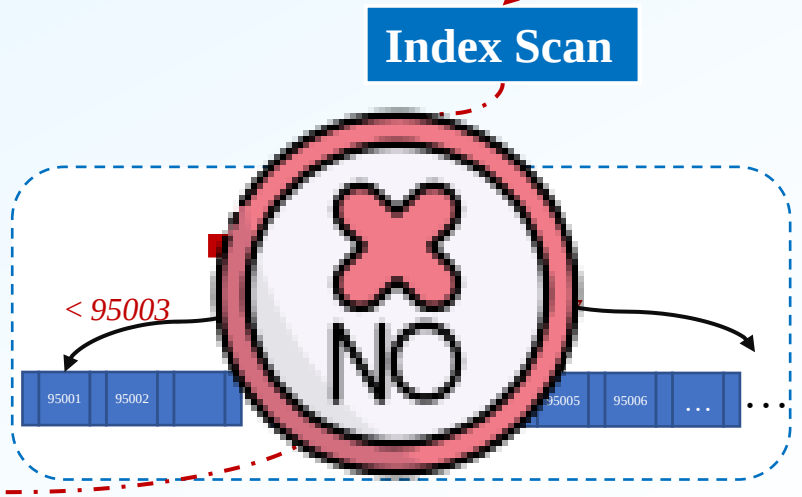
…

索引扫描（比较谓词、聚簇索引）

B(R)=2000，T(R)=100000，(80B/tuple，50tuple/block)

- **索引扫描 Index Scan**
  - 前面讨论的过滤谓词都是“=”
  - 如果是比较谓词“<, >, ≤, ≥”？
    - 对于Table Scan仍然是B(R)（2000）次I/O操作
    - 对于Index Scan
      - 聚簇索引
        - <v：先定位到v对应的叶子(*h*次)，然后从其指向的数据块顺序向前扫描所有剩余块？？？

```
SELECT department
FROM Student
WHERE Stuid < 95003;
```





95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
…	…	…	…	…

95051	…	…	…	…
95052	…	…	…	…
95053	…	…	…	…
95054	…	…	…	…
…	…	…	…	…

…

索引扫描（比较谓词、聚簇索引）

B(R)=2000，T(R)=100000，(80B/tuple，50tuple/block)

- **索引扫描 Index Scan**
  - 前面讨论的过滤谓词都是“=”
  - 如果是比较谓词“<, >, ≤, ≥”？
    - 对于Table Scan仍然是B(R)（2000）次I/O操作
    - 对于Index Scan
      - 聚簇索引
        - <v：直接顺序扫描数据块，直到v（不使用索引）
        - I/O次数：$B(R)*T(\sigma_{A<v}(R))/T(R)$

```
SELECT department
FROM Student
WHERE Stuid < 95003;
```



95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
…	…	…	…	…

95051	…	…	…	…
95052	…	…	…	…
95053	…	…	…	…
95054	…	…	…	…
…	…	…	…	…

…

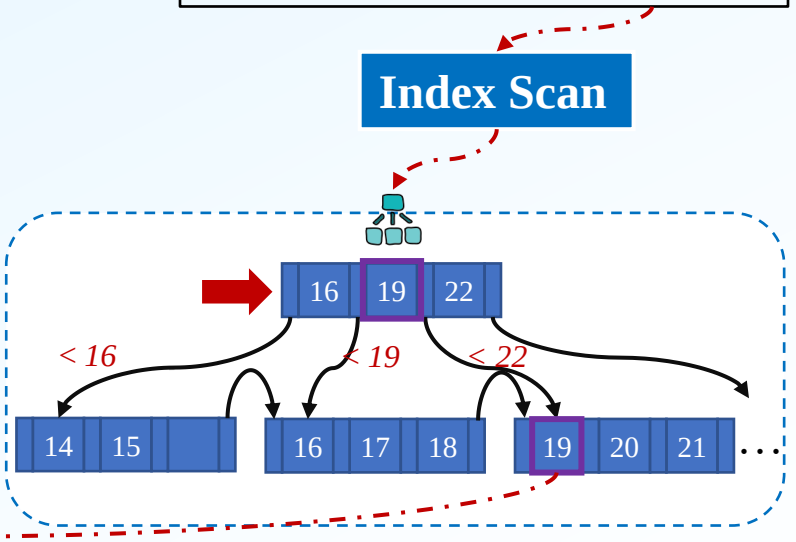
索引扫描（比较谓词、非聚簇索引）

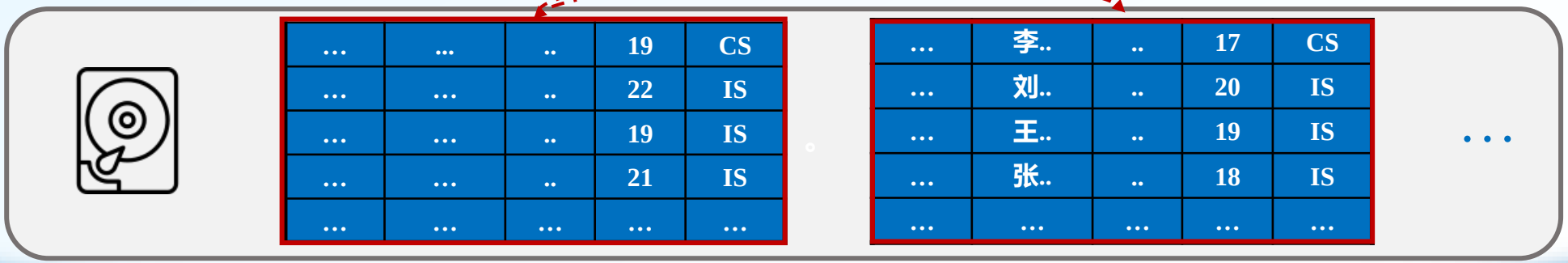
B(R)=2000，T(R)=100000，(80B/tuple，50tuple/block)

索引扫描 Index Scan

- 前面讨论的过滤谓词都是“=”
- 如果是比较谓词“<, >, ≤, ≥”？
  - 对于Table Scan仍然是B(R)（2000）次I/O操作
  - 对于Index Scan
    - 非聚簇索引
      - ≥v：先定位到v对应的叶子(h次)，然后顺序扫描叶子提取所有指向记录的指针。
      - 根据获取的指针读取对应数据块（随机I/O）

```
SELECT department
FROM Student
WHERE Age ≥ 19;
```





…	…	..	19	CS
…	…	..	22	IS
…	…	..	19	IS
…	…	..	21	IS
…	…	…	…	…

…	李..	..	17	CS
…	刘..	..	20	IS
…	王..	..	19	IS
…	张..	..	18	IS
…	…	…	…	…

…

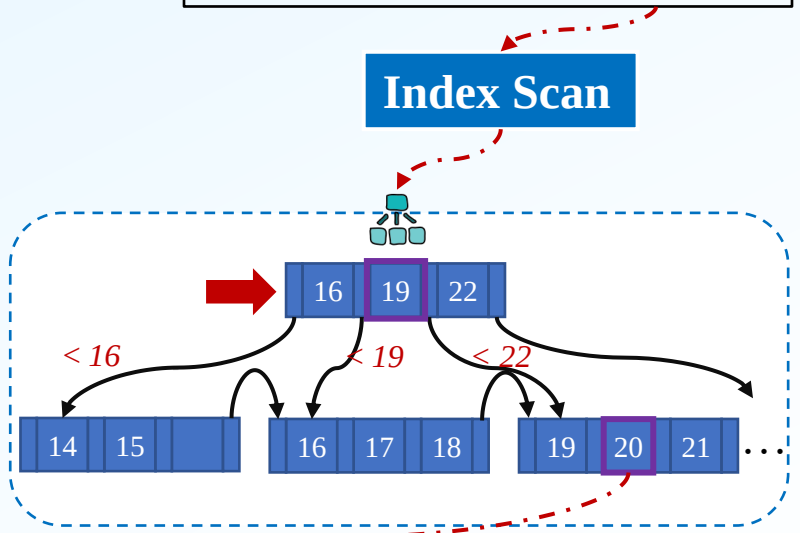
索引扫描（比较谓词、非聚簇索引）

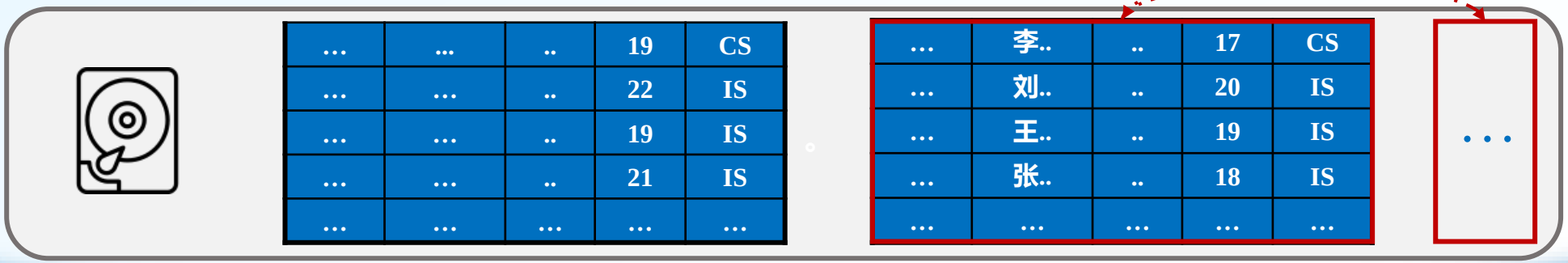
B(R)=2000，T(R)=100000，(80B/tuple，50tuple/block)

索引扫描 Index Scan

- 前面讨论的过滤谓词都是“=”
- 如果是比较谓词“<, >, ≤, ≥”？
  - 对于Table Scan仍然是B(R)（2000）次I/O操作
  - 对于Index Scan
    - 非聚簇索引
      - ≥v：先定位到v对应的叶子(h次)，然后顺序扫描叶子提取所有指向记录的指针。
      - 根据获取的指针读取对应数据块（随机I/O）

```
SELECT department
FROM Student
WHERE Age ≥ 19;
```





…	…	..	19	CS
…	…	..	22	IS
…	…	..	19	IS
…	…	..	21	IS
…	…	…	…	…

…	李..	..	17	CS
…	刘..	..	20	IS
…	王..	..	19	IS
…	张..	..	18	IS
…	…	…	…	…

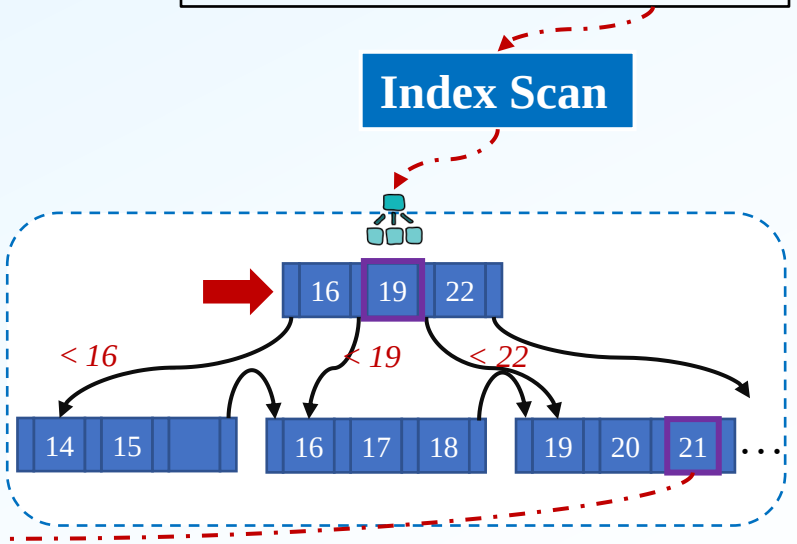
索引扫描（比较谓词、非聚簇索引）

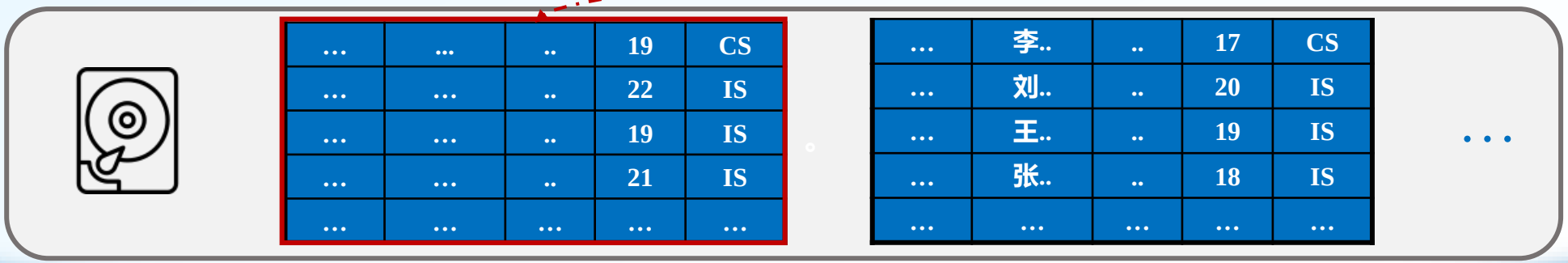
B(R)=2000，T(R)=100000，(80B/tuple，50tuple/block)

索引扫描 Index Scan

- 前面讨论的过滤谓词都是“=”
- 如果是比较谓词“<, >, ≤, ≥”？
  - 对于Table Scan仍然是B(R)（2000）次I/O操作
  - 对于Index Scan
    - 非聚簇索引
      - ≥v：先定位到v对应的叶子(h次)，然后顺序扫描叶子提取所有指向记录的指针。
      - 根据获取的指针读取对应数据块（随机I/O）
        - $T(\sigma_{A\geq v}(R))$次！！！
      - 合计I/O：$h+\gamma+T(\sigma_{A\geq v}(R))$

```
SELECT department
FROM Student
WHERE Age ≥ 19;
```





…	…	..	19	CS
…	…	..	22	IS
…	…	..	19	IS
…	…	..	21	IS
…	…	…	…	…

…	李..	..	17	CS
…	刘..	..	20	IS
…	王..	..	19	IS
…	张..	..	18	IS
…	…	…	…	…

…

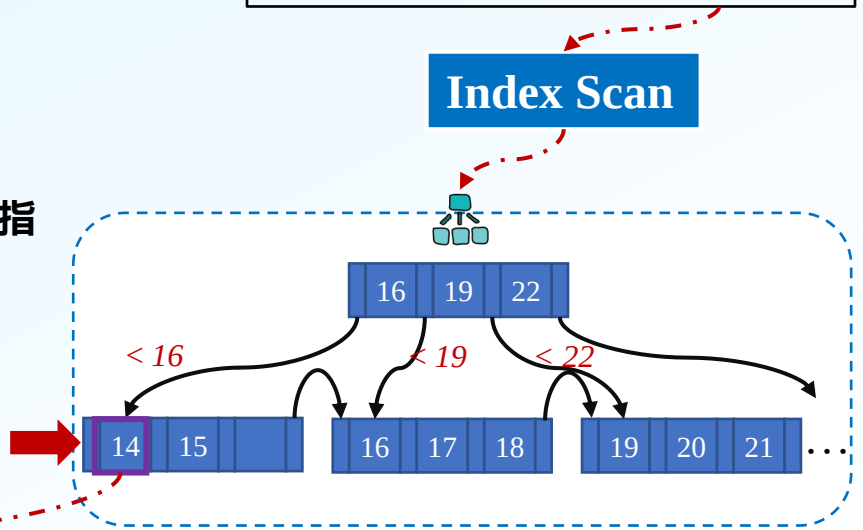
索引扫描（比较谓词、非聚簇索引）

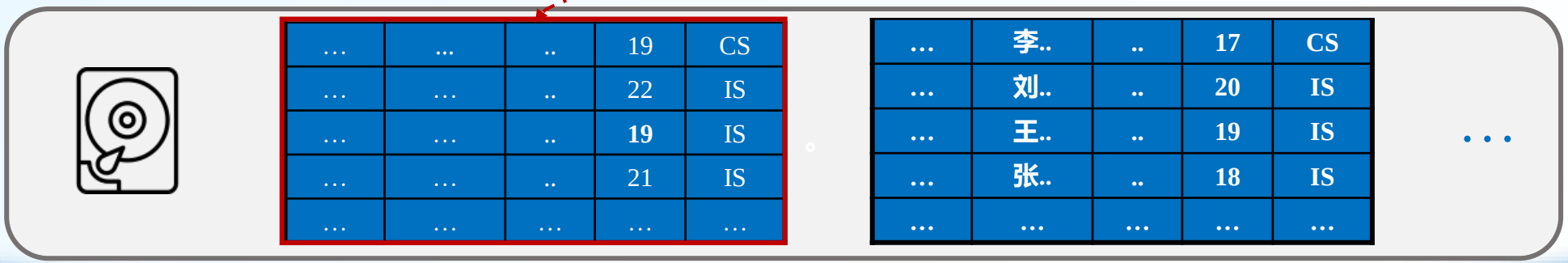
B(R)=2000，T(R)=100000，(80B/tuple，50tuple/block)

索引扫描 Index Scan

- 前面讨论的过滤谓词都是“=”
- 如果是比较谓词“<, >, ≤, ≥”？
  - 对于Table Scan仍然是B(R)（2000）次I/O操作
  - 对于Index Scan
    - 非聚簇索引
      - <v：先定位到v对应的叶子(h次)，然后顺序扫描叶子提取所有指向记录的指针。
      - 根据获取的指针读取对应数据块（随机I/O）
        - $T(\sigma_{A<v}(R))$次
      - 合计I/O：$\gamma+T(\sigma_{A<v}(R))$

```
SELECT department
FROM Student
WHERE Age < 19;
```





…	…	..	19	CS
…	…	..	22	IS
…	…	..	**19**	IS
…	…	..	21	IS
…	…	…	…	…

…	李..	..	17	CS
…	刘..	..	20	IS
…	王..	..	19	IS
…	张..	..	18	IS
…	…	…	…	…

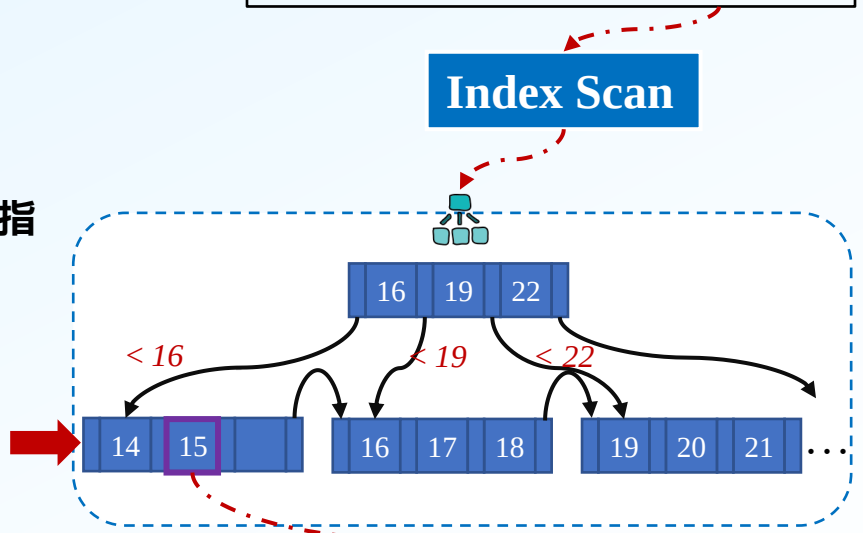
索引扫描（比较谓词、非聚簇索引）

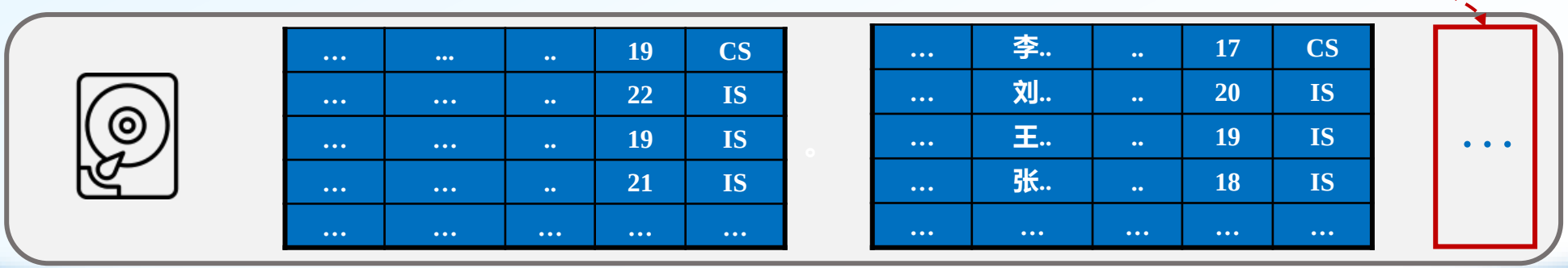
B(R)=2000，T(R)=100000，(80B/tuple，50tuple/block)

索引扫描 Index Scan

- 前面讨论的过滤谓词都是“=”
- 如果是比较谓词“<, >, ≤, ≥”？
  - 对于Table Scan仍然是B(R)（2000）次I/O操作
  - 对于Index Scan
    - 非聚簇索引
      - <v：先定位到v对应的叶子(h次)，然后顺序扫描叶子提取所有指向记录的指针。
      - 根据获取的指针读取对应数据块（随机I/O）
        - $T(\sigma_{A<v}(R))$次
      - 合计I/O：$\gamma+T(\sigma_{A<v}(R))$

```
SELECT department
FROM Student
WHERE Age < 19;
```





…	…	..	19	CS
…	…	..	22	IS
…	…	..	19	IS
…	…	..	21	IS
…	…	…	…	…

…	李..	..	17	CS
…	刘..	..	20	IS
…	王..	..	19	IS
…	张..	..	18	IS
…	…	…	…	…

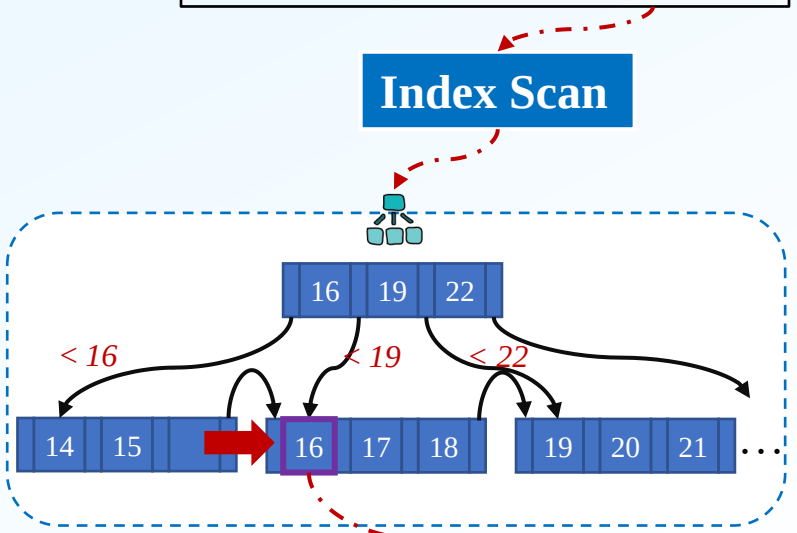
索引扫描（比较谓词、非聚簇索引）

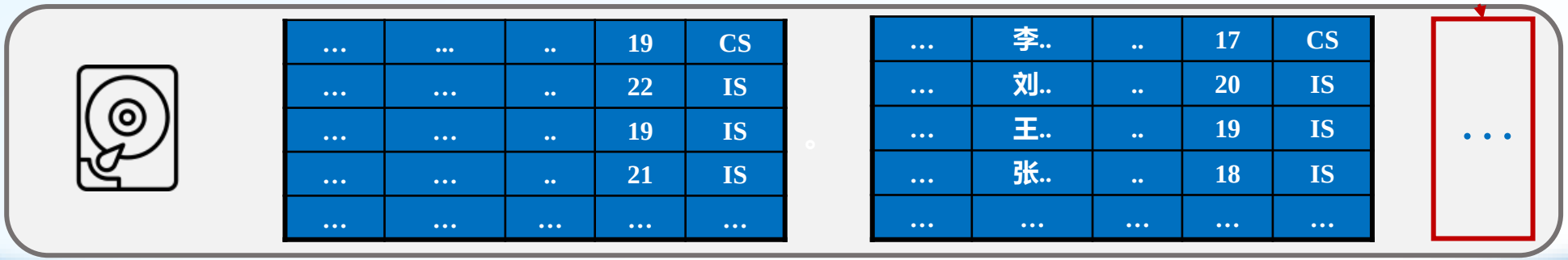
B(R)=2000，T(R)=100000，(80B/tuple，50tuple/block)

索引扫描 Index Scan

- 前面讨论的过滤谓词都是“=”
- 如果是比较谓词“<, >, ≤, ≥”？
  - 对于Table Scan仍然是B(R)（2000）次I/O操作
  - 对于Index Scan
    - 非聚簇索引
      - <v：先定位到v对应的叶子(h次)，然后顺序扫描叶子提取所有指向记录的指针。
      - 根据获取的指针读取对应数据块（随机I/O）
        - $T(\sigma_{A<v}(R))$次
      - 合计I/O：$\gamma+T(\sigma_{A<v}(R))$

```
SELECT department
FROM Student
WHERE Age < 19;
```





…	…	..	19	CS
…	…	..	22	IS
…	…	..	19	IS
…	…	..	21	IS
…	…	…	…	…

…	李..	..	17	CS
…	刘..	..	20	IS
…	王..	..	19	IS
…	张..	..	18	IS
…	…	…	…	…

索引扫描（比较谓词、非聚簇索引）

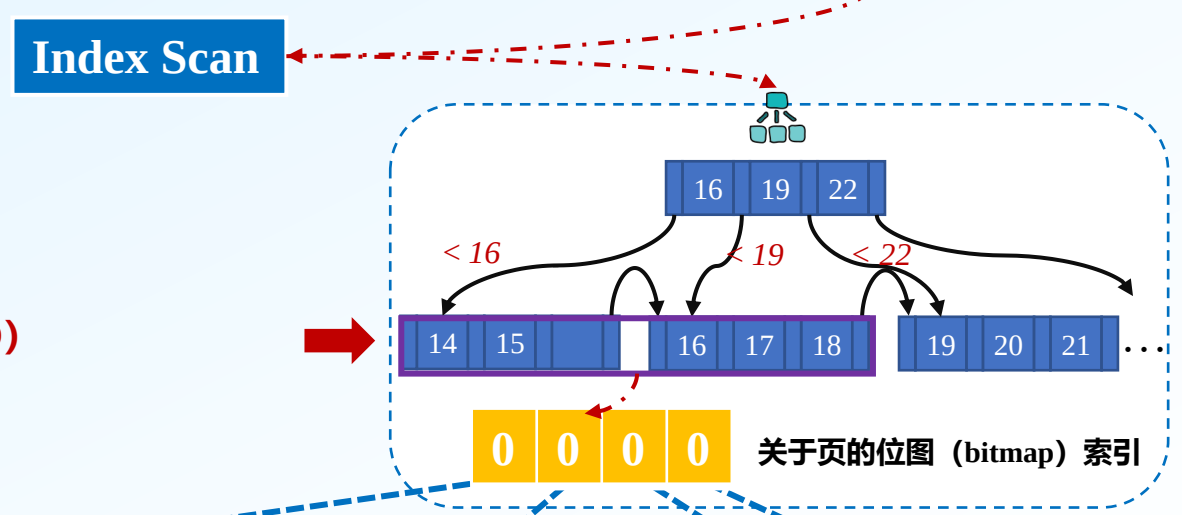
B(R)=2000，T(R)=100000，(80B/tuple，50tuple/block)

索引扫描 Index Scan

- 前面讨论的过滤谓词都是“=”
- 如果是比较谓词“<, >, ≤, ≥”？
  - 对于Table Scan仍然是B(R)（2000）次I/O操作
  - 对于Index Scan
    - 非聚簇索引
      - <v：先定位到v对应的叶子(h次)，然后顺序扫描叶子提取所有指向c指针。
      - 根据获取的指针读取对应数据块（随机I/O）
        - $T(\sigma_{A<v}(R))$次
      - 合计I/O：$\gamma+T(\sigma_{A<v}(R))$



```
SELECT department
FROM Student
WHERE Age < 19;
```





…	…	..	19	CS
…	…	..	22	IS
…	…	..	19	IS
…	…	..	21	IS
…	…	…	…	…

…	李..	..	17	CS
…	刘..	..	20	IS
…	王..	..	19	IS
…	张..	..	18	IS
…	…	…	…	…

…

索引扫描（比较谓词、非聚簇索引）

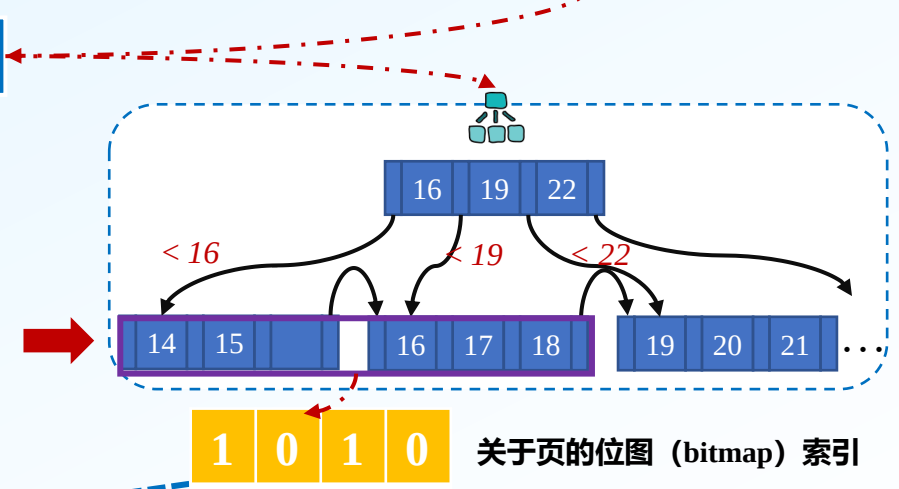
B(R)=2000，T(R)=100000，(80B/tuple，50tuple/block)

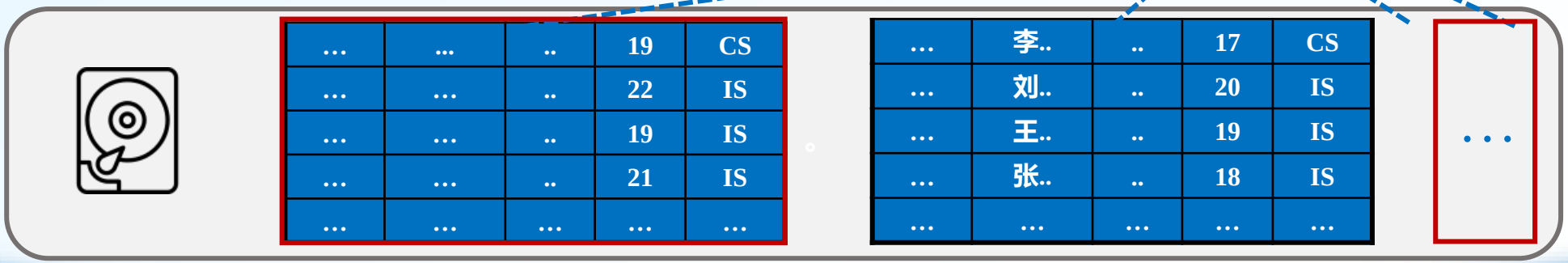
索引扫描 Index Scan

- 前面讨论的过滤谓词都是“=”
- 如果是比较谓词“<, >, ≤, ≥”？
  - 对于Table Scan仍然是B(R)（2000）次I/O操作
  - 对于Index Scan
    - 非聚簇索引
      - <v：先定位到v对应的叶子(h次)，然后顺序扫描叶子提取所有指向c指针。
      - 根据获取的指针读取对应数据块（随机I/O）
        - $T(\sigma_{A<v}(R))$次
      - 合计I/O：$\gamma+T(\sigma_{A<v}(R))$

$$B(R) \times \frac{T(\sigma_{A<v}(R))}{T(R)}$$

```
SELECT department
FROM Student
WHERE Age < 19;
```





…	…	..	19	CS
…	…	..	22	IS
…	…	..	19	IS
…	…	..	21	IS
…	…	…	…	…

…	李..	..	17	CS
…	刘..	..	20	IS
…	王..	..	19	IS
…	张..	..	18	IS
…	…	…	…	…

索引扫描（比较谓词、非聚簇索引）

B(R)=2000，T(R)=100000，(80B/tuple，50tuple/block)

- **索引扫描 Index Scan**
  - 前面讨论的过滤谓词都是“=”
  - 如果是比较谓词“<, >, ≤, ≥”？
    - 对于Table Scan仍然是B(R)（2000）次I/O操作
    - 对于Index Scan
      - 聚簇索引
        - ≥v：先定位到v对应的叶子(h次)，然后从其指向的数据块顺序扫描所有剩余块
        - <v：直接顺序扫描数据块，直到v（不使用索引）
      - 非聚簇索引
        - ≥v：先定位到v对应的叶子(h次)，然后顺序扫描叶子提取所有指向记录的指针
        - <v：直接顺序扫描所有叶子，并提取指向记录的指针，直到v
        - 根据获取的指针读取对应数据块（随机I/O）



95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
…	…	…	…	…

Index Scan

```
SELECT department
FROM Student
WHERE age < 19;
```

Table Scan vs. Index Scan

- B(R) = 2000， T(R) = 100000，(80B/tuple，50tuple/block)
- $S_p(R)$：谓词条件 p 在关系 R 上的选择率: $\frac{T(\sigma_p(R))}{T(R)}$
- $\lceil \log_{n/2}(T(R)) \rceil$； γ：$S_p(R)*T(R)/n$； n：B+树分支因子

95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
…	…	…	…	…

= 谓词	Table Scan	Index Scan	
		聚簇索引	非聚簇索引
搜索码唯一	平均B(R)/2	$\lceil \log_{n/2}(T(R)) \rceil$+1	$\lceil \log_{n/2}(T(R)) \rceil$+1
搜索码不唯一	B(R)	$\lceil \log_{n/2}(T(R)) \rceil$+$S_p(R)*B(R)$	$\lceil \log_{n/2}(T(R)) \rceil$+min$\{S_p(R)*T(R), B(R)\}$
		顺序I/O	随机I/O

谓词	Table Scan	Index Scan	
		聚簇索引	非聚簇索引
≥	B(R)	$\lceil \log_{n/2}(T(R)) \rceil$+$S_p(R)*B(R)$	$\lceil \log_{n/2}(T(R)) \rceil$+$S_p(R)*T(R)/n$+$S_p(R)*B(R)$
<	B(R)	$S_p(R)*B(R)$	$S_p(R)*T(R)/n$+$S_p(R)*B(R)$

Table Scan vs. Index Scan

- B(R) = 2000，T(R) = 100000，(80B/tuple，50tuple/block)
- Recall: *n*一般取100左右(参考上一章94页PPT)
- *h*：⌈ $\log_{50}(100000)$ ⌉=3； γ：$S_p(R)$*1000；$0 \le S_p(R) \le 1$

95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
…	…	…	…	…

= 谓词	Table Scan	Index Scan	
		聚簇索引	非聚簇索引
搜索码唯一	平均1000	4	4
搜索码不唯一	2000	$3+2000S_p(R)$	$3+\min\{100000S_p(R), 2000\}$

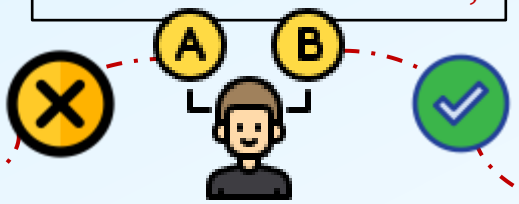
顺序I/O　　随机I/O

谓词	Table Scan	Index Scan	
		聚簇索引	非聚簇索引
≥	2000	$3+2000S_p(R)$	$3+1000S_p(R)+2000S_p(R)$
<	2000	$2000S_p(R)$	$1000S_p(R)+2000S_p(R)$

Table Scan vs. Index Scan

95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
…	…	…	…	…

```
SELECT department
FROM Student
WHERE StuID = 95044;
```



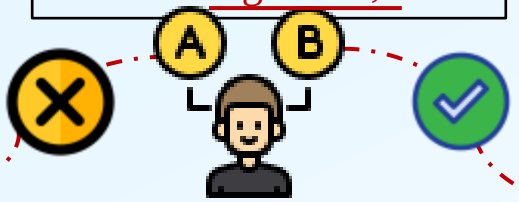
= 谓词	Table Scan	Index Scan	
		聚簇索引	非聚簇索引
搜索码唯一	平均1000	4	4
搜索码不唯一	2000	$3+2000S_p(R)$	$3+\min\{100000S_p(R), 2000\}$
		顺序I/O	随机I/O

谓词	Table Scan	Index Scan	
		聚簇索引	非聚簇索引
≥	2000	$3+2000S_p(R)$	$3+1000S_p(R)+2000S_p(R)$
<	2000	$2000S_p(R)$	$1000S_p(R)+2000S_p(R)$

Table Scan vs. Index Scan

95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
…	…	…	…	…

```
SELECT department
FROM Student
WHERE Age = 19;
```



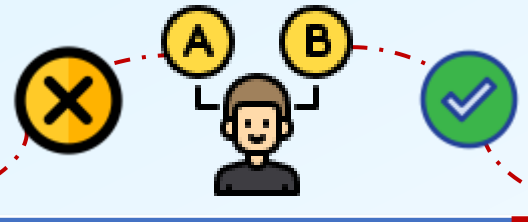
= 谓词	Table Scan	Index Scan	
		if... 聚簇索引	非聚簇索引
搜索码唯一	平均1000	4	4
搜索码不唯一	2000	$3+2000S_p(R)$	$3+\min\{100000S_p(R), 2000\}$
		顺序I/O	随机I/O

谓词	Table Scan	Index Scan	
		聚簇索引	非聚簇索引
≥	2000	$3+2000S_p(R)$	$3+1000S_p(R)+2000S_p(R)$
<	2000	$2000S_p(R)$	$1000S_p(R)+2000S_p(R)$

Table Scan vs. Index Scan

95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
…	…	…	…	…

```
SELECT department
FROM Student
WHERE Age = 19;
```



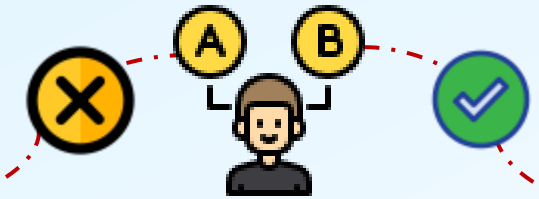
= 谓词	Table Scan	Index Scan	
		聚簇索引	if… 非聚簇索引
搜索码唯一	平均1000	4	4
搜索码不唯一	2000	$3+2000S_p(R)$	$3+\min\{100000S_p(R), 2000\}$
		顺序I/O	随机I/O

谓词	Table Scan	Index Scan	
		聚簇索引	非聚簇索引
≥	2000	$3+2000S_p(R)$	$3+1000S_p(R)+2000S_p(R)$
<	2000	$2000S_p(R)$	$1000S_p(R)+2000S_p(R)$

Table Scan vs. Index Scan

95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
…	…	…	…	…

DBEDU 可在DBEDU-MOCHA子系统自行尝试实际案例




```
SELECT department
FROM Student
WHERE Age = 19;
```

= 谓词	Table Scan	Index Scan	
		聚簇索引	if… 非聚簇索引
搜索码唯一	平均1000	4	4
搜索码不唯一	2000	$3+2000S_p(R)$	$3+\min\{100000S_p(R), 2000\}$

顺序I/O　　随机I/O

Q A

$S_p(R)=\frac{T(\sigma_p(R))}{T(R)}$

谓词	Table Scan	Index Scan	
		聚簇索引	非聚簇索引
≥	2000	$3+2000S_p(R)$	$3+1000S_p(R)+2000S_p(R)$
<	2000	$2000S_p(R)$	$1000S_p(R)+2000S_p(R)$

选择率(Selectivity)将在下一节：统计信息详细介绍

总结

- **全表扫描 Table Scan (a.k.a. Sequential Scan)**
  - = 谓词、比较谓词无差别
- **索引扫描 Index Scan**
  - = 谓词 vs. 比较谓词
  - 搜索码唯一 vs. 不唯一
  - 聚簇索引 vs. 非聚簇索引
- **其他扫描方法**
  - **仅索引扫描 Index Only Scan**
    - 与索引扫描相同，除了没有访问表，只访问索引键
      - 当SELECT的所有目标列都在索引中
      - 不用再返回表去读取数据
      - 依赖于（第二章提到的）索引文件



嵌套循环、哈希连接、排序合并

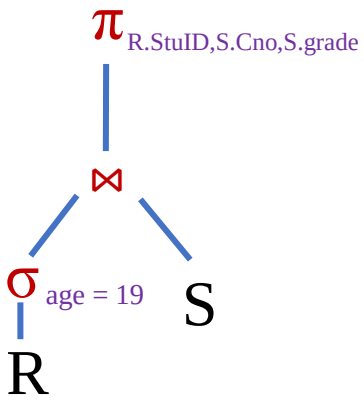
- **嵌套循环 Nested Loop**
  - 扫描外表（左表）；
  - 对于外表的每个符合条件的记录，根据连接条件，扫描内表 （右表）来执行连接。

95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
…	…	…	…	…

95001	CS2002	80
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
…	…	…

- **排序合并 Sort Merge**
  - 如果结果集尚未排序，扫描外表并根据连接键对结果集进行排序；
  - 如果结果集尚未排序，扫描内表并根据连接键对结果集进行排序；
  - 根据连接条件合并两个结果集 。
- **哈希连接 Hash Join**
  - 扫描内表以创建基于连接键的哈希结构（这是索引的一种形式）；
  - 扫描外表，对于每个符合条件的记录，使用哈希结构连接内表。

```
SELECT R.StuID, S.Cno, S.grade
FROM Student R, SC S
WHERE age = 19 AND R.StuID=S.Sno;
```



嵌套循环

- **嵌套循环 Nested Loop**
  - 扫描外表（左表）；
  - 对于外表的每个符合条件的记录，根据连接条件，扫描内表（右表）来执行连接。

```
for each tuple i in R:
    for each tuple j in S:
        if iθj, 拼接i和j并输出到Join结果
```

95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
95005	陈刚	男	19	CS

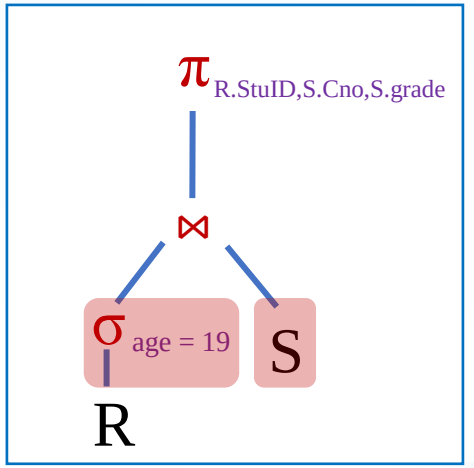
95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58

95002	刘晨	女	19	IS
95004	张立	男	19	IS
95005	陈刚	男	19	CS

```
SELECT R.StuID, S.Cno, S.grade
FROM Student R, SC S
WHERE age = 19 AND R.StuID=S.Sno;
```

⋈

95002	刘晨	女	19	IS	CS2012	92
95004	张立	男	19	IS	CE2001	86
95004	张立	男	19	IS	CE2004	78
95005	陈刚	男	19	CS	CS2031	58



嵌套循环

嵌套循环 Nested Loop

```
for each tuple i in R:
    for each tuple j in S:
        if iθj, 拼接i和j并输出到Join结果
```

I/O数量?

- 外层循环：B(R)
- 内层循环：T(R)*B(S)
- 合计： B(R)+T(R)*B(S)

示例：
B(R)=2000，T(R)=100000；
B(S)=500，T(S)=20000

B(R)+T(R)*B(S)=2000+100000*500=50,002,000 I/Os

如果左右表交换呢?

B(S)+T(S)*B (R)=500+20000*2000=40,000,500 I/Os

R

95002	刘晨	女	19	IS
95004	张立	男	19	IS
95005	陈刚	男	19	CS

S

95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58

⋈

95002	刘晨	女	19	IS	CS2012	92
95004	张立	男	19	IS	CE2001	86
95004	张立	男	19	IS	CE2004	78
95005	陈刚	男	19	CS	CS2031	58

嵌套循环

嵌套循环 Nested Loop

```
for each tuple i in R:
    for each tuple j in S:
        if iθj, 拼接i和j并输出到Join结果
```

基于块访问方式的改进 Block Nested Loop

```
for each 块r in R:
    for each 块s in S:
        for each tuple i in 块r :
            for each tuple j in 块s :
                if iθj, 拼接i和j并输出到Join结果
```

I/O数量?

- 外层循环：B(R)
- 内层循环：B(R)*B(S)
- 合计： B(R)+B(R)*B(S)

结论：小表做外表（左表）

如果使用缓冲池辅助呢?

R

95002	刘晨	女	19	IS
95004	张立	男	19	IS
95005	陈刚	男	19	CS

S

95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58

$\bowtie$

95002	刘晨	女	19	IS	CS2012	92
95004	张立	男	19	IS	CE2001	86
95004	张立	男	19	IS	CE2004	78
95005	陈刚	男	19	CS	CS2031	58

示例：
B(R)=2000，T(R)=100000；
B(S)=500，T(S)=20000
B(R)+B(R)*B(S)=2000+2000*500=1,002,000 I/Os

如果左右表交换呢?

B(S)+B(S)*B (R)=500+500*2000=1,000,500 I/Os

嵌套循环

基于块访问方式的改进 Block Nested Loop

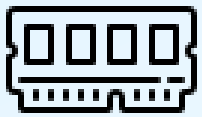
```
for each 缓冲池中的块r in R:
    for each 块s in S:
        for each tuple i in 缓冲池中的块r :
            for each tuple j in 块s:
                if iθj, 拼接i和j并输出到Join页
```

R

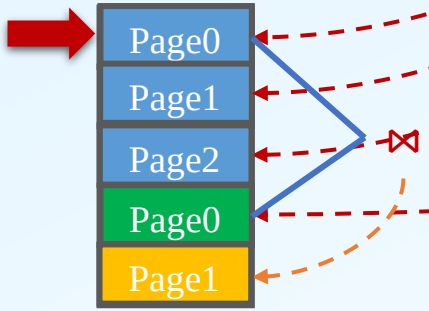
95002	刘晨	女	19	IS
95004	张立	男	19	IS
95005	陈刚	男	19	CS

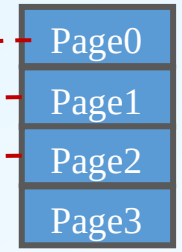
S

95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58



缓冲池





⋈

95002	刘晨	女	19	IS	CS2012	92
95004	张立	男	19	IS	CE2001	86
95004	张立	男	19	IS	CE2004	78
95005	陈刚	男	19	CS	CS2031	58

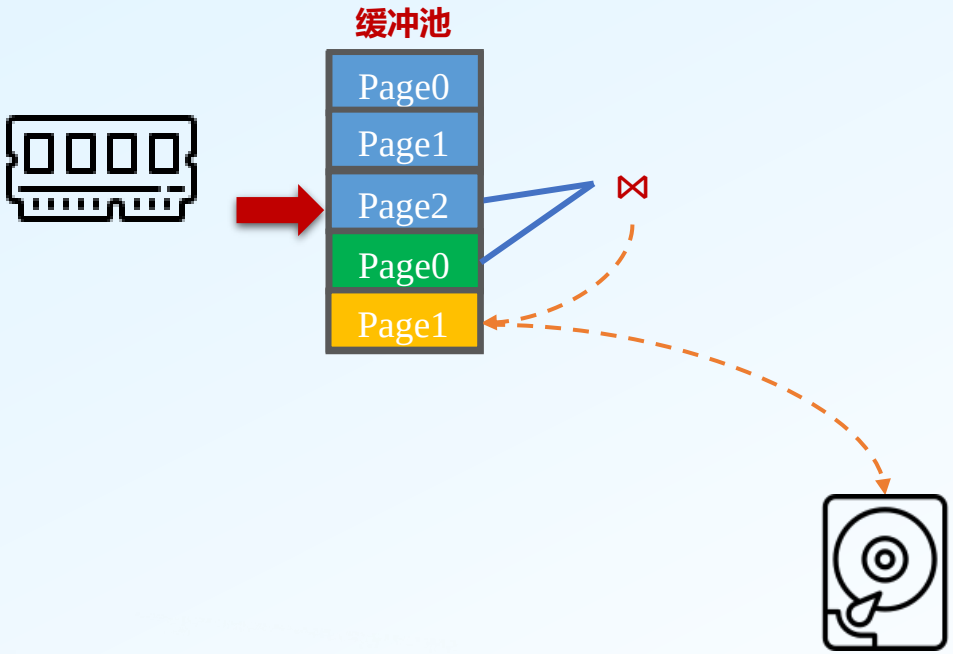


- 尽可能把左表载入缓冲池
- 分别保留1帧用于缓冲
  - 内表
  - 结果
- 结果存满一帧（页）即可落盘
- 故：可用于缓冲左表的共有#Buffer-2帧

嵌套循环

■ 基于块访问方式的改进 Block Nested Loop

```
for each 缓冲池中的块r in R:
    for each 块s in S:
        for each tuple i in 缓冲池中的块r :
            for each tuple j in 块s:
                if iθj, 拼接i和j并输出到Join页
```



R

95002	刘晨	女	19	IS
95004	张立	男	19	IS
95005	陈刚	男	19	CS

S

95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58

Page0
Page1
Page2
Page3

⋈

95002	刘晨	女	19	IS	CS2012	92
95004	张立	男	19	IS	CE2001	86
95004	张立	男	19	IS	CE2004	78
95005	陈刚	男	19	CS	CS2031	58

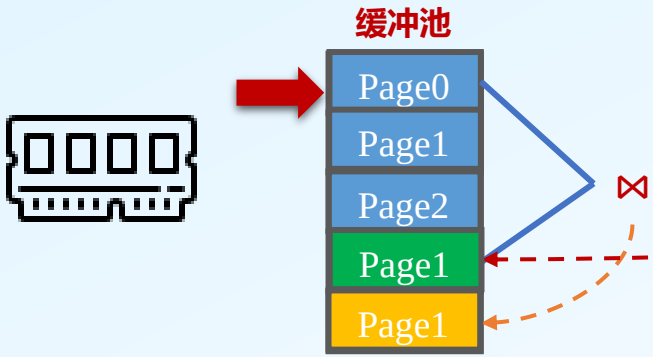
Page0
Page1
Page2
Page3
Page4

- 尽可能把左表载入缓冲池
- 分别保留1帧用于缓冲
  - 内表
  - 结果
- 结果存满一帧（页）即可落盘
- 故：可用于缓冲左表的共有#Buffer-2帧

嵌套循环

基于块访问方式的改进 Block Nested Loop

```
for each 缓冲池中的块r in R:
    for each 块s in S:
        for each tuple i in 缓冲池中的块r :
            for each tuple j in 块s:
                if iθj, 拼接i和j并输出到Join页
```



R

95002	刘晨	女	19	IS
95004	张立	男	19	IS
95005	陈刚	男	19	CS

S

95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58

Page0
Page1
Page2
Page3

⋈

95002	刘晨	女	19	IS	CS2012	92
95004	张立	男	19	IS	CE2001	86
95004	张立	男	19	IS	CE2004	78
95005	陈刚	男	19	CS	CS2031	58

Page0
Page1
Page2
Page3
Page4

- 尽可能把左表载入缓冲池
- 分别保留1帧用于缓冲
  - 内表
  - 结果
- 结果存满一帧（页）即可落盘
- 故：可用于缓冲左表的共有#Buffer-2帧

嵌套循环

基于块访问方式的改进 Block Nested Loop

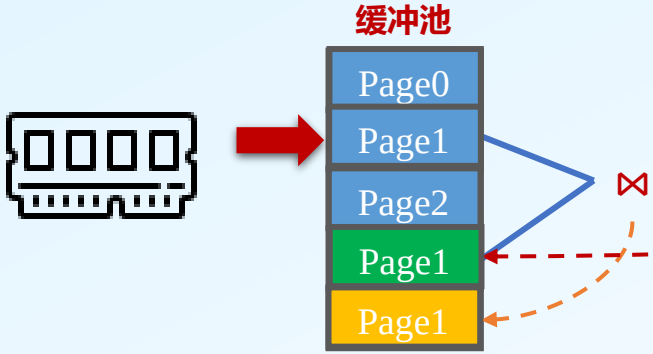
```
for each 缓冲池中的块r in R:
    for each 块s in S:
        for each tuple i in 缓冲池中的块r :
            for each tuple j in 块s:
                if iθj, 拼接i和j并输出到Join页
```

R

95002	刘晨	女	19	IS
95004	张立	男	19	IS
95005	陈刚	男	19	CS

S

95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58



Page0
Page1
Page2
Page3

⋈

95002	刘晨	女	19	IS	CS2012	92
95004	张立	男	19	IS	CE2001	86
95004	张立	男	19	IS	CE2004	78
95005	陈刚	男	19	CS	CS2031	58

Page0
Page1
Page2
Page3
Page4

- 尽可能把左表载入缓冲池
- 分别保留1帧用于缓冲
  - 内表
  - 结果
- 结果存满一帧（页）即可落盘
- 故：可用于缓冲左表的共有#Buffer-2帧

嵌套循环

■ 基于块访问方式的改进 Block Nested Loop

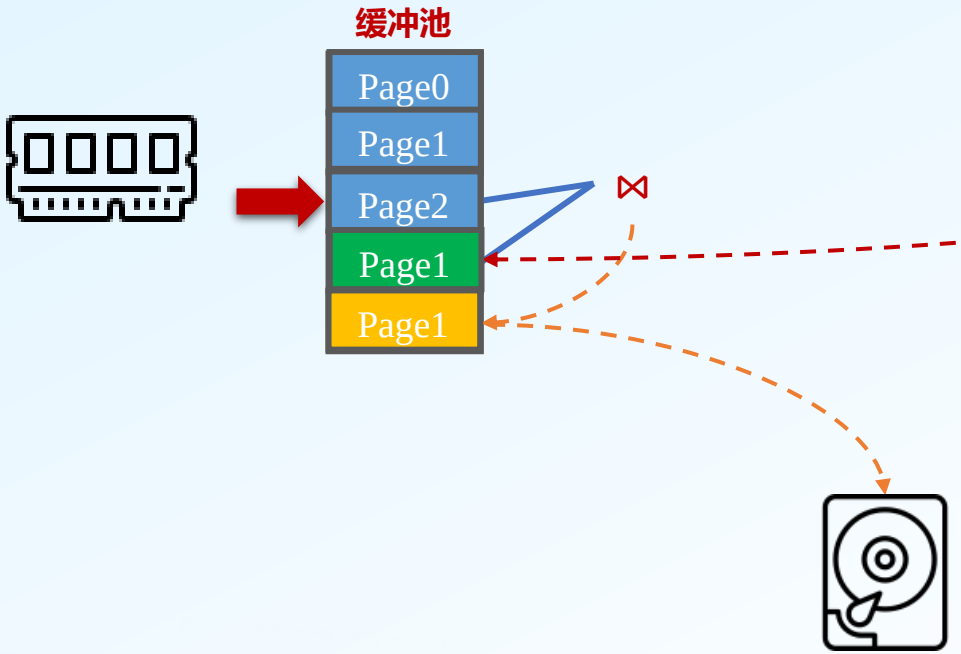
```
for each 缓冲池中的块r in R:
    for each 块s in S:
        for each tuple i in 缓冲池中的块r :
            for each tuple j in 块s:
                if iθj, 拼接i和j并输出到Join页
```

R

95002	刘晨	女	19	IS
95004	张立	男	19	IS
95005	陈刚	男	19	CS

S

95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58



- 尽可能把左表载入缓冲池
- 分别保留1帧用于缓冲
  - 内表
  - 结果
- 结果存满一帧（页）即可落盘
- 故：可用于缓冲左表的共有#Buffer-2帧

嵌套循环

■ 基于块访问方式的改进 Block Nested Loop

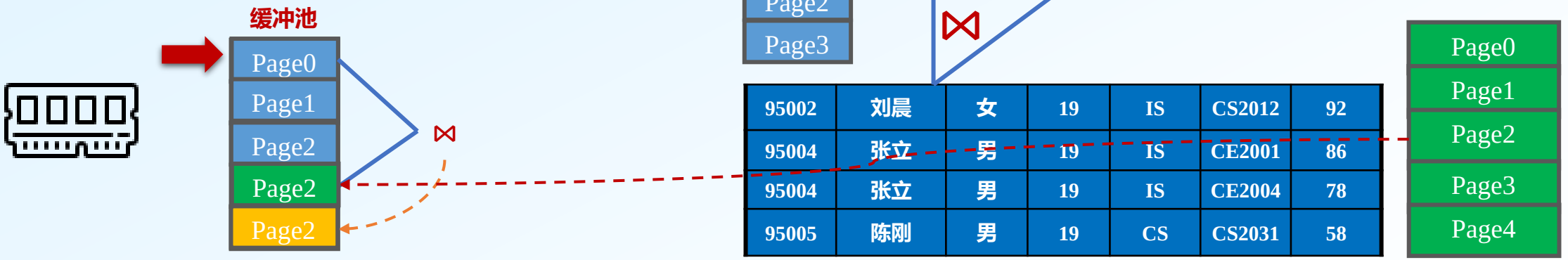
```
for each 缓冲池中的块r in R:
    for each 块s in S:
        for each tuple i in 缓冲池中的块r :
            for each tuple j in 块s:
                if iθj, 拼接i和j并输出到Join页
```

R

95002	刘晨	女	19	IS
95004	张立	男	19	IS
95005	陈刚	男	19	CS

S

95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58

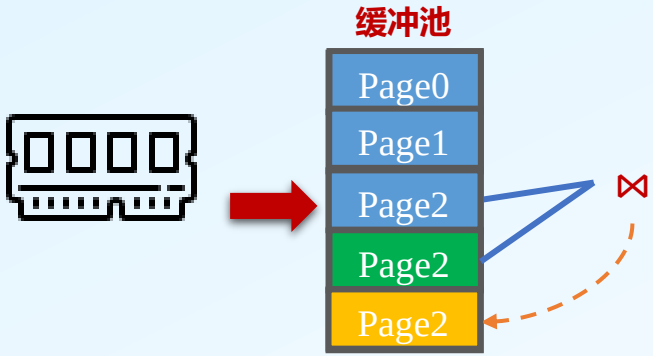


- 尽可能把左表载入缓冲池
- 分别保留1帧用于缓冲
  - 内表
  - 结果
- 结果存满一帧（页）即可落盘
- 故：可用于缓冲左表的共有#Buffer-2帧

嵌套循环

■ 基于块访问方式的改进 Block Nested Loop

```
for each 缓冲池中的块r in R:
    for each 块s in S:
        for each tuple i in 缓冲池中的块r :
            for each tuple j in 块s:
                if iθj, 拼接i和j并输出到Join页
```



R

95002	刘晨	女	19	IS
95004	张立	男	19	IS
95005	陈刚	男	19	CS

S

95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58

Page0
Page1
Page2
Page3

⋈

95002	刘晨	女	19	IS	CS2012	92
95004	张立	男	19	IS	CE2001	86
95004	张立	男	19	IS	CE2004	78
95005	陈刚	男	19	CS	CS2031	58

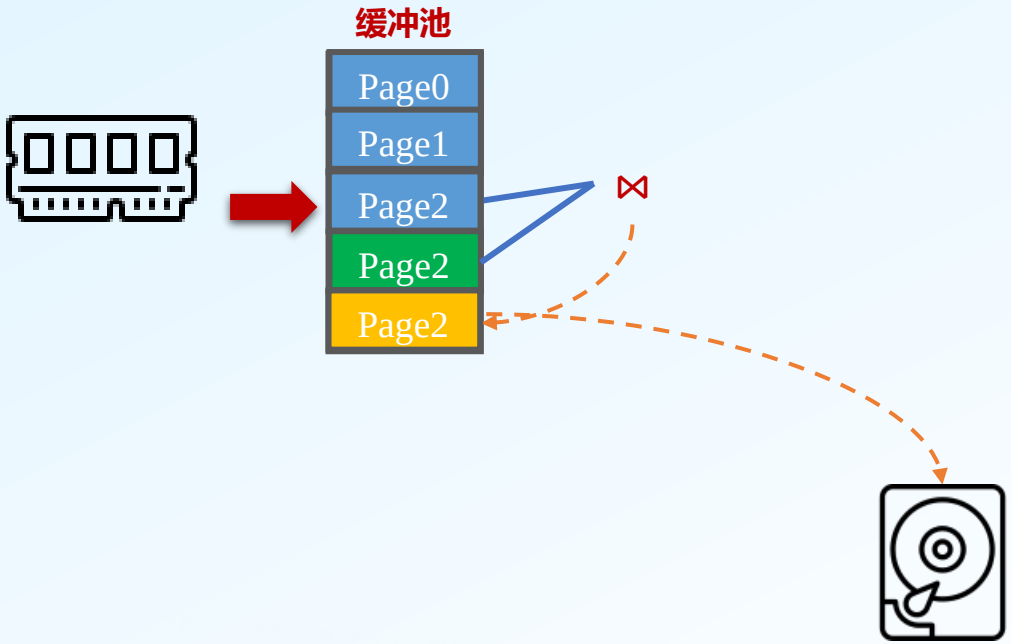
Page0
Page1
Page2
Page3
Page4

- 尽可能把左表载入缓冲池
- 分别保留1帧用于缓冲
  - 内表
  - 结果
- 结果存满一帧（页）即可落盘
- 故：可用于缓冲左表的共有#Buffer-2帧

嵌套循环

■ 基于块访问方式的改进 Block Nested Loop

```
for each 缓冲池中的块r in R:
    for each 块s in S:
        for each tuple i in 缓冲池中的块r :
            for each tuple j in 块s:
                if iθj, 拼接i和j并输出到Join页
```



R

95002	刘晨	女	19	IS
95004	张立	男	19	IS
95005	陈刚	男	19	CS

S

95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58

Page0
Page1
Page2
Page3

⋈

95002	刘晨	女	19	IS	CS2012	92
95004	张立	男	19	IS	CE2001	86
95004	张立	男	19	IS	CE2004	78
95005	陈刚	男	19	CS	CS2031	

Page0
Page1
Page2
Page3
Page4

- 尽可能把左表载入缓冲池
- 分别保留1帧用于缓冲
  - 内表
  - 结果
- 结果存满一帧（页）即可落盘
- 故：可用于缓冲左表的共有#Buffer-2帧

嵌套循环

基于块访问方式的改进 Block Nested Loop

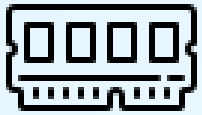
```
for each 缓冲池中的块r in R:
    for each 块s in S:
        for each tuple i in 缓冲池中的块r :
            for each tuple j in 块s:
                if iθj, 拼接i和j并输出到Join页
```

R

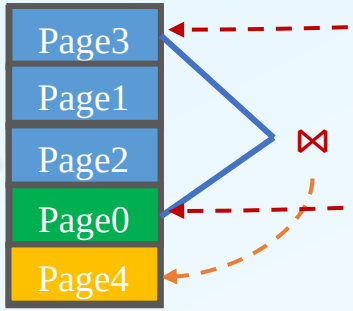
95002	刘晨	女	19	IS
95004	张立	男	19	IS
95005	陈刚	男	19	CS

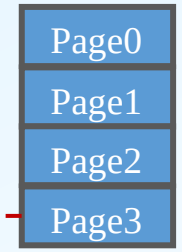
S

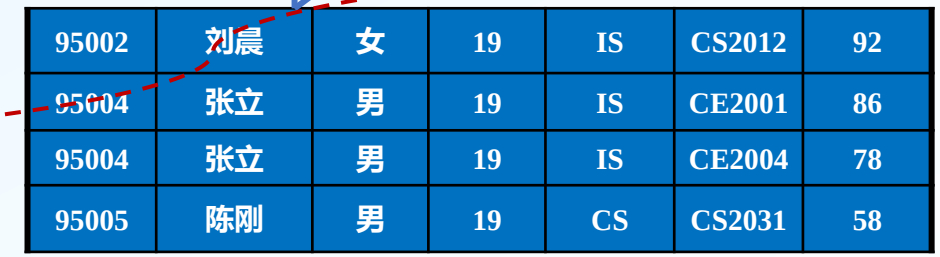
95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58



缓冲池







95002	刘晨	女	19	IS	CS2012	92
95004	张立	男	19	IS	CE2001	86
95004	张立	男	19	IS	CE2004	78
95005	陈刚	男	19	CS	CS2031	58



- 尽可能把左表载入缓冲池
- 分别保留1帧用于缓冲
  - 内表
  - 结果
- 结果存满一帧（页）即可落盘
- 故：可用于缓冲左表的共有#Buffer-2帧

嵌套循环

■ 基于块访问方式的改进 Block Nested Loop

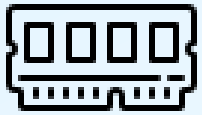
```
for each 缓冲池中的块r in R:
    for each 块s in S:
        for each tuple i in 缓冲池中的块r :
            for each tuple j in 块s:
                if iθj, 拼接i和j并输出到Join页
```

R

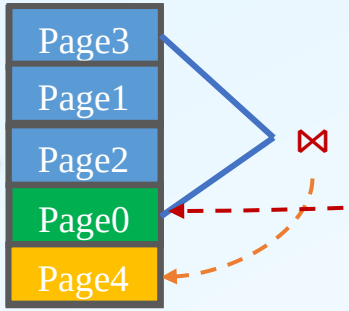
95002	刘晨	女	19	IS
95004	张立	男	19	IS
95005	陈刚	男	19	CS

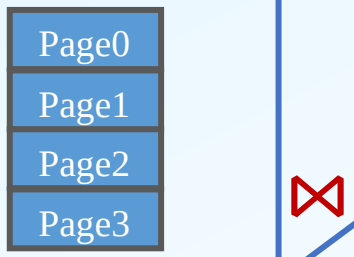
S

95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58



缓冲池





95002	刘晨	女	19	IS	CS2012	92
95004	张立	男	19	IS	CE2001	86
95004	张立	男	19	IS	CE2004	78
95005	陈刚	男	19	CS	CS2031	58





- 尽可能把左表载入缓冲池
- 分别保留1帧用于缓冲
  - 内表
  - 结果
- 结果存满一帧（页）即可落盘
- 故：可用于缓冲左表的共有#Buffer-2帧

嵌套循环

基于块访问方式的改进 Block Nested Loop

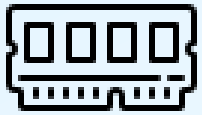
```
for each 缓冲池中的块r in R:
    for each 块s in S:
        for each tuple i in 缓冲池中的块r :
            for each tuple j in 块s:
                if iθj, 拼接i和j并输出到Join页
```

R

95002	刘晨	女	19	IS
95004	张立	男	19	IS
95005	陈刚	男	19	CS

S

95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58

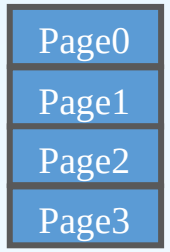


缓冲池





- 内表屡次参与θ谓词的运算
- 可为内表针对θ谓词维护一个临时索引，以加速匹配
- 索引嵌套循环（教材420页，§10.7.3）



95002	刘晨	女	19	IS	CS2012	92
95004	张立	男	19	IS	CE2001	86
95004	张立	男	19	IS	CE2004	78
95005	陈刚	男	19	CS	CS2031	58



- I/O数量?
  - 外层循环：B(R)
  - 内层循环：⌈B(R)/(#Buffer-2)⌉*B(S)
  - 合计： B(R)+⌈B(R)/(#Buffer-2)⌉*B(S)
  - 如果#Buffer-2 >= B(R)
    - B(R)+B(S) !!!



- 尽可能把左表载入缓冲池
- 分别保留1帧用于缓冲
  - 内表
  - 结果
- 结果存满一帧（页）即可落盘
- 故：可用于缓冲左表的共有#Buffer-2帧

排序归并

- **排序归并 Sort Merge**
  - 如果结果集尚未排序，扫描外表并根据连接键对结果集进行排序；
  - 如果结果集尚未排序，扫描内表并根据连接键对结果集进行排序；
  - 根据连接条件归并两个结果集

步骤1：排序

R,S分别按照连接码排序

步骤2：归并

两表分别维护1个游标

按顺序检查匹配

R

95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
95005	陈刚	男	19	CS

S

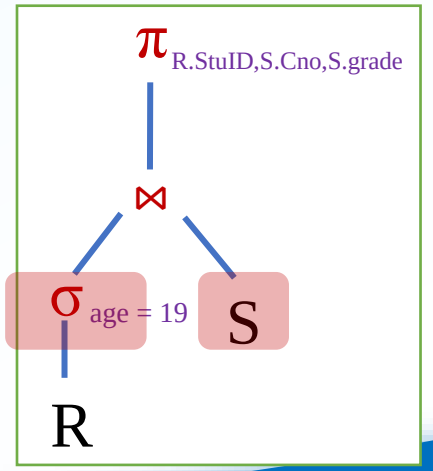
95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58

95002	刘晨	女	19	IS
95004	张立	男	19	IS
95005	陈刚	男	19	CS

SELECT R.StuID, S.Cno, S.grade
FROM Student R, SC S
WHERE age = 19 AND
R.StuID=S.Sno;

⋈

95002	刘晨	女	19	IS	CS2012	92
95004	张立	男	19	IS	CE2001	86
95004	张立	男	19	IS	CE2004	78
95005	陈刚	男	19	CS	CS2031	58



排序归并

➢ 排序归并 Sort Merge

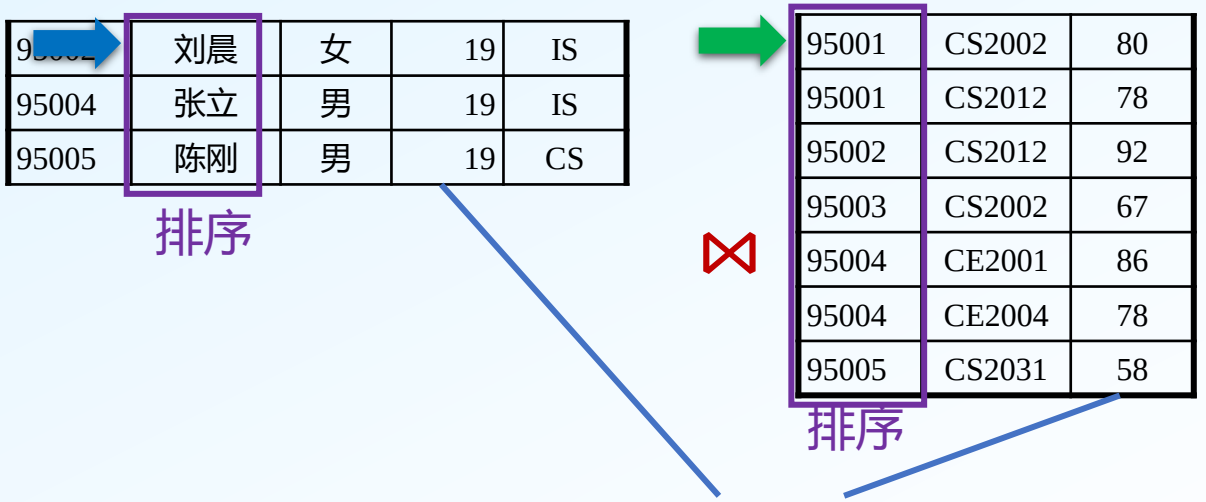
步骤1：排序

R,S分别按照连接码排序

步骤2：归并

两表分别维护1个游标

按顺序检查匹配




```
SELECT R.StuID, S.Cno, S.grade
FROM Student R, SC S
WHERE age = 19 AND
R.StuID=S.Sno;
```

➢ 排序归并 Sort Merge

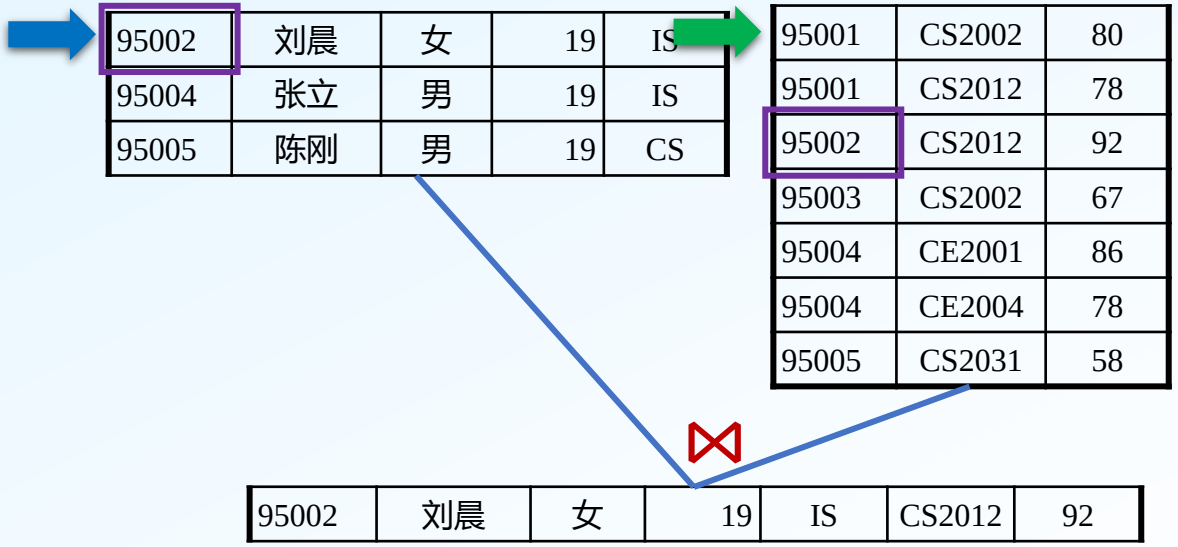
步骤1：排序

R,S分别按照连接码排序

步骤2：归并

两表分别维护1个游标

按顺序检查匹配




```
SELECT R.StuID, S.Cno, S.grade
FROM Student R, SC S
WHERE age = 19 AND
R.StuID=S.Sno;
```

排序归并

➢ 排序归并 Sort Merge

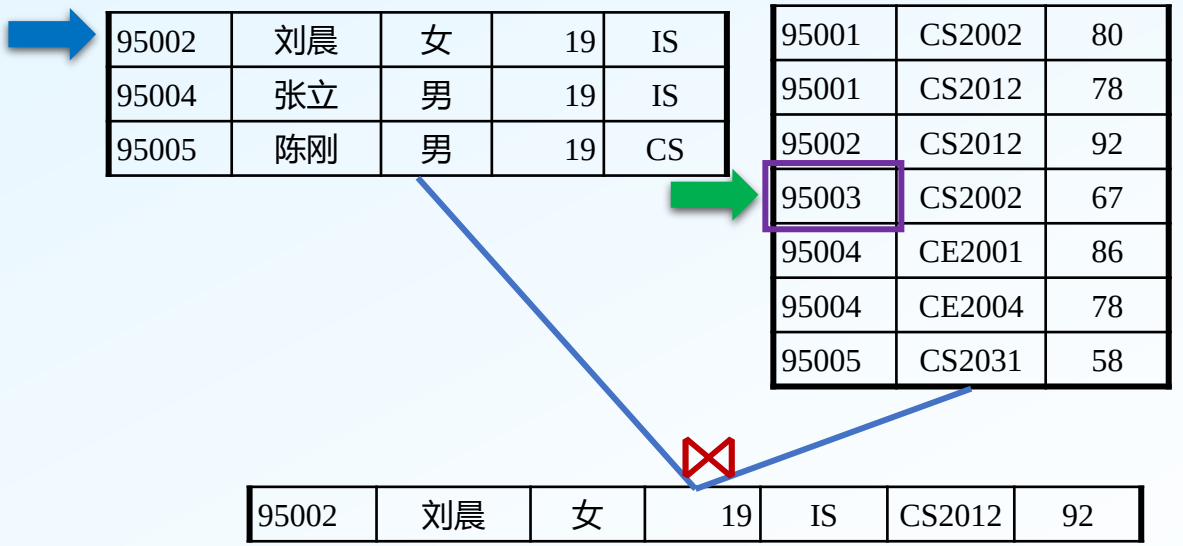
步骤1：排序

R,S分别按照连接码排序

步骤2：归并

两表分别维护1个游标

按顺序检查匹配




```
SELECT R.StuID, S.Cno, S.grade
FROM Student R, SC S
WHERE age = 19 AND
R.StuID=S.Sno;
```

排序归并

➢ 排序归并 Sort Merge

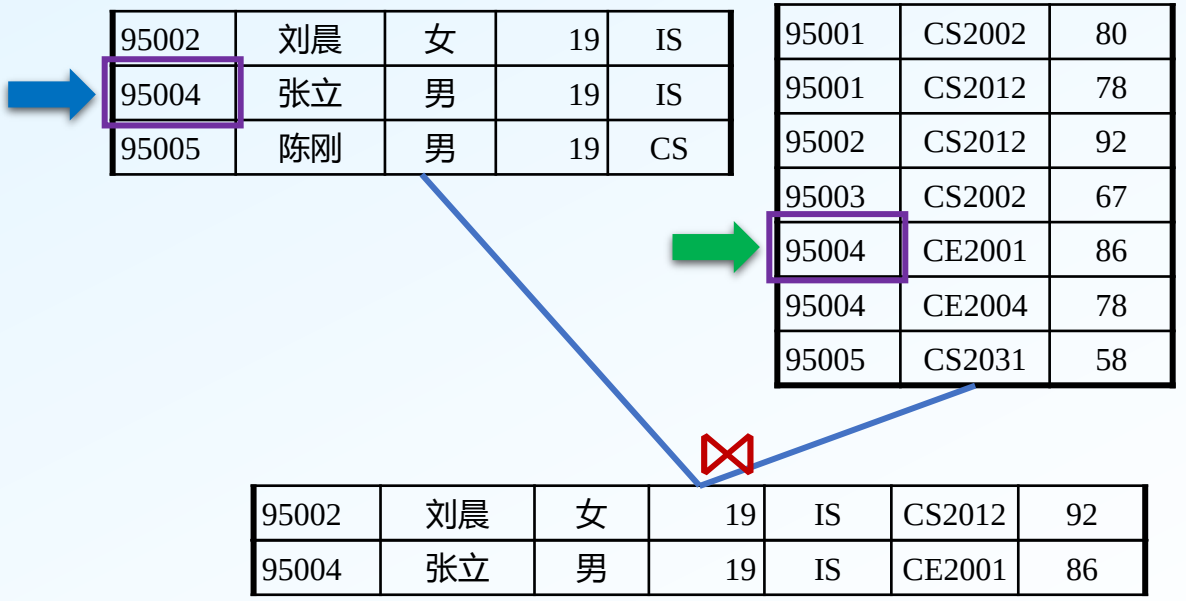
步骤1：排序

R,S分别按照连接码排序

步骤2：归并

两表分别维护1个游标

按顺序检查匹配



95002	刘晨	女	19	IS
95004	张立	男	19	IS
95005	陈刚	男	19	CS

95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58

95002	刘晨	女	19	IS	CS2012	92
95004	张立	男	19	IS	CE2001	86

```
SELECT R.StuID, S.Cno, S.grade
FROM Student R, SC S
WHERE age = 19 AND
R.StuID=S.Sno;
```

排序归并

➢ 排序归并 Sort Merge

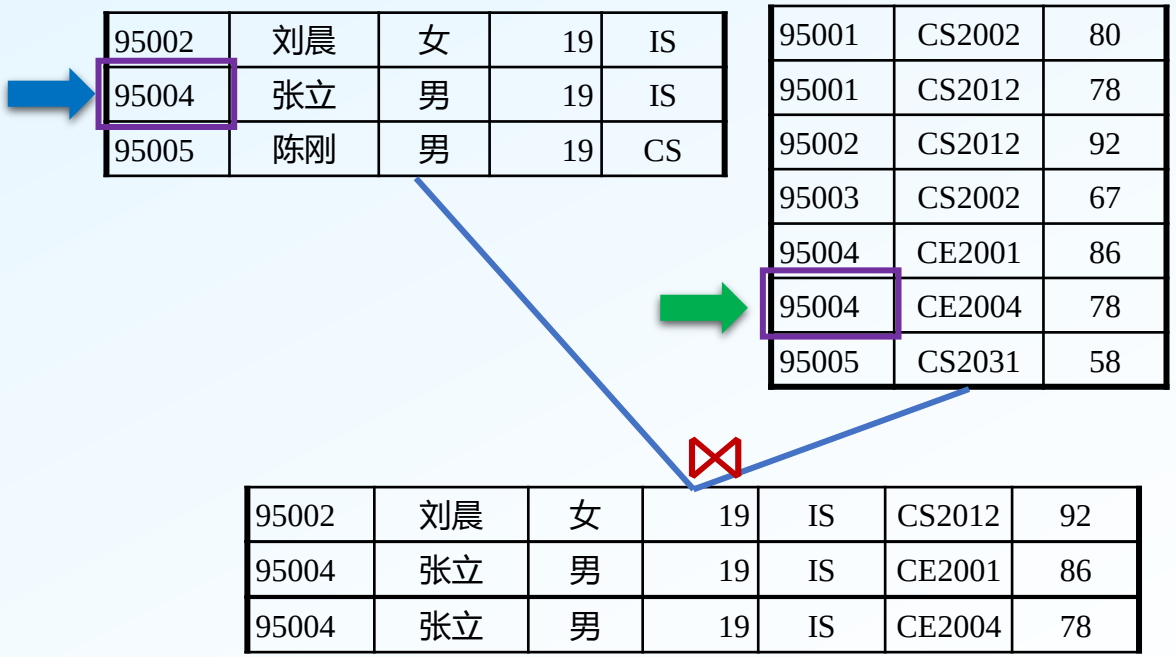
步骤1：排序

R,S分别按照连接码排序

步骤2：归并

两表分别维护1个游标

按顺序检查匹配



95002	刘晨	女	19	IS
95004	张立	男	19	IS
95005	陈刚	男	19	CS

95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58

95002	刘晨	女	19	IS	CS2012	92
95004	张立	男	19	IS	CE2001	86
95004	张立	男	19	IS	CE2004	78

```
SELECT R.StuID, S.Cno, S.grade
FROM Student R, SC S
WHERE age = 19 AND
R.StuID=S.Sno;
```

排序归并

➢ 排序归并 Sort Merge

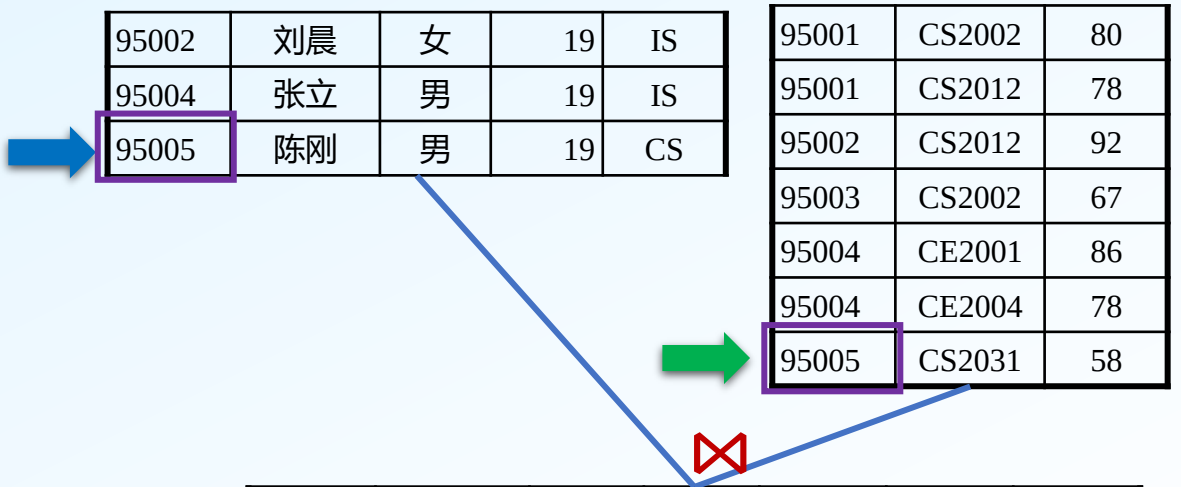
步骤1：排序

R,S分别按照连接码排序

步骤2：归并

两表分别维护1个游标

按顺序检查匹配



95002	刘晨	女	19	IS	CS2012	92
95004	张立	男	19	IS	CE2001	86
95004	张立	男	19	IS	CE2004	78
95005	陈刚	男	19	CS	CS2031	58

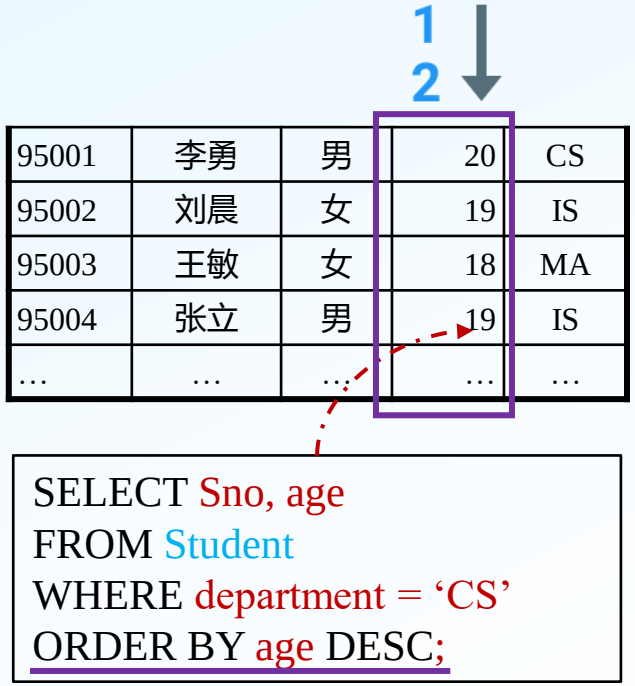
➢ I/O数量?

- 排序： Sort(B(R)), Sort(B(S))
- 合并： B(R)+B(S)
- 合计： 排序+归并
- 排序的cost
  - $2B(R) \times (1 + \lceil \log_2 B(R) \rceil)$

```
SELECT R.StuID, S.Cno, S.grade
FROM Student R, SC S
WHERE age = 19 AND
R.StuID=S.Sno;
```

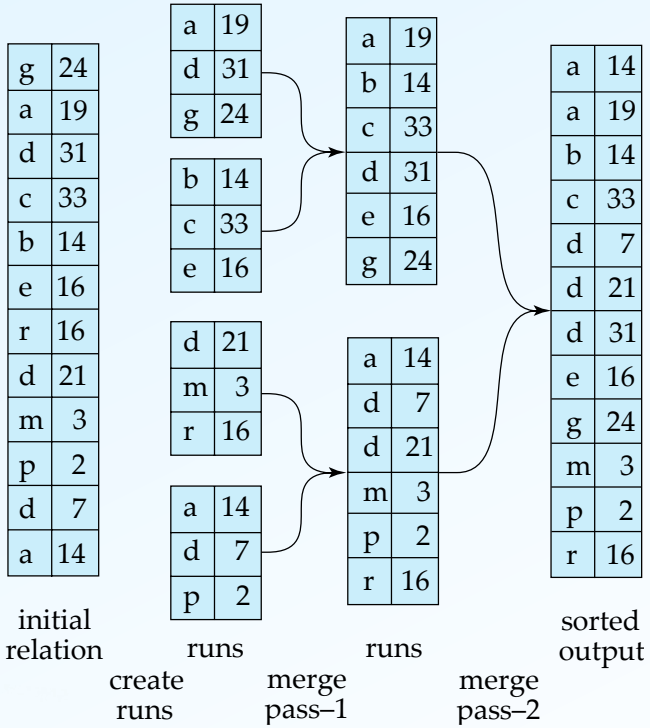
排序归并-外归并排序

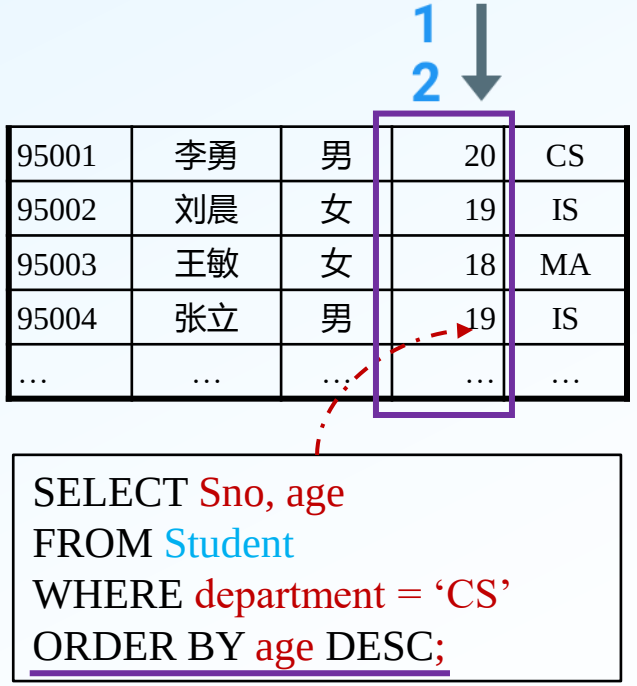
- **排序归并Sort**
  - SQL对应的ORDER BY子句要求
  - 当内存足够时，直接使用快排等排序算法即可
    - 平均时间复杂度$O(n\log_2 n)$
- **What if 内存不够?**
  - 借助外存的排序算法
    - 需要考虑外存的块（页）读写特点
  - 基于外存的外归并排序（External Merge Sort）
    - 基于分治思想，待排序数据集分成多个归并段（run），每归并段独立排序
      - 步骤1：排序
        - 对内存中的归并段排序，结果写回外存文件
      - 步骤2：归并
        - 归并多个已排序文件



排序归并-外归并排序

- **归并段（run）**
  - 一个键/值（Key/Value）列表
  - Key：用于排序的属性列
  - Value：元组ID/元组内容

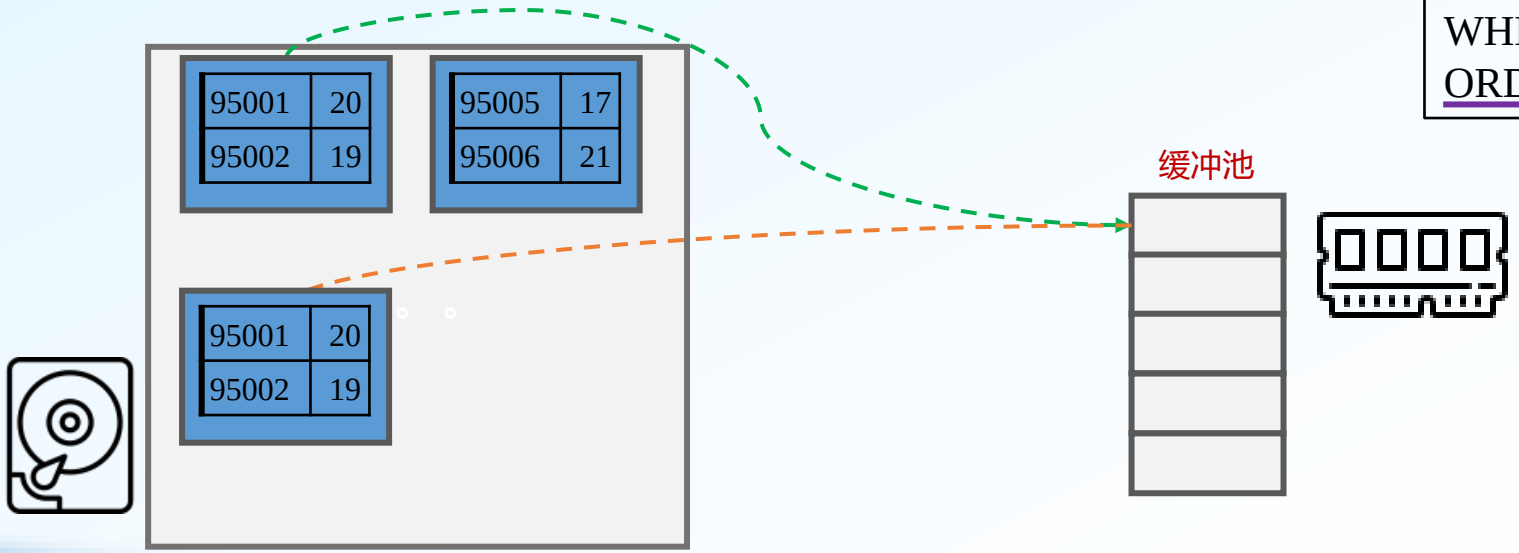




排序归并-2路外归并排序

- **n路外归并排序**
  - **n：每次合并的归并段数量**
  - **设 n=2，完整的用于排序的K/V列表共占M页**

第0趟：
依次读入M个块，在内存中进行排序，排好序的数据依次写回到外存
第1,2,…趟：
递归的合并两个归并段为一个新归并段（长度为原来2倍）
上述过程使用缓冲池中的3个帧（2个用于输入、一个输出）



1
2

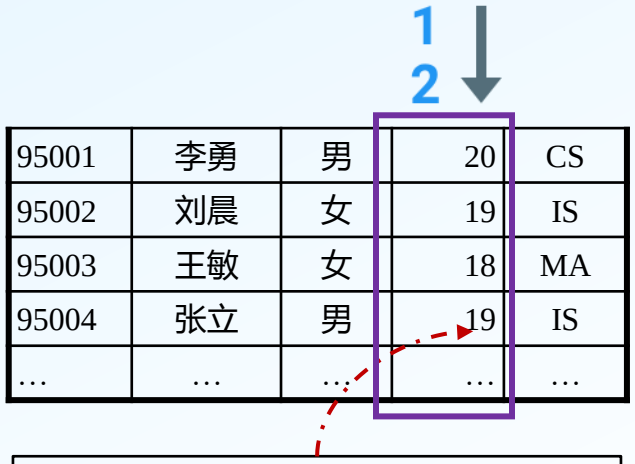
95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
…	…	…	…	…

```
SELECT Sno, age
FROM Student
WHERE department = 'CS'
ORDER BY age DESC;
```

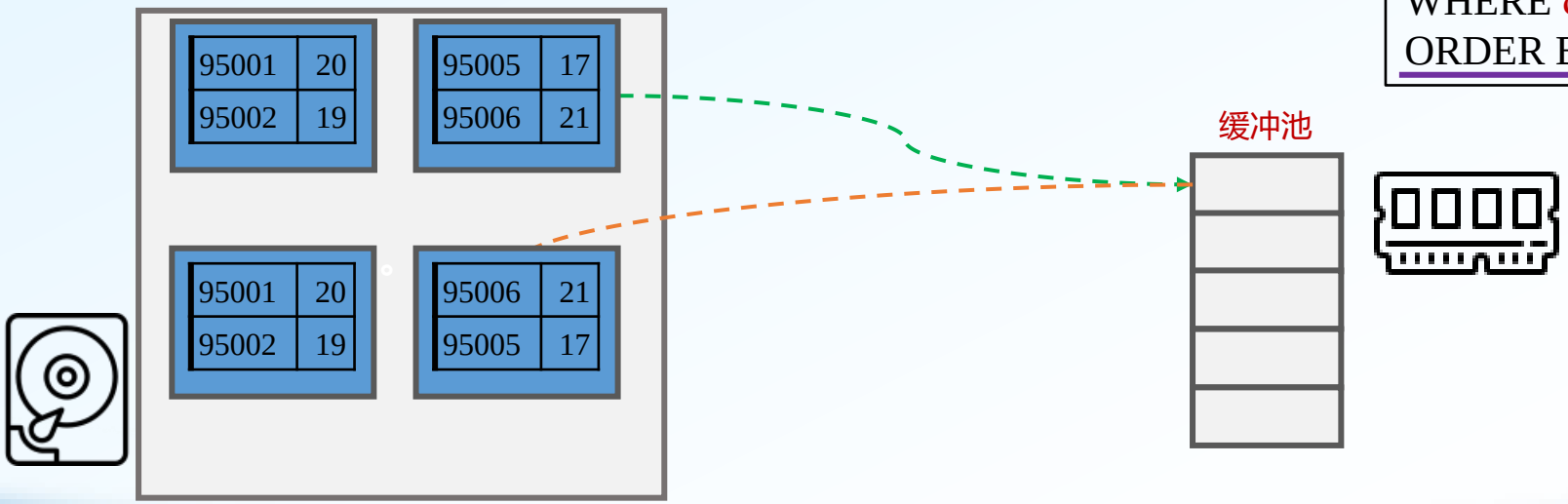
排序归并-2路外归并排序

- **n路外归并排序**
  - **n：每次合并的归并段数量**
  - **设 n=2，完整的用于排序的K/V列表共占M页**

第0趟：
依次读入M个块，在内存中进行排序，排好序的数据依次写回到外存
第1,2,…趟：
递归的合并两个归并段为一个新归并段（长度为原来2倍）
上述过程使用缓冲池中的3个帧（2个用于输入、一个输出）



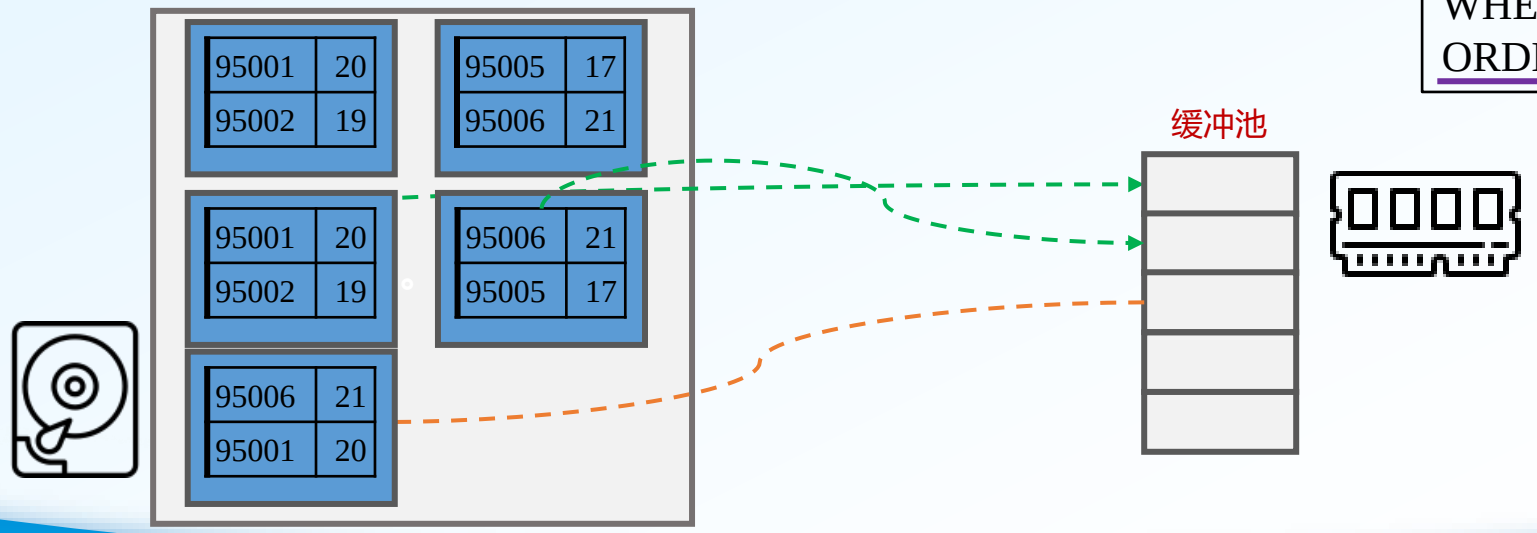

```
SELECT Sno, age
FROM Student
WHERE department = 'CS'
ORDER BY age DESC;
```



排序归并-2路外归并排序

- **n路外归并排序**
  - **n：每次合并的归并段数量**
  - **设 n=2，完整的用于排序的K/V列表共占M页**

第0趟：
依次读入M个块，在内存中进行排序，排好序的数据依次写回到外存
第1,2,…趟：
递归的合并两个归并段为一个新归并段（长度为原来2倍）
上述过程使用缓冲池中的3个帧（2个用于输入、一个输出）



1
2

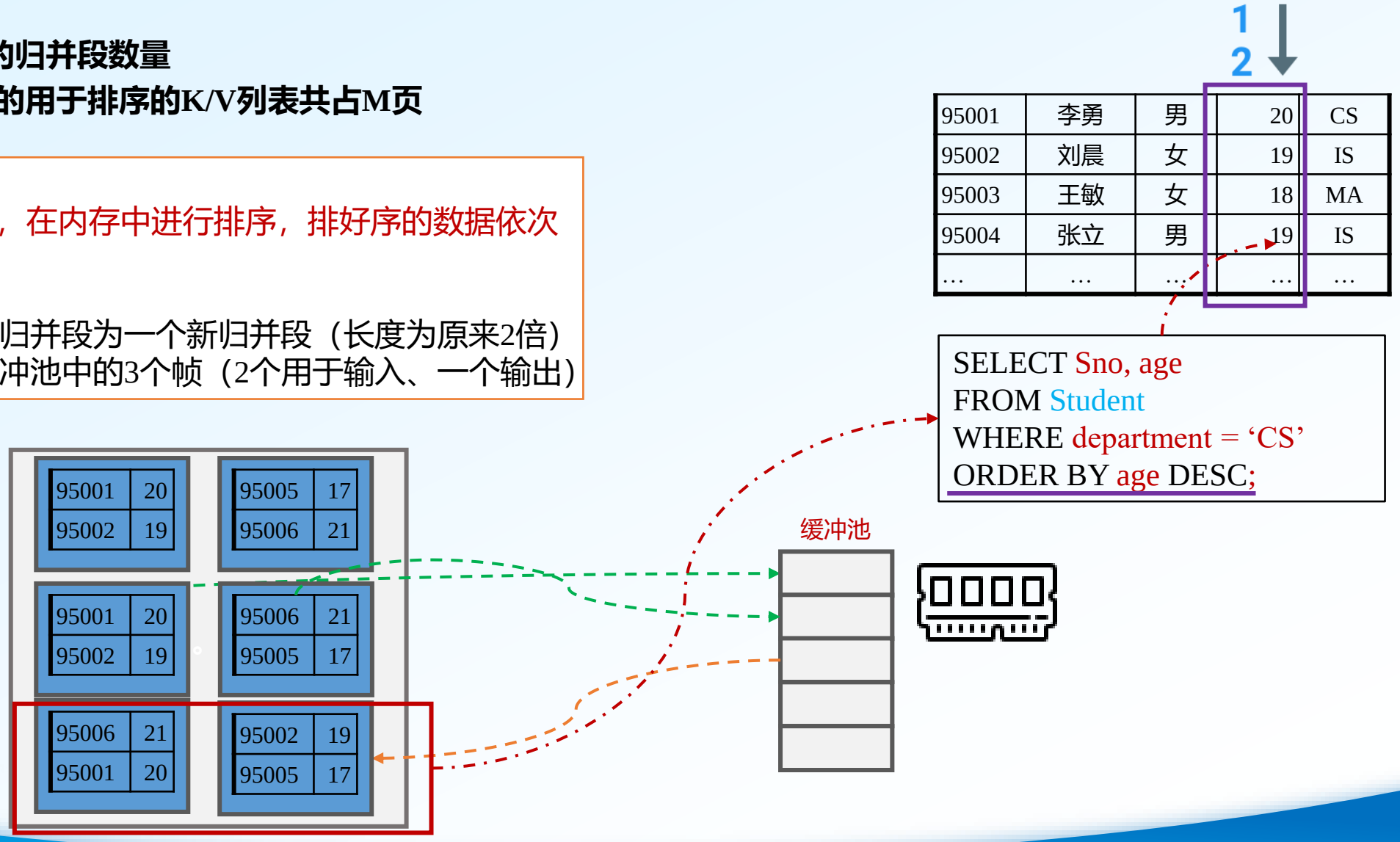
95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
…	…	…	…	…

```
SELECT Sno, age
FROM Student
WHERE department = 'CS'
ORDER BY age DESC;
```

排序归并-2路外归并排序

- **n路外归并排序**
  - **n：每次合并的归并段数量**
  - **设 n=2，完整的用于排序的K/V列表共占M页**

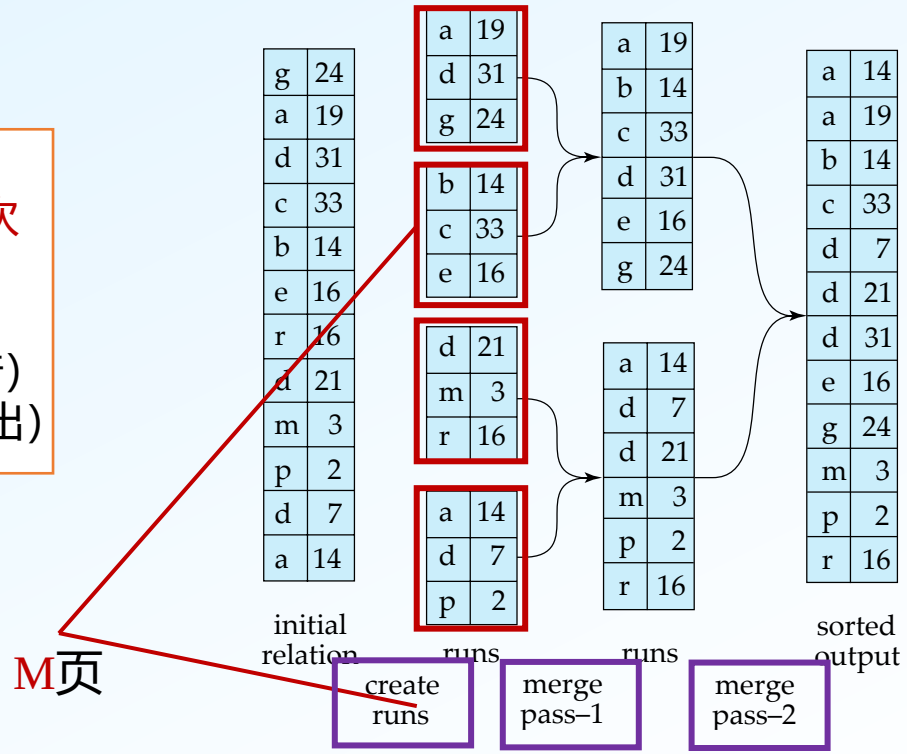
第0趟：
依次读入M个块，在内存中进行排序，排好序的数据依次写回到外存
第1,2,…趟：
递归的合并两个归并段为一个新归并段（长度为原来2倍）
上述过程使用缓冲池中的3个帧（2个用于输入、一个输出）



排序归并-2路外归并排序

第0趟：
依次读入M个块，在内存中进行排序，排好序的数据依次写回到外存
第1,2,…趟：
递归的合并两个归并段为一个新归并段（长度为原来2倍）
上述过程使用缓冲池中的3个帧（2个用于输入、一个输出）

- **I/O数量?**
  - 趟数（树高）： $\mathbf{1 + \lceil \log_2 M \rceil}$
  - 每趟： $\mathbf{2M}$
  - 合计： $\mathbf{2M \times (1 + \lceil \log_2 M \rceil)}$



(每页读入->排序->写回外存)

哈希连接

- **哈希连接 Hash Join**
  - 扫描内表以创建基于连接键的哈希结构（这是索引的一种形式）；
  - 扫描外表，对于每个符合条件的记录，使用哈希结构连接内表。

95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
95005	陈刚	男	19	CS

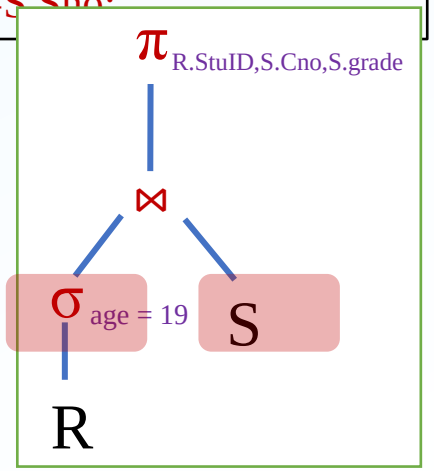
95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58

95002	刘晨	女	19	IS
95004	张立	男	19	IS
95005	陈刚	男	19	CS

⋈

```
SELECT R.StuID, S.Cno,
S.grade
FROM Student R, SC S
WHERE age = 19 AND
R.StuID=S.Sno;
```

95002	刘晨	女	19	IS	CS2012	92
95004	张立	男	19	IS	CE2001	86
95004	张立	男	19	IS	CE2004	78
95005	陈刚	男	19	CS	CS2031	58



- **步骤1：构造**
  - **扫描R表，在连接码上用哈希函数*h*将R中的元组映射到不同的桶**
- **步骤2：探查**
  - **扫描S表，用*h*将S中的元组找到对应的桶，在对应的桶内进行匹配**

哈希连接

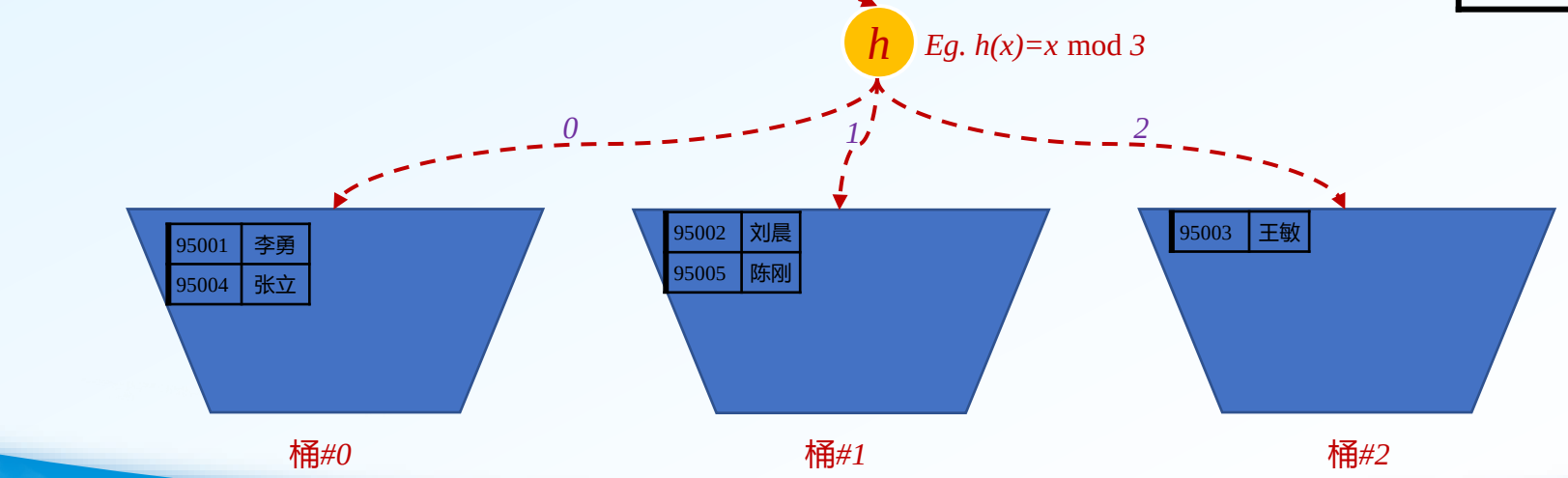
步骤1：构造

扫描R表，在连接码上用哈希函数*h*将R中的元组映射到不同的桶

95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
95005	陈刚	男	17	CS

⋈

95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58



哈希连接

步骤1：构造

扫描R表，在连接码上用哈希函数*h*将R中的元组映射到不同的桶

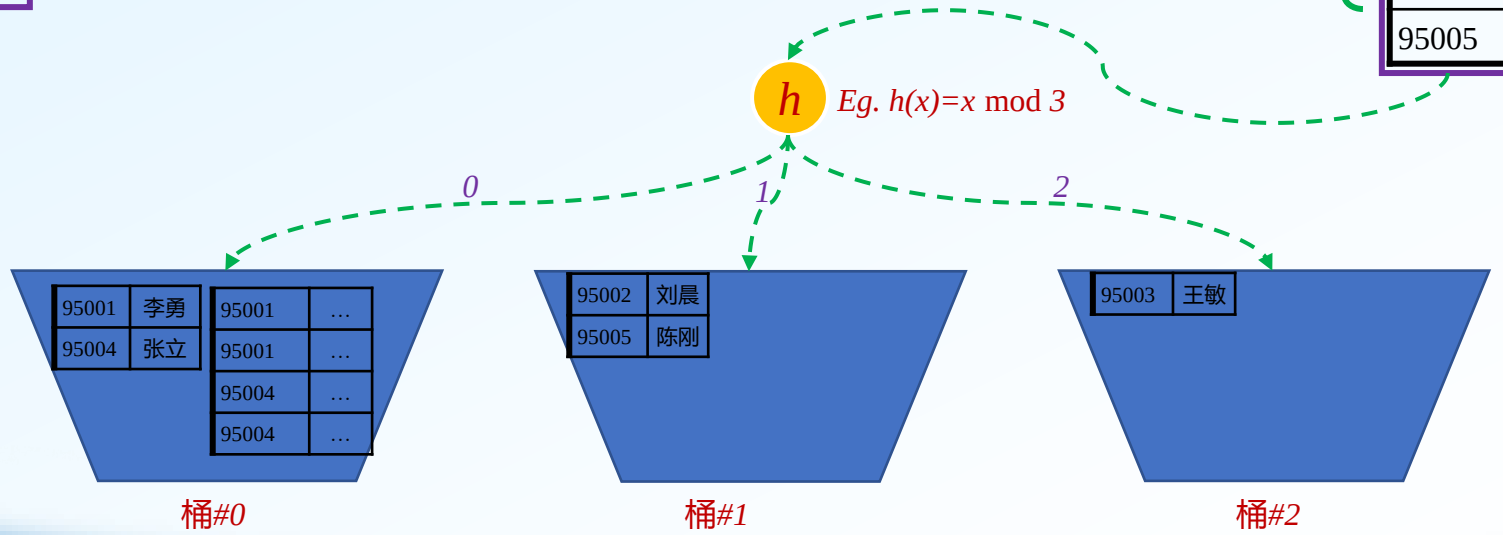
步骤2：探查

扫描S表，用*h*将S中的元组找到对应的桶，在对应的桶内进行匹配

95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
95005	陈刚	男	17	CS

⋈

95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58



哈希连接

步骤1：构造

扫描R表，在连接码上用哈希函数*h*将R中的元组映射到不同的桶

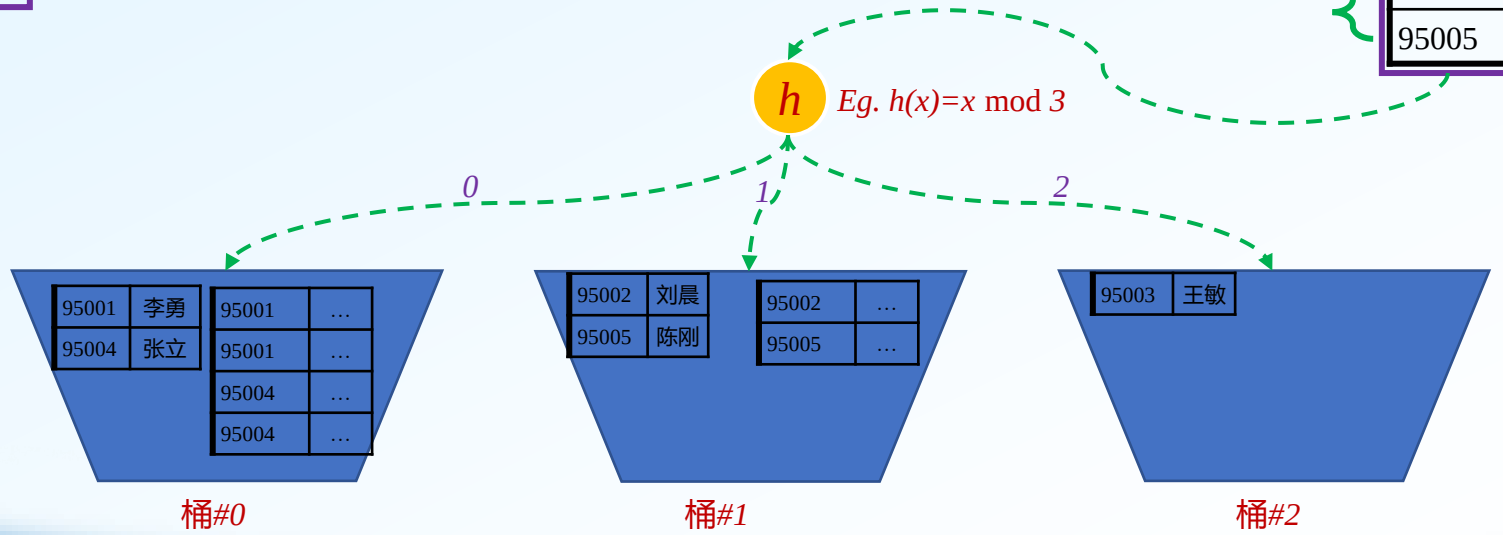
步骤2：探查

扫描S表，用*h*将S中的元组找到对应的桶，在对应的桶内进行匹配

95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
95005	陈刚	男	17	CS

⋈

95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58



哈希连接

步骤1：构造

扫描R表，在连接码上用哈希函数*h*将R中的元组映射到不同的桶

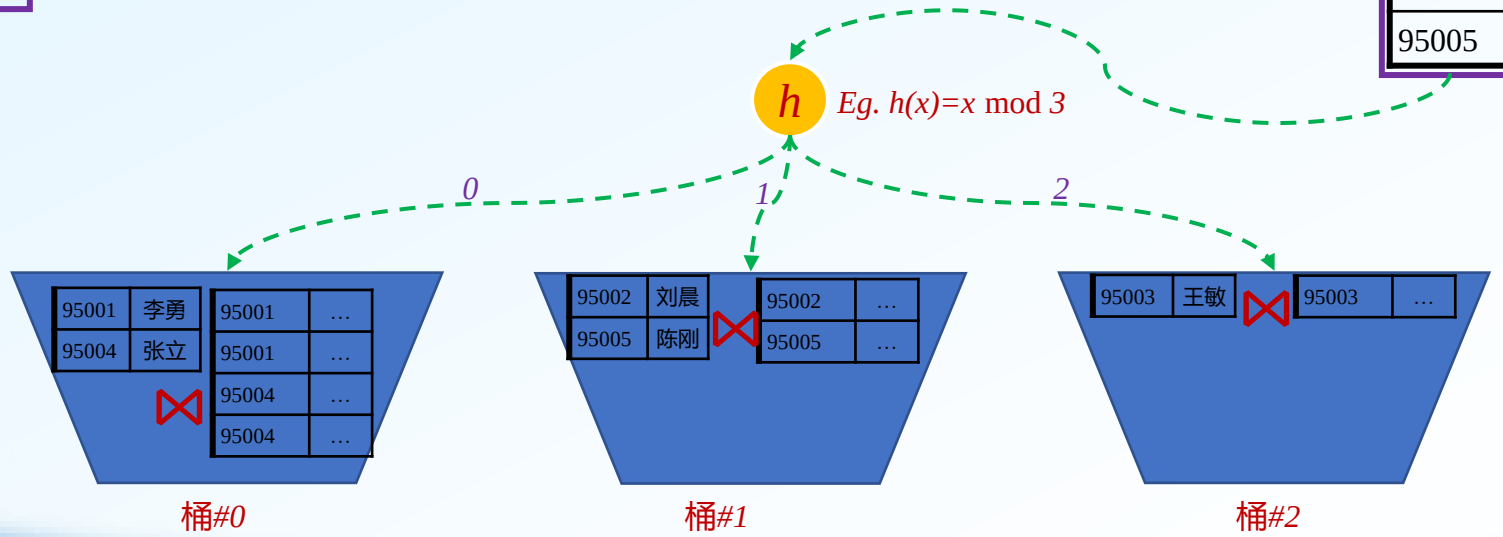
步骤2：探查

扫描S表，用*h*将S中的元组找到对应的桶，在对应的桶内进行匹配

95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
95005	陈刚	男	17	CS

⋈

95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58

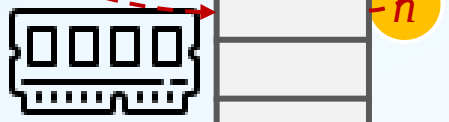


哈希连接

◆ 如果表太大不能一次放入内存?

95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
95005	陈刚	男	17	CS

I/O次数:
2B(R)



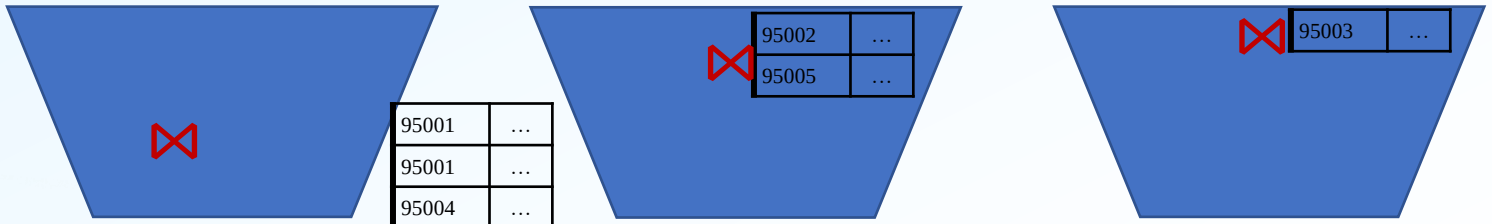
缓冲池



Eg. h(x)=x mod 3



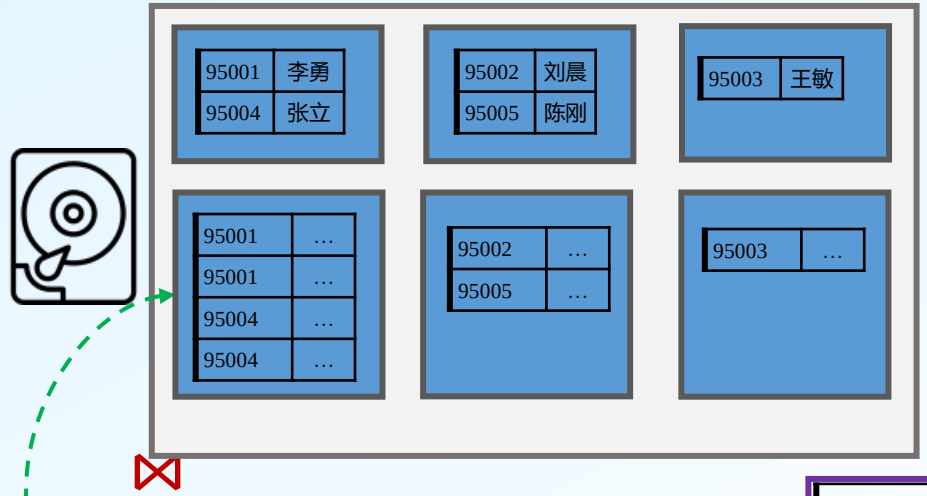
95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58



哈希连接

◆ 如果表太大不能一次放入内存?

95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
95005	陈刚	男	17	CS



I/O次数:
2B(R)+2B(S)

缓冲池

h

Eg. h(x)=x mod 3

95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58

桶#0　桶#1　桶#2

哈希连接

◆ 如果表太大不能一次放入内存?

95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
95005	陈刚	男	17	CS



95001	李勇
95004	张立

95002	刘晨
95005	陈刚

95003 王敏

95001	…
95001	…
95004	…
95004	…

95002	…
95005	…

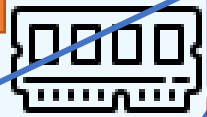
95003 …

⋈

95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58

I/O次数:
2B(R)+2B(S)

缓冲池



h *Eg. h(x)=x mod 3*

输出

95001	李勇	男	20	CS	CS2002	80
95001	李勇	男	20	CS	CS2012	78
95004	张立	男	19	IS	CE2001	86
95004	张立	男	19	IS	CE2004	78

95001	李勇
95004	张立

95001	…
95001	…
95004	…
95004	…

⋈

⋈

⋈

桶#0

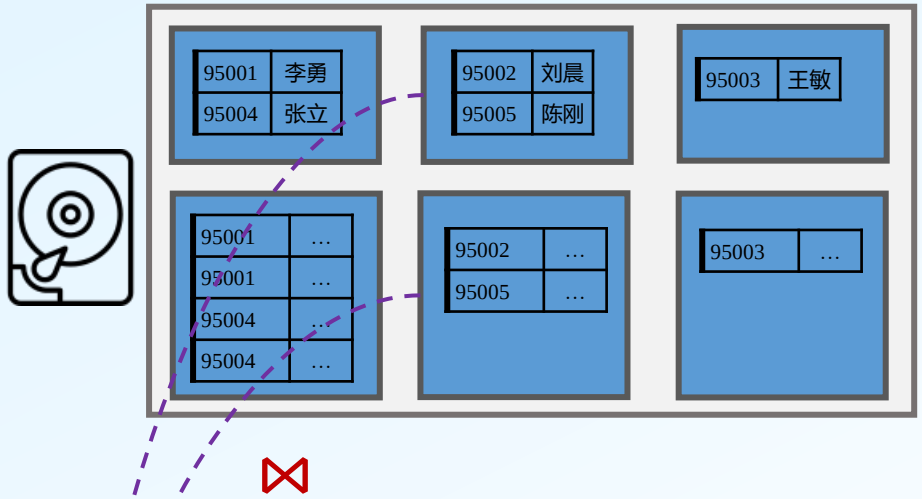
桶#1

桶#2

哈希连接

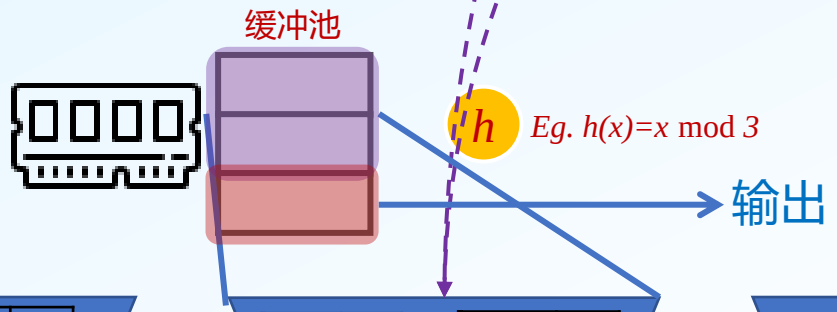
◆ 如果表太大不能一次放入内存?

95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
95005	陈刚	男	17	CS

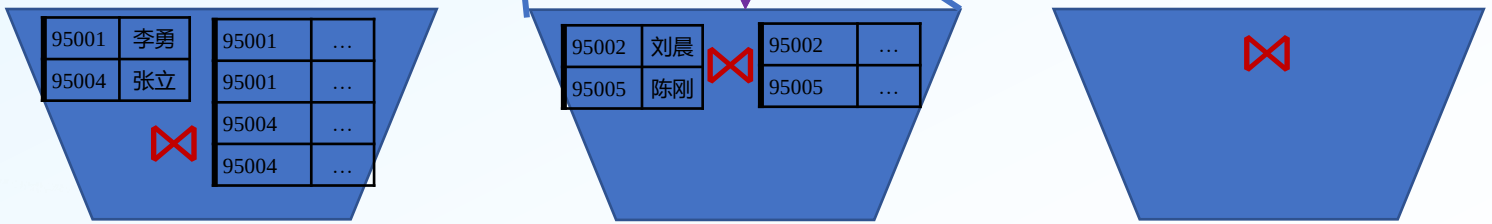


95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58

I/O次数:
2B(R)+2B(S)



95001	李勇	男	20	CS	CS2002	80
95001	李勇	男	20	CS	CS2012	78
95004	张立	男	19	IS	CE2001	86
95004	张立	男	19	IS	CE2004	78
95002	刘晨	女	19	IS	CS2012	92
95005	陈刚	男	17	CS	CS2031	58

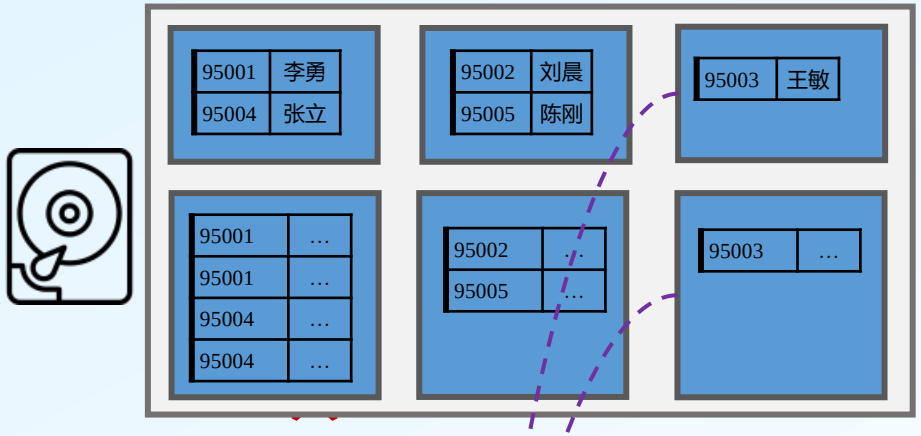


桶#0 桶#1 桶#2

哈希连接

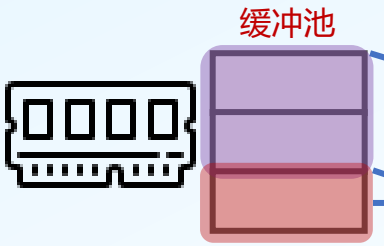
◆ 如果表太大不能一次放入内存?

95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS
95005	陈刚	男	17	CS



95001	CS2002	80
95001	CS2012	78
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
95004	CE2004	78
95005	CS2031	58

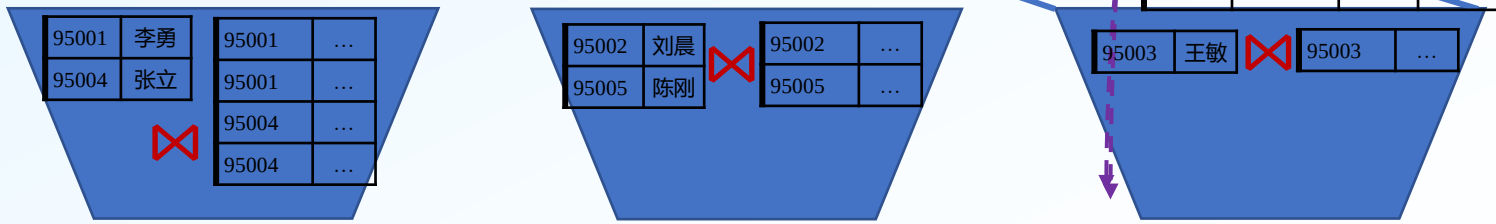
I/O次数:
2B(R)+2B(S)
+B(R)+B(S)
=3(B(R)+B(S))



h *Eg.* $h(x)=x \bmod 3$

输出

95001	李勇	男	20	CS	CS2002	80
95001	李勇	男	20	CS	CS2012	78
95004	张立	男	19	IS	CE2001	86
95004	张立	男	19	IS	CE2004	78
95002	刘晨	女	19	IS	CS2012	92
95005	陈刚	男	17	CS	CS2031	58
95003	王敏	女	18	MA	CS2002	67



桶#0 桶#1 桶#2

嵌套循环 vs. 排序合并 vs. 哈希连接

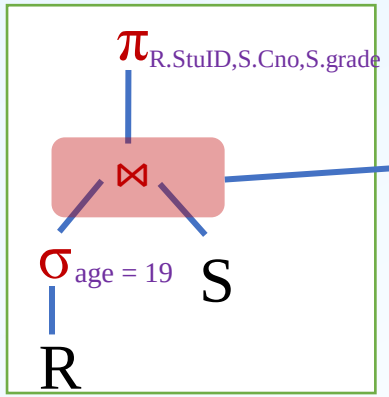
Recall: Index Scan vs. Table Scan

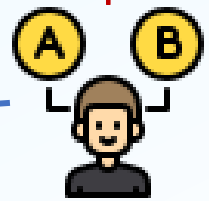
选择率$S_p(R)=\frac{T(\sigma_p(R))}{T(R)}$有多大?

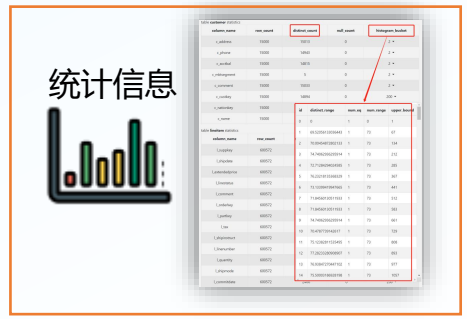
谓词	嵌套循环	排序合并	哈希连接
<, >, ≤, ≥	B(R)+⌈B(R)/(#Buffer-2)⌉*B(S)	NA	NA
=	B(R)+⌈B(R)/(#Buffer-2)⌉*B(S)	B(R)+B(S) +2B(R)(**1**+ ⌈$\log_2 \boldsymbol{B}(\boldsymbol{R})$⌉) +2B(S)(**1**+ ⌈$\log_2 \boldsymbol{B}$(S)⌉)	3(B(R)+B(S))

```
SELECT R.StuID, S.Cno, S.grade
FROM Student R, SC S
WHERE age = 19 AND R.StuID=S.Sno;
```

B(R')、B(S)、#Buffer有多大？？？







总结

- **范围谓词**
  - 嵌套循环
- **=谓词**
  - 如果R或S已经有序（连接属性上）
    - 排序合并
      - B(R)+B(S)+~~2B(R)(1+ ⌈log2*B*(*R*)⌉)~~+2B(S)(1+ ⌈log2*B*(S)⌉)
      - B(R)+B(S)+2B(R)(1+ ⌈log2*B*(*R*)⌉)+~~2B(S)(1+ ⌈log2*B*(S)⌉)~~
      - B(R)+B(S)+~~2B(R)(1+ ⌈log2*B*(*R*)⌉)+2B(S)(1+ ⌈log2*B*(S)⌉)~~
  - 大多数其他情况下
    - 哈希连接
  - DBMS需要进行代价估计来决定实际物理操作符的选择
    - 依靠统计信息和代价模型

Hash Agg. & Sort Agg.

- **SUM ( ) /MAX ( ) /MIN()/…GROUP BY的物理实现**
  - 同一属性同值元组分组
    - “同值” 类比于“=谓词”
    - “=谓词”：Hash Join、Sort Merge
  - 在每个分组上应用聚集函数输出一个标量
- **哈希聚集 Hash Aggregator**
  - 在扫描目标表时建立一个临时（关于聚集函数目标列的）哈希表；
  - 对于每个扫描到的记录，检查哈希表中是否存在既有值，并进行相应的聚集函数运算
- **排序聚集 Sort Aggregator**
  - 先对（聚集函数的）目标列排序；
  - 在排序的基础上合并同值记录为分组，对分组进行聚集函数计算

95001	CS2002	80
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
…	…	…

```
SELECT Sno,AVG(grade)
FROM SC
GROUP BY Sno;
```

去重、投影、外连接

- 去重
  - 参考GROUP BY
    - 基于SORT
    - 基于HASH
- 投影
  - 每个元组投影
  - 去重
- 集合运算（并、交、差）
  - R上建立哈希索引
  - S上探查
    - 补充、保留、移除
- 外连接
  - 先计算内连接、然后扩充元组
  - 直接对三种连接操作进行修改

95001	CS2002	80
95002	CS2012	92
95003	CS2002	67
95004	CE2001	86
…	…	…

```
SELECT Sno,AVG(grade)
FROM SC
GROUP BY Sno;
```

本章内容

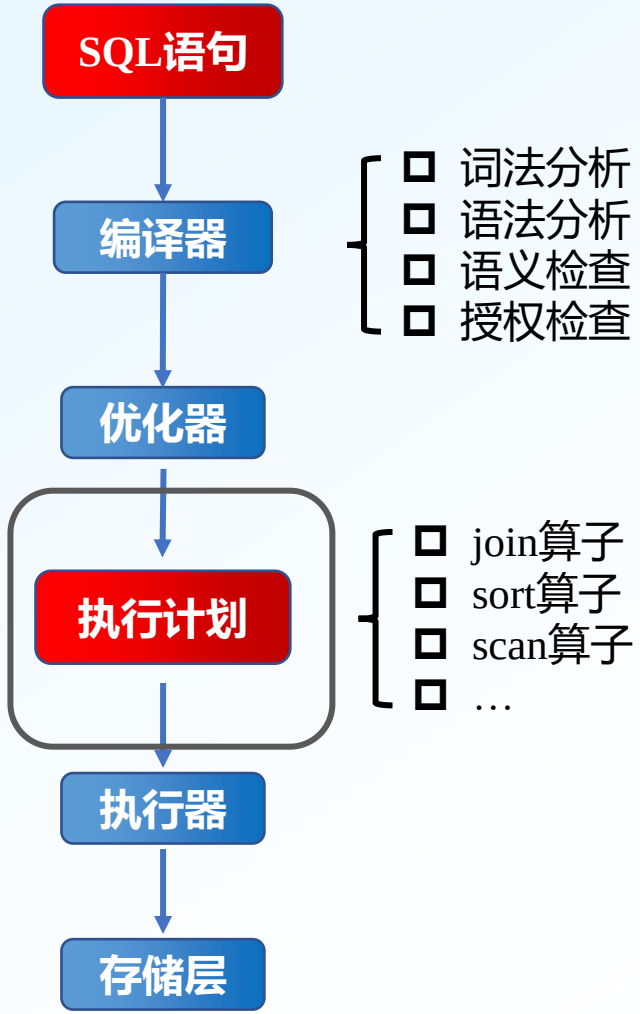


一、查询解析

二、逻辑重写

三、物理算子

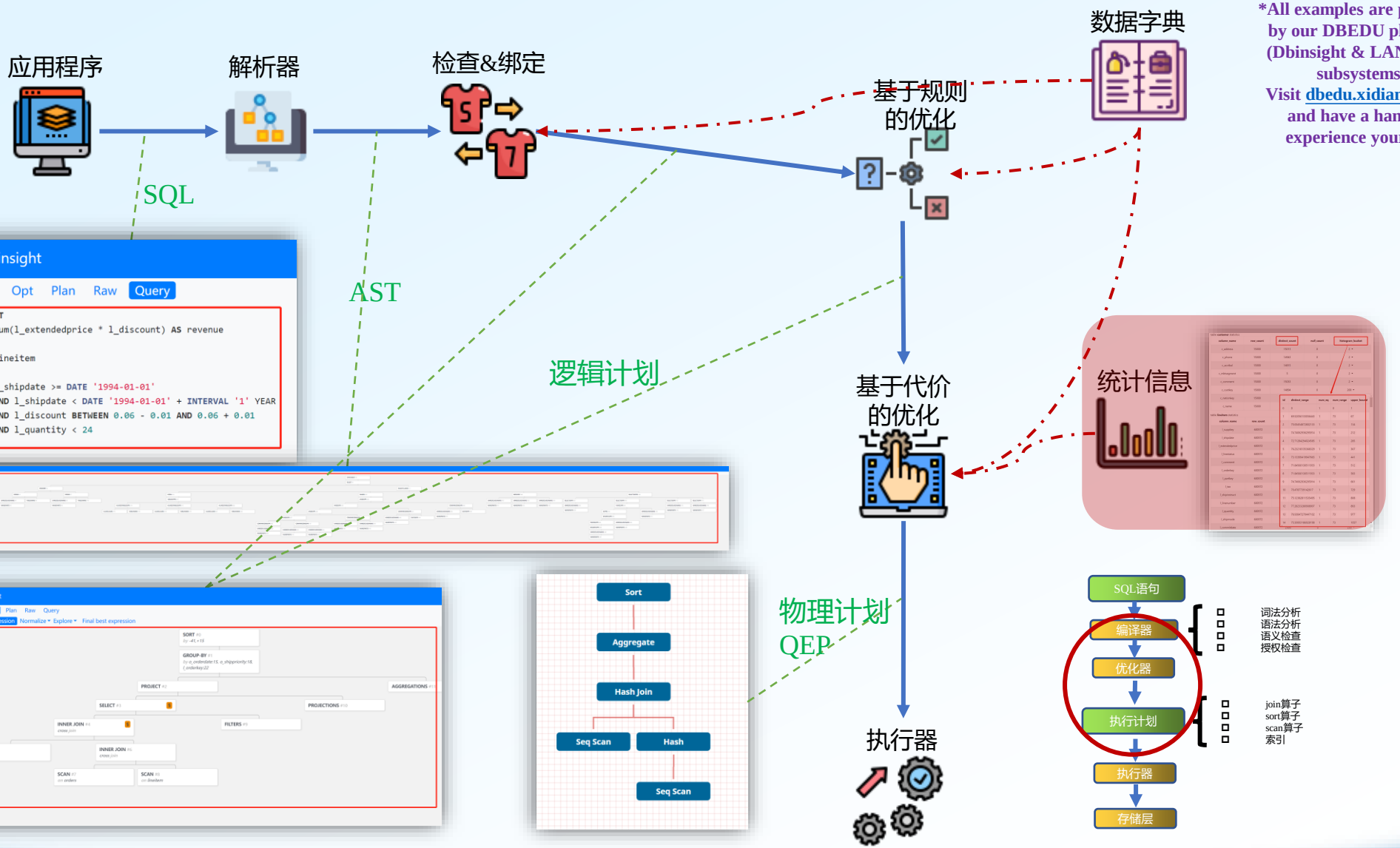
四、查询优化



SQL引擎pipeline

*All examples are provided by our DBEDU platform (Dbinsight & LANTERN subsystems) Visit dbedu.xidian.edu.cn and have a hand-on experience yourself!

DBEDU



RBO & CBO

- **基于规则的优化（Rule-based Optimization, RBO）**
  - 查询的逻辑重写，以尽可能减少不合理的开销
  - 基于（关系代数的等价变换）规则
  - 调整操作顺序
  - 生成：由逻辑运算符组成的逻辑计划（树）
    - 逻辑运算符：关系代数的算子（π，σ，⋈，…）
  - 与关系实例无关
- **基于代价的优化（Cost-based Optimization, CBO）**
  - 查询的物理优化
  - 使用代价模型，对操作进行代价估计
  - 衡量所有可能的执行方式，选择执行代价最小的
  - 生成：由物理操作符组成的物理执行计划（树）
    - 物理操作符：关系代数算子的具体物理实现 （1 -> n映射）
  - 与关系实例相关
    - 需要使用实例的统计信息

本节内容

上节内容

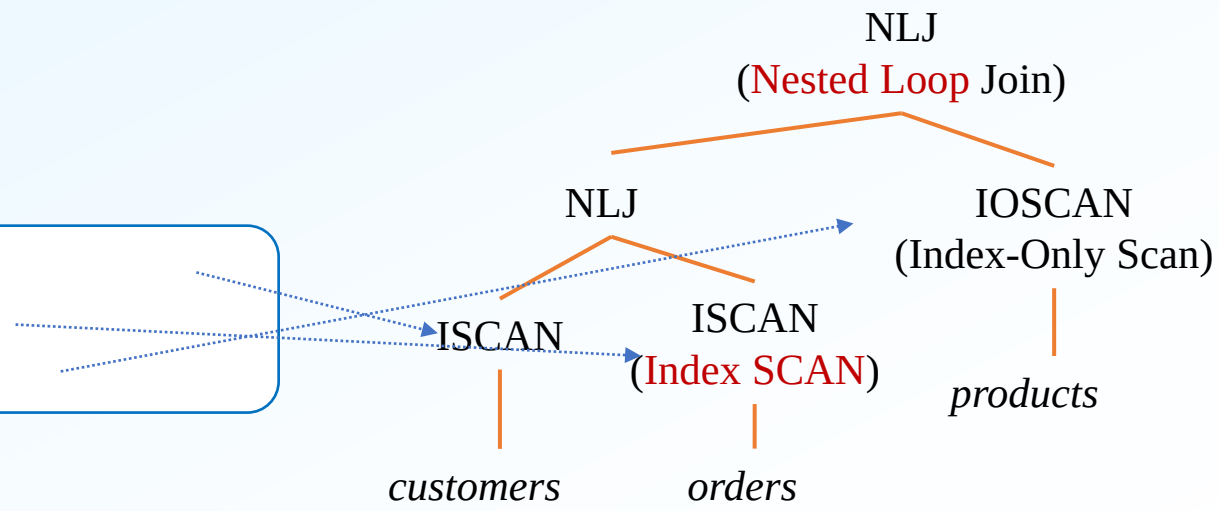
访问计划

■ **一个SQL查询的示例:**

```
select o.date,o.quantity,p.name
from customers c,orders o,products p
where c.id = ?
        and c.id = o.id
        and p.id = o.pid
        and o.date between ? and ?
order by o.date
```

- **访问方法:** 数据访问算法
  - 表扫描
  - 索引扫描、仅索引扫描及其它
- **连接方法:** 连接两张表的算法
  - 查找联接;
  - 合并连接;
  - 哈希连接。
- **访问计划:** 为了执行一条SQL查询语句而将一系列访问方法和连接方法缝合在一起。

customers: index on cid
orders: index on cid, date
products: index on id, name



➢ 查询范例：

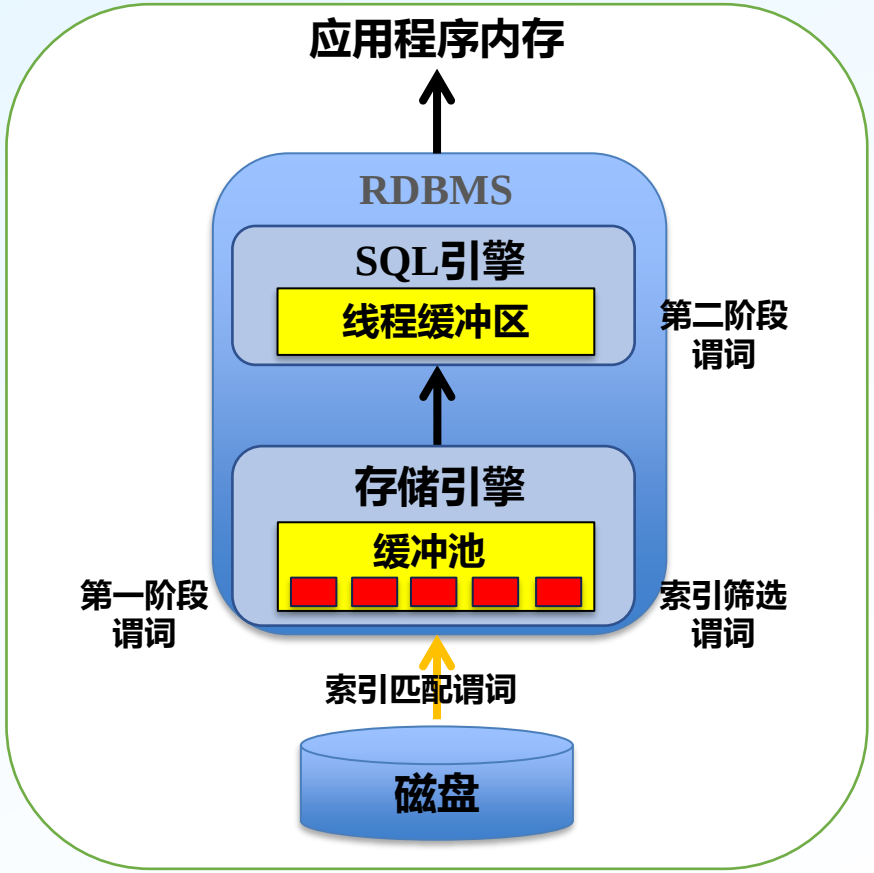
```
SELECT o.date, o.quantity, p.name
FROM customers c,                    (10,000,000)
     orders o,                       (50,000,000)
     products p                      (10,000)
WHERE c.id = ?                       ⟹ o.cid = ?
  AND c.id = o.cid
  AND p.id = o.pid
  AND o.date BETWEEN ? AND ?
ORDER BY o.date
```

➢ 表的索引：

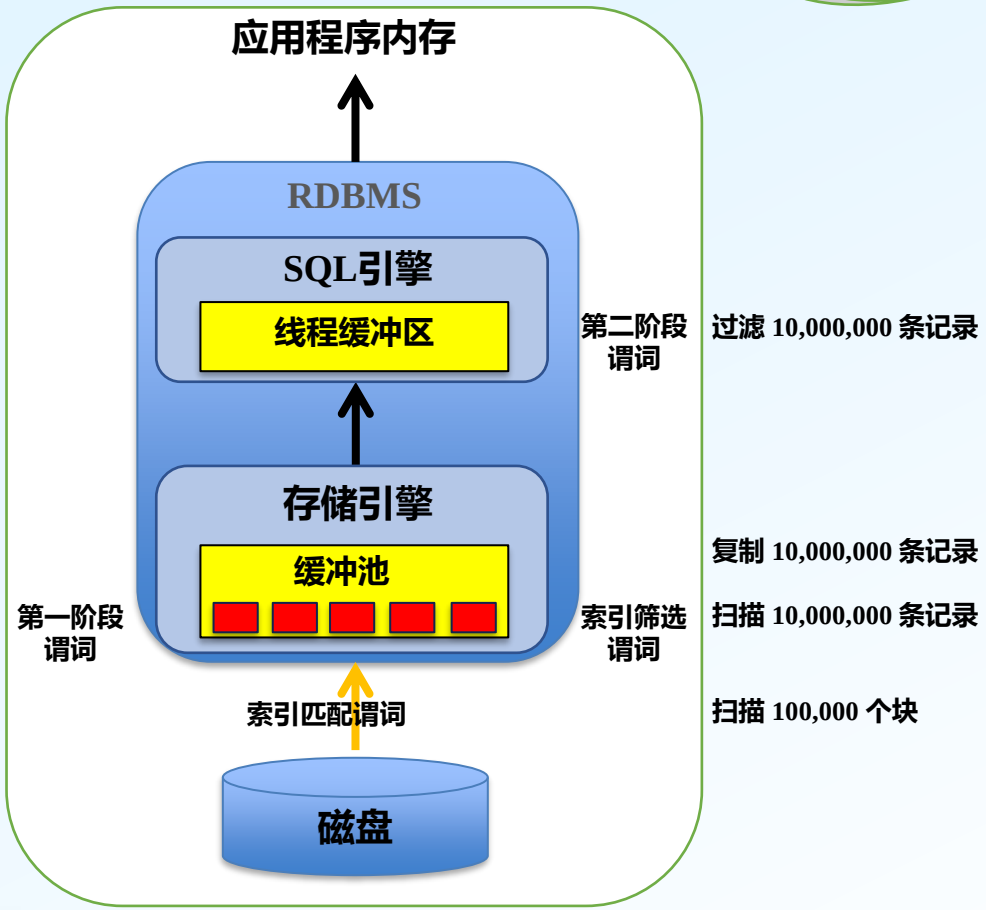
customers: id
orders: cid, date
products: id, name

➢ c - o - p 性能分析：

- ➢ 如果 '*c.id* = ?' 是第二阶段谓词?



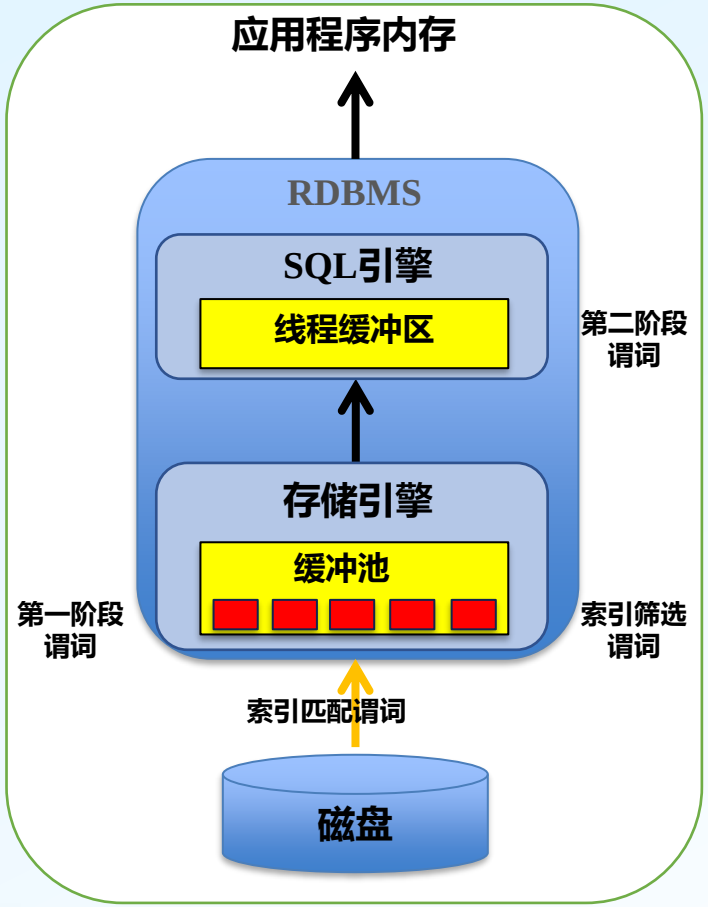
- c - o - p 性能分析：
  - 如果 ‘c.id = ?’ 是第二阶段谓词?



- c表只能用全表扫描(table scan)：
  - 扫描 100,000 个 I/O 块
  - 扫描 10,000,000 条记录
  - 复制 10,000,000 条记录
  - 过滤 10,000,000 条记录
  - 最终剩下 1 条记录

- 开销估算：
  - I/O 开销：1ms × 100,000 = 100秒
  - CPU 开销：
    - 存储引擎：5us × 10,000,000 = 50秒
    - 复制开销：5us × 10,000,000 = 50秒
    - 过滤开销：5us × 10,000,000 = 50秒
    - 总CPU开销：150秒
  - 总开销：250秒

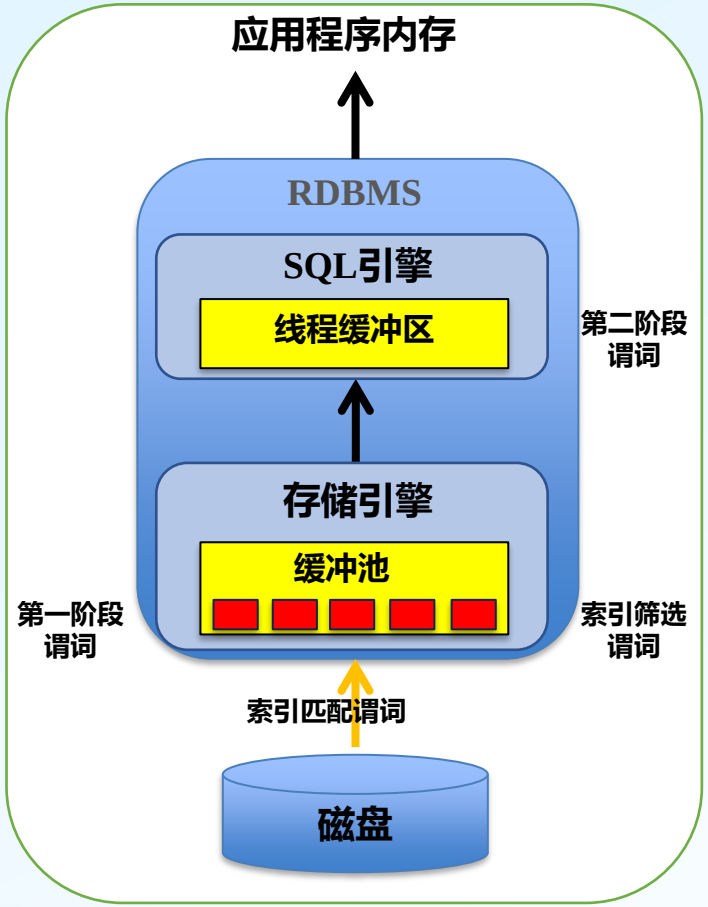
- c - o - p 性能分析：
  - 如果 ‘c.id = ?’ 是第一阶段谓词?



- c表只能用全表扫描(table scan)：
  - 扫描 100,000 个 I/O 块
  - 扫描 10,000,000 条记录
  - 复制 10,000,000 条记录
  - 复制 1 条记录
  - 最终剩下 1 条记录

- 开销估算：
  - I/O 开销：1ms × 100,000 = 100秒
  - CPU 开销：
    - 存储引擎：5us × 10,000,000 = 50秒
    - 过滤开销：5us × 10,000,000 = 50秒
    - 复制开销：5us × 1 = 5us
    - 总CPU开销：100秒
  - 总开销：200秒

- c - o - p 性能分析：
  - 如果 'c.id = ?' 是索引匹配谓词?



复制 1 条记录
扫描 1 个键(key)和 1 条记录

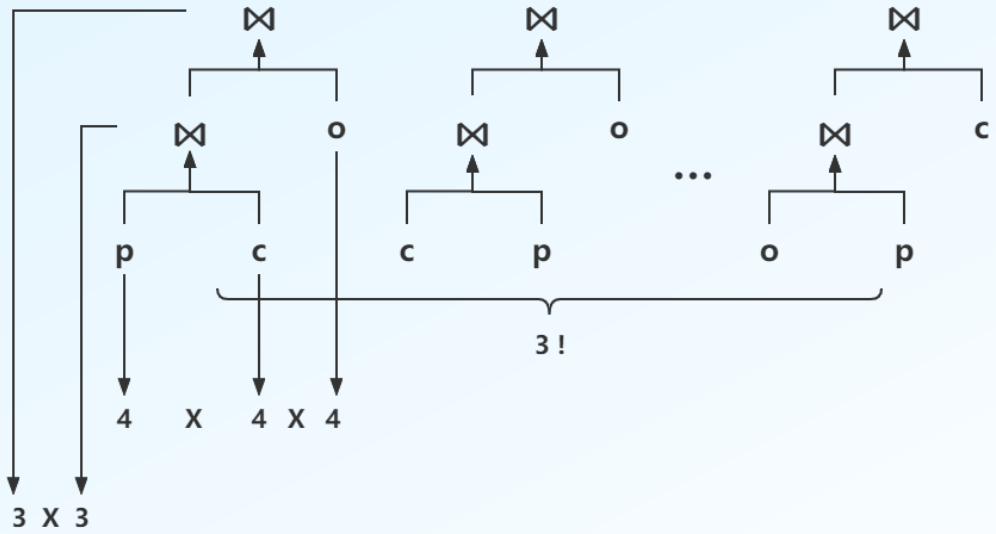
扫描 5+1 = 6 个块

- c表只能用索引扫描(index scan)：
  - 扫描 4+1 = 5 个索引 I/O 块
  - 扫描 1 个索引键
  - 扫描 1 个表(table)的 I/O 块
  - 扫描 1 条记录
  - 复制 1 条记录，最终剩下 1 条记录

- 开销估算：
  - I/O 开销：1ms × (5+1) = 6ms
  - CPU 开销：
    - 存储引擎：5us × (1+1) = 10us
    - 复制开销：5us × 1 = 5us
    - 总CPU开销：15us
  - 总开销：6ms

访问路径

- **SQL是一种声明式语言而不是过程式语言！**
  - 连接操作是结合的和交换的 - 搜索空间大
  - 关系可以顺序访问，也可以通过索引访问 - 访问方法多样
  - 关系可以对称连接，也可以定向连接 - 连接方法多样



3! = 6 连接序列

假设每个表（p,c,o共3个表）有3个索引，底层RDBMS支持3个连接算法

访问路径总数：

3! X (4 x 4 x 4) x (3 x 3) = 3,456

访问路径

- SQL是一种声明式语言而不是过程式语言！
- 一般来说，一个有*n*表连接的查询块可能有

$$n! \cdot (i+1)^n \cdot j^{n-}$$

个访问路径，假设底层RDBMS支持 *j* 个连接算法，并且每个表平均有 *i* 个索引。

- 查询重写、查询并行和其他优化技术可以解锁更多的访问路径。
- SQL语言的查询优化是一个 NP-Hard 问题！

查询优化器组件

- **SQL语言的查询优化是一个NP-Hard问题！**

- **优化过程：选择最优的访问路径：**
  - **① 需要解决以下几个基本问题**
    - **代价估算；**
    - **连接顺序选择**
    - **物理算子选择**
  - **② 枚举访问路径**

统计信息和开销模型

- **开销模型（Cost model）：**
  - DBMS用于从众多访问路径中选择执行计划的基础
  - 估算代价需要了解计算过程的各个步骤以及每个步骤处理的数据量；
  - 总开销等于各个步骤开销之和；
  - 总开销等于I/O开销和CPU开销之和；
    - I/O开销包括索引 I/O 和 数据 I/O；
    - CPU开销包括索引 的CPU开销、缓冲池中处理数据的CPU开销；
  - I/O开销与CPU开销都与每个步骤处理的数据量密切相关，因此需要用到基数估算和选择率估计

$$cost = \sum_i cost_i = cost_i^{I/O} + cost_i^{CPU}$$

  - I/O开销中需要区分顺序I/O和随机I/O

统计信息和开销模型

- **开销模型（Cost model）：**
  - I/O开销与CPU开销都与每个步骤处理的数据量密切相关，因此需要用到基数估算和选择率估计

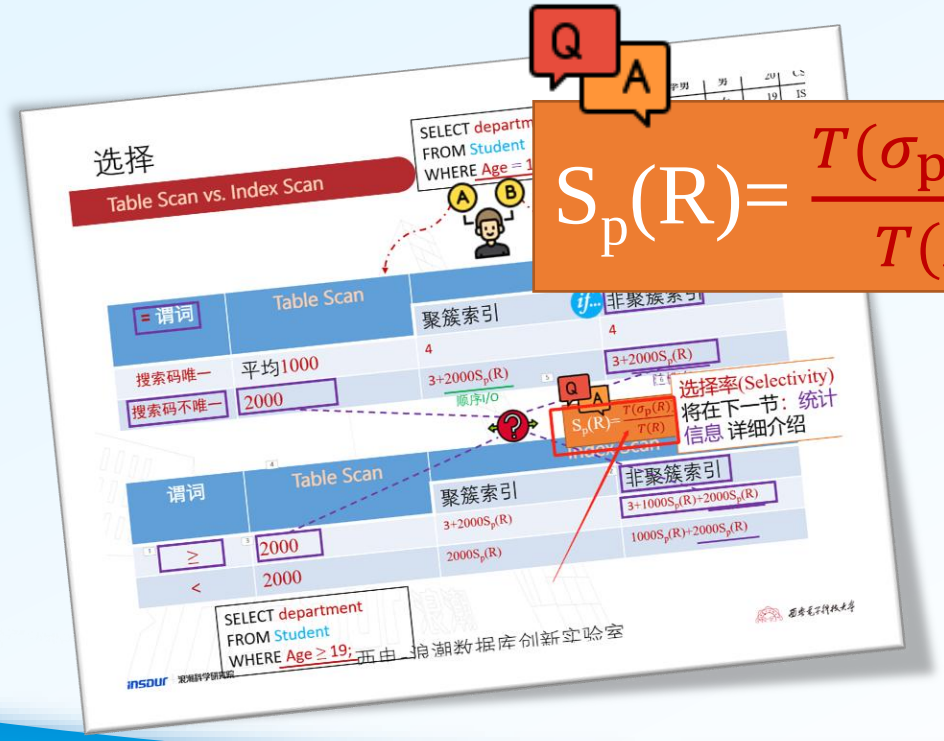
$$cost = \sum_i cost_i = cost_i^{I/O} + cost_i^{CPU}$$

  - 开销模型中
    - CPU和I/O开销的标量归一
    - 需要区分顺序I/O和随机I/O
    - 为了选择唯一的执行计划，需要将右侧归一为标量
      - 可采用相对比例
        - E.g. $cost^{I/O}=400*cost^{RAM}$； $cost^{Rand\ I/O}=4*cost^{Seq\ I/O}$; …
      - 或绝对大小参考值
        - 部署调用微评测基准探测
          - CPU时钟
          - 存储介质物理特性

统计信息和开销模型

■ 基本框架

- SQL优化器旨在对查询语句的等价计划进行开销估算，寻找最低查询成本的执行计划
- 开销模型使用收集到的**统计信息**以及构造的不同**开销公式**，估计某个特定查询计划的成本，帮助优化器从众多备选方案中找到开销最低的计划。



$$S_p(R)=\frac{T(\sigma_p(R))}{T(R)}$$

所有问题的答案！



统计信息和开销模型

➢ 统计信息

Q

A

DBMS 会自动收集所有索引列以及最多k

g. k=100）个非索引列的统计信息，以

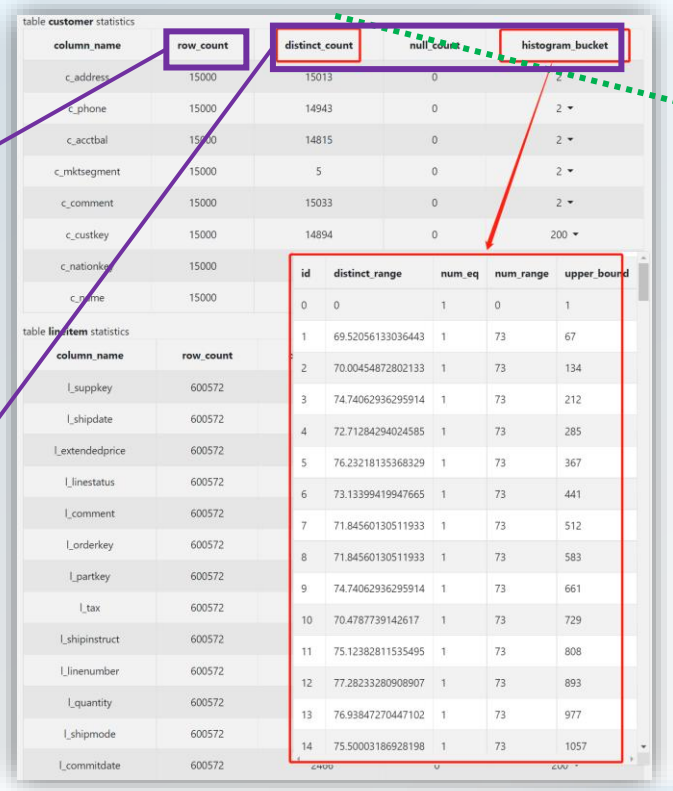
$$S_p(R)=\frac{T(\sigma_p(R))}{T(R)}$$

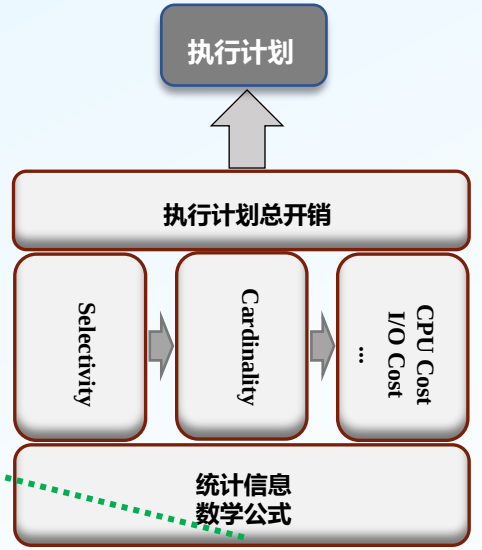
标数据特征的

估算。DBMS维护（ANALYZE）的统计

信息包括：

- ➢ 表的统计信息：总行数 T(R)；
- ➢ 列的统计信息：
  - ➢ 不同值的数目 Card(R,A)
  - ➢ NULL值的数目 Null(R,A)
  - ➢ 直方图 Hist(R,A)





统计信息和开销模型

- **选择率**
  - 满足谓词条件p的元组在全部元组中的占比
  - DBMS维护（ANALYZE）的统计信息：
    - 表的统计信息：总行数 T(R)；
    - 列的统计信息：
      - 不同值的数目Card(R,A)
      - NULL值的数目Null(R,A)
      - 直方图 Hist(R,A)
  - 如何计算$S_p(R)$?

$$S_p(R)=\frac{T(\sigma_p(R))}{T(R)}$$

95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	CS
95005	陈刚	男	17	CS
95006	赵珊	女	19	IS

```
SELECT *
FROM Student
WHERE StuID = '95002';
```

```
SELECT *
FROM Student
```

均匀分布 OR not?

```
SELECT *
FROM Student
```

属性独立 OR not?

```
SELECT *
FROM Student
WHERE Sage < 17 AND Ssex='男' AND
Department='CS' AND Sname LIKE '李%';
```

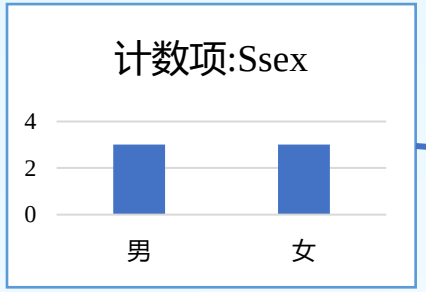
Card(R, A)	$S_p(R)$?
$T(R)$	$\frac{1}{Card(R,A)}=\frac{1}{T(R)}$
2	$\frac{1}{Card(R,A)}=\frac{1}{2}$
4	$\frac{1}{Card(R,A)}=\frac{1}{4}$
	$S_{p1}(R)*S_{p2}(R)*S_{p3}(R)*S_{p4}(R)$

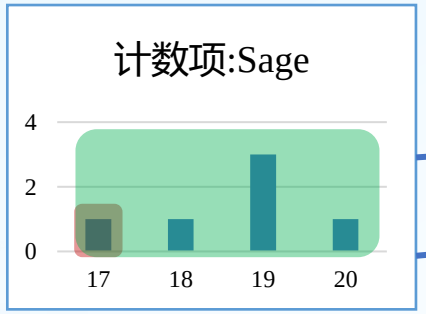
统计信息和开销模型

- **统计直方图**
  - **直方图Hist(R,A)**
  - **如何计算** $S_p(R)$?

95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	CS
95005	陈刚	男	17	CS
95006	赵珊	女	19	IS

$$S_p(R)=\frac{T(\sigma_p(R))}{T(R)}$$






```
SELECT *
FROM Student
WHERE StuID =
'95002';
```

```
SELECT *
FROM Student
```

均匀分布 OR not?

```
SELECT *
FROM Student
WHERE Sage < 18;
```

$S_p(R)=\frac{\text{红}}{\text{绿}}=1/6$

Card(R, A)	$S_p(R)$?
$T(R)$	$\frac{1}{Card(R,A)}=\frac{1}{T(R)}$
2	$\frac{1}{Card(R,A)}=\frac{1}{2}$
4	红/绿 = 1/6

统计信息和开销模型

Ø 组合谓词

$$S_p(R)=\frac{T(\sigma_p(R))}{T(R)}$$

- 谓词条件$p=p1 \wedge p2 \wedge \ldots$
- 如何计算$S_p(R)$?
  - 在进行CBO时，$S_p(R)$本质上是概率
  - 当Sage, Ssex, Department, Sname相互独立时
    - $S_p(R)= S_{p1}(R)* S_{p2}(R)*\ldots$
  - E.g.
    - T(Student)=6;
    - $S_{Ssex=男}(Student)=1/2$；$S_{dept.=CS}(Student)=1/2$
    - if... Ssex 与Department相互独立，选择率$S_p(Student)=1/2*1/2=0.2$
    - BUT众所周知，’CS’



$$S_{Ssex=男 \wedge dept.=CS}(Student)=S_{Ssex=男}(Student)=S_{Ssex=男}(Student)=0.5$$

属性独立 OR not?

95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	CS
95005	陈刚	男	17	CS
95006	赵珊	女	19	IS

```
SELECT *
FROM Student
WHERE Ssex='男' AND Department='CS';
```

$S_p(R)$?

$S_{p1}(R)* S_{p2}(R)* S_{p3}(R)* S_{p4}(R)$

统计信息和开销模型

- So far，我们目前回答了以下问题
  - 什么是开销模型

$$cost = \sum_i cost_i = cost_i^{I/O} + cost_i^{CPU}$$

    - 用于寻找最优执行计划
  - 为什么要定义和计算选择率
    - 用于计算
    - 选择合适的物理操作符，以及连接顺序
  - 选择率的计算需要哪些统计信息
    - 总行数、基数、直方图
- BUT，还有一些问题
  - 如何维护统计信息?
    - DBMS定期后台更新（教材11.3.6节）
    - 主动收集ANALYZE
  - 多列的选择率（不独立时）？

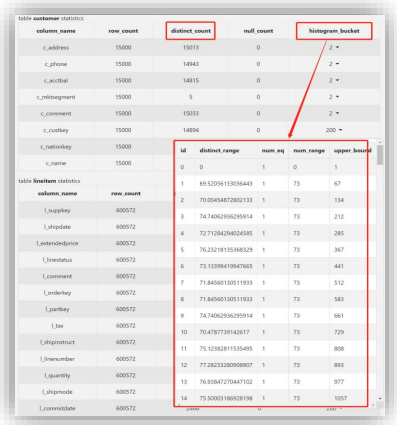
◆ 基数、直方图估计

- 基数：蓄水池采样->hyperloglog
- 直方图：蓄水池采样->等宽/等深直方图

本章阅读材料（开销模型）

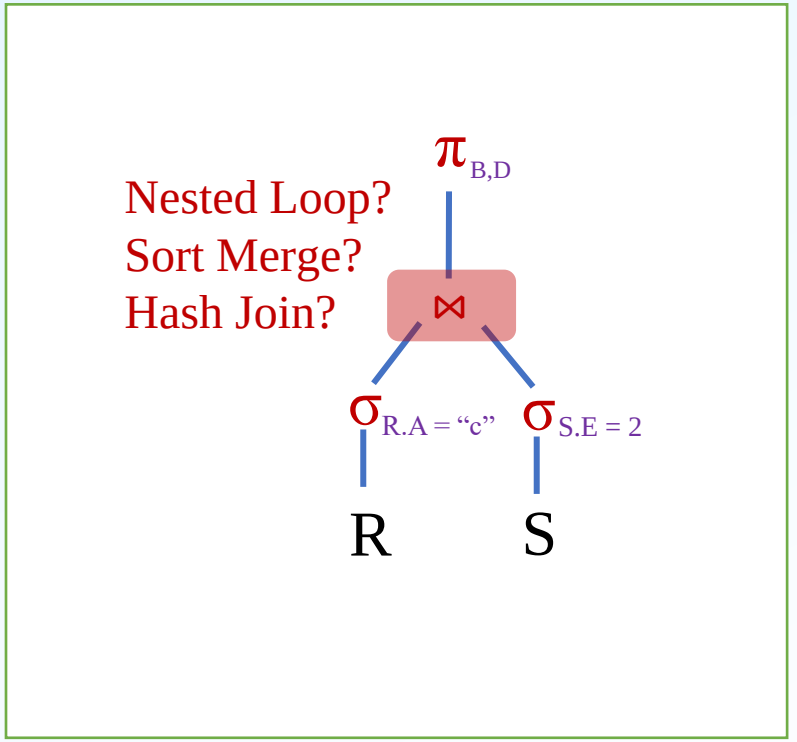


DBEDU 课程实训平台DBEDU（DBinsight）



统计信息和开销模型

- **统计信息和开销模型的局限性**
  - 统计信息选择只能支持原始表
    - 无法支持查询过程中的中间结果
    - 需要从原始表的统计信息出发
      - 级联估计
      - 可靠性级联递减
  - 多列选择率
    - 多列组合空间爆炸！！！

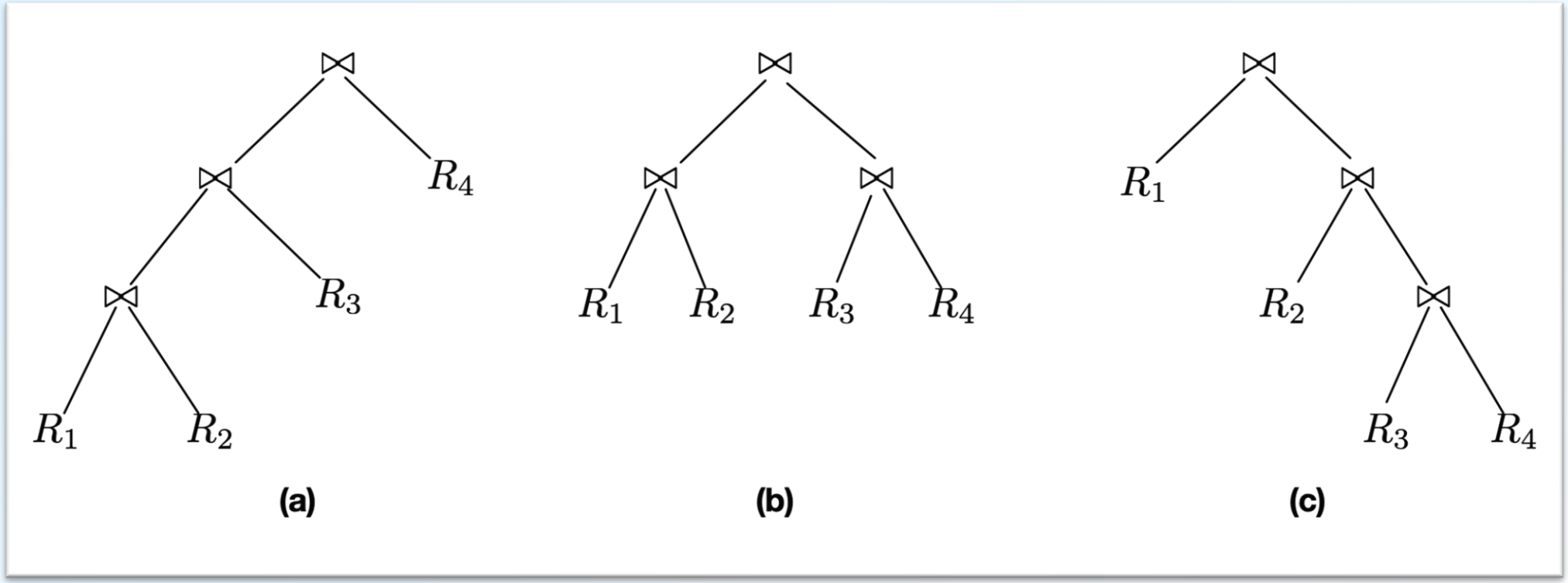


连接顺序选择

- **连接树（join tree）：表示n个关系连接时采用的连接顺序**
  - 二叉树
  - 内部节点都是连接运算
  - 每个节点与参与连接的关系一一对应

- **连接树分类**
  - **左深连接树：**内部节点的右子节点都是叶节点
  - **右深连接树：**内部节点的左子节点都是叶节点
  - **浓密树：**不是左深树或右深树

连接顺序选择（连接树示例）



左深树

$((R_1 \bowtie R_2) \bowtie R_3) \bowtie R_4$

浓密树

$((R_1 \bowtie R_2) \bowtie (R_3 \bowtie R_4))$

右深树

$R_1 \bowtie (R_2 \bowtie (R_3 \bowtie R_4))$

连接顺序选择（连接树数量）

- **n个关系构成的结构不同的连接树个数为：**
  - $C_n = \frac{1}{n}\binom{2n-2}{n-1} = \frac{(2n-2)!}{n!(n-1)!}$
  - 恰好为著名的**卡特兰数**（Catalan number）
- **对于每个连接树结构，n个关系的n!个全排列中每个都对应一组连接树**
  - 不同连接树个数为：$\mathrm{NTree}(n) = n!\,C_n = \frac{(2n-2)!}{(n-1)!}$
  - 规模迅速增加，$\mathrm{NTree}(10) \approx 10^{10}$
- **只考虑部分连接树——左深连接树 n!**

连接顺序选择（左深树）

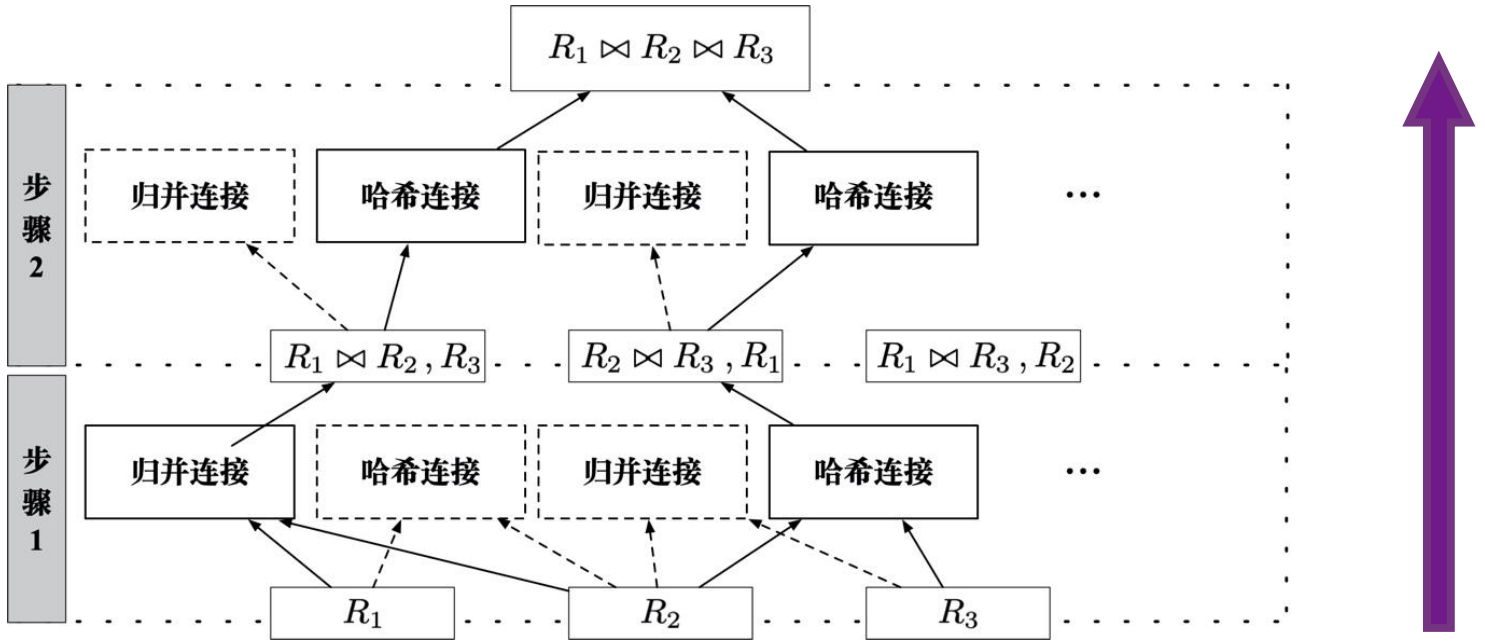
- **相较于所有连接树，左深树数量很小 n!：**
  - 对于n个关系连接的情况，只有1个确定的左深树结构
  - 例如，当$n = 10$时，左深树只占所有连接树数量的$\frac{1}{4862}$
  - 左深3628800 vs 稠密17643225600
- **左深连接树每个右子节点均为基本关系，利于连接运算高效执行**
  - 例如，嵌套循环连接可以利用基本关系已经建立的索引，执行更加高效
  - 避免将连接中一些过大的中间结果物化到磁盘上

- **连接顺序选择算法**
  - **动态规划**
  - **贪心算法**
  - **遗传算法**

物理算子选择

- 利用动态规划的物理算子选择
  - 在动态规划中添加每个算子的物理算子代价
- 最初从空的计划树开始，在第一步中先确定对哪两个关系进行连接

三个关系$R_1(A,B), R_2(A,C), R_3(A,D)$，某查询要得到$R_1 \bowtie R_2 \bowtie R_3$的结果



查询优化器组件

- **SQL语言的查询优化是一个NP-Hard问题！**

- **优化过程：选择最优的访问路径：**

  ① 需要解决以下几个基本问题

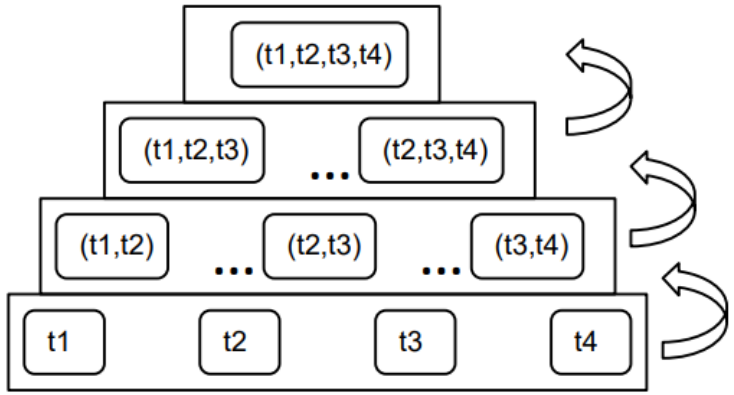
  - **代价估算；**
  - **连接顺序选择**
  - **物理算子选择**

  ② 枚举访问路径

访问计划枚举

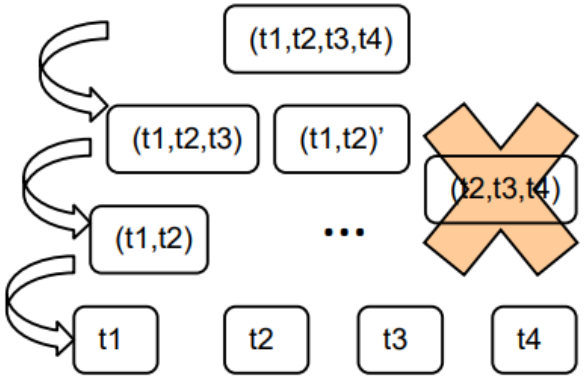
自底向上（例如R系统）

- 直观，易于实现
- 难以使用剪枝策略
- 无法提前停止



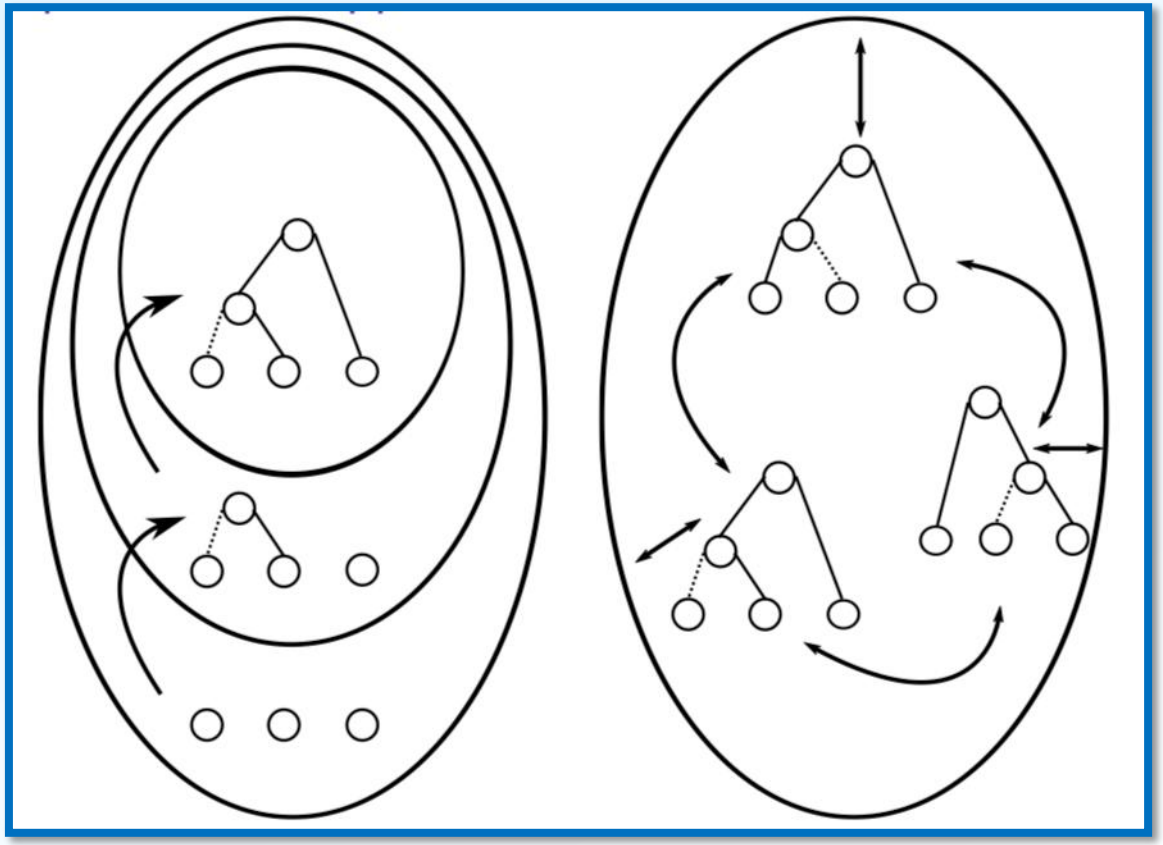
自顶向下（例如Cascades）

- 实现比较复杂
- 易于添加新规则
- 可以进行大量剪枝



访问计划枚举（自底向上 vs. 自顶向下）

- **自底向上（System R）：枚举计划并选择代价最低的计划**
- **自顶向下（Cascades）：优化的过程中不断剪枝**



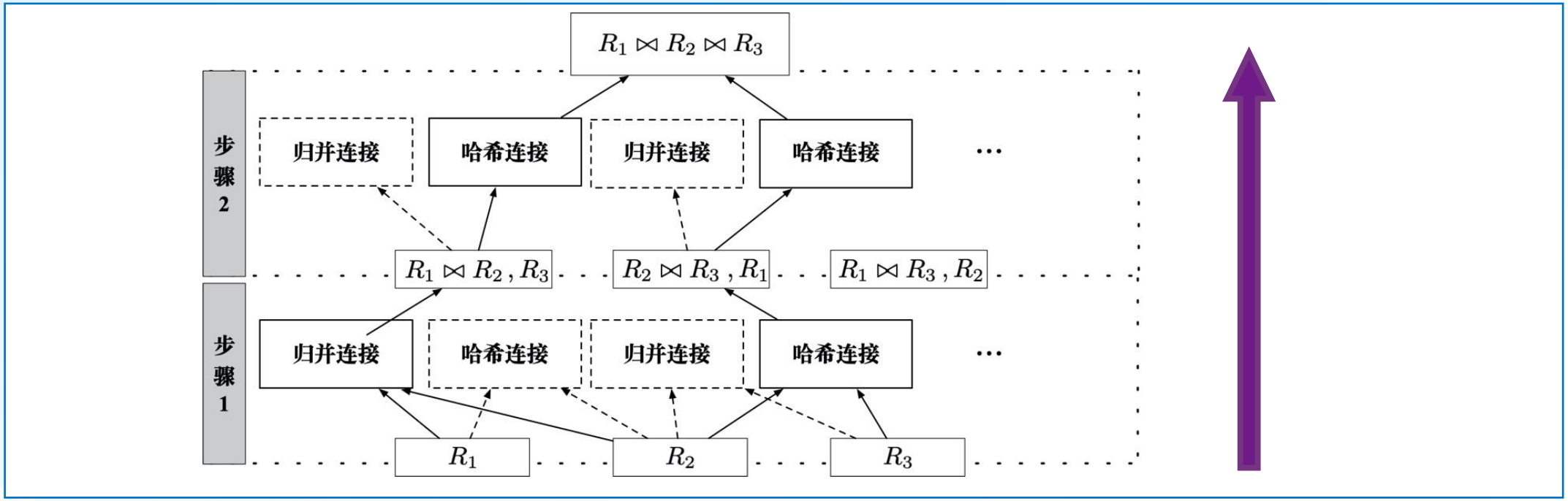
访问计划枚举（自底向上）

- **自底向上（System R）：**
  - 动态规划是一种典型的“自底向上”的方法
  - 由空白的物理计划开始，从叶子节点出发，考虑每个子表达式，对可能的转换方法搜索，结合代价估计，每一步中选取代价最小的物理运算，最终确定整个计划树

- **动态规划算法中，每个子表达式仅代价最小的计划会被保留**
  - 然而物理计划转换问题并不满足最优子结构的性质
  - 原始的动态规划算法并不能保证全局的最优

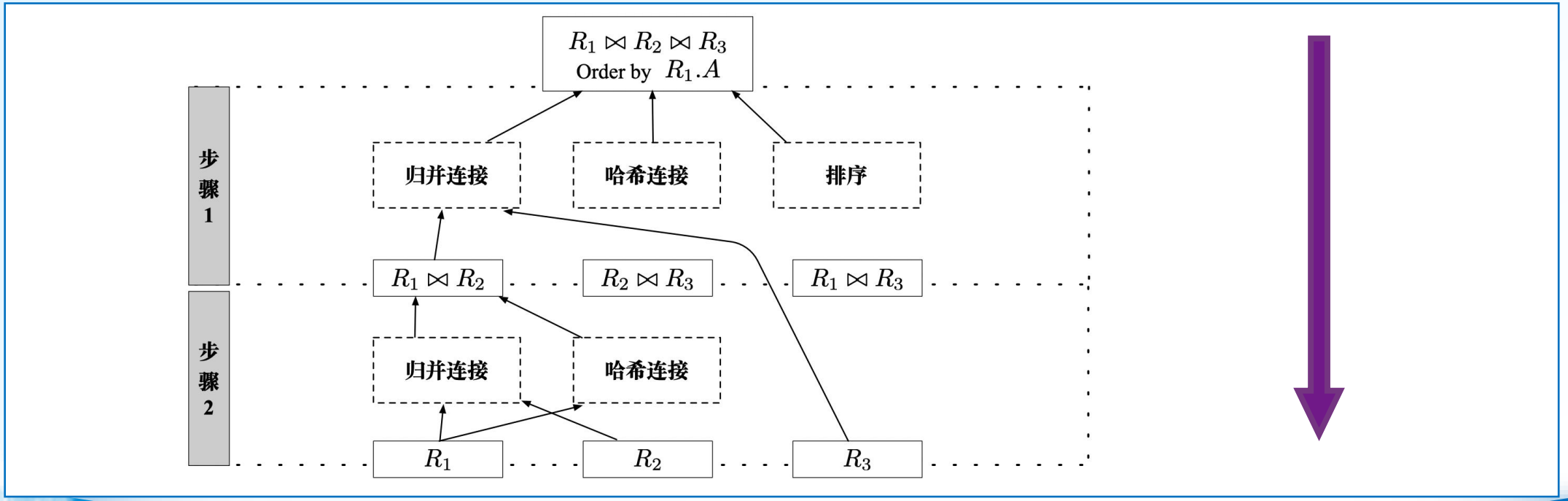
访问计划枚举（自底向上）

- 确定计划的过程中，通过比较估计代价的大小，对每个连接运算选择代价最小的物理算法
- **优点：** 容易实现，不需要穷尽搜索就能找到较为合理的执行计划
- **缺点：** 仍然没有摆脱启发式转换，优化时间长，添加规则繁琐。



访问计划枚举（自顶向下）

- **自顶向下（Cascades）：**
  - **通用的基于关系代数等价转换和代价模型的查询优化器**
    - 易于添加新的算子和等价转换规则
    - 重点关注数据的物理特性
    - 自顶向下，并利用分支界定剪枝



访问计划枚举（自顶向下）

- **自顶向下（Cascades）：**
  - 以查询的最终输出为起点，由计划树的根节点向下进行搜索，不断添加为得到最终输出需要的逻辑算子和物理算子，最终确定要采用的计划树
  - 整个过程中可以用分支界定（branch and bound）进行剪枝

  - **优点**：
    - 相较于两阶段的方法，统一搜索会产生更多的转换，优化效果可能更好
    - 可以提前中止
    - 添加规则简单
  - **缺点**：
    - 空间开销更大
    - 优化慢
    - 添加规则繁琐

查询优化总结

RBO-CBO

- **基于规则的优化（Rule-based Optimization, RBO）**
  - 查询的逻辑重写，以尽可能减少不合理的开销
  - 基于（关系代数的等价变换）规则
  - 与关系实例无关
- **基于代价的优化（Cost-based Optimization, CBO）**
  - 查询的物理优化
  - 使用代价模型，对操作进行代价估计
  - 生成：由物理操作符组成的物理执行计划（树）
  - 与关系实例相关
- **RBO+CBO**
  - 甚至会占据整个查询响应时间的一半以上
  - 且无法确保找到最优执行计划（NP问题）
  - 仍然不得不做
- **Otherwise,**

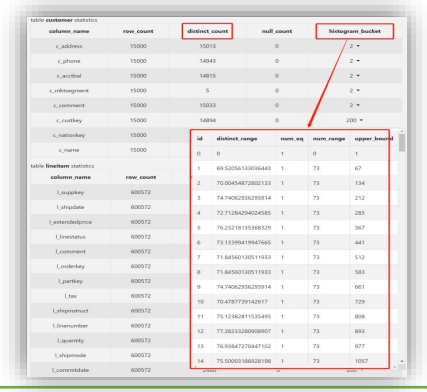




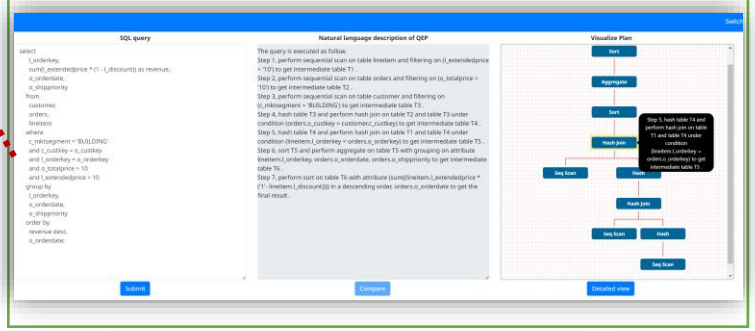
DBEDU



课程实训平台DBEDU.XIDIAN（DBinsight）



课程实训平台DBEDU.XIDIAN（LANTERN）

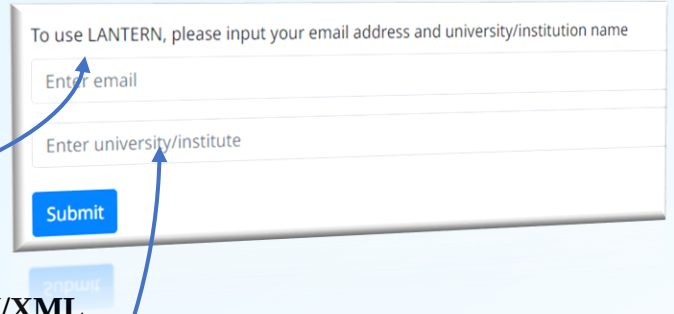


https://dbedu.xidian.edu.cn

本章实验

使用DBEDU以下各子系统实际体验查询处理和优化

- DBinsight
  - 使用预设的查询或自拟ad-hoc查询体验查询处理的全流程
  - 至少覆盖AST、统计信息、优化步骤、执行计划及对应的代价
- LANTERN
  - 使用预设的查询或自拟ad-hoc查询生成对应的执行计划，尝试理解原始执行计划的JSON/XML
  - 使用LANTERN系统生成该计划的自然语言解释，再次理解对应的计划
- ARENA
  - 使用预设的查询或自拟ad-hoc查询提交查询请求，系统获得优化器选择的执行计划QEP
  - 设置对应的参数，用系统生成QEP之外的可能候选计划AQP
  - 利用系统对比AQP与QEP，尝试分析并理解优化器为何选择QEP而放弃了AQP
- ChatQPT
  - 基于大语言模型的一站式交互型学习工具，实现LANTERN、ARENA等子系统的串联学习
  - http://8.219.239.191/（课程内部使用，暂不对外开放，要求每人每天利用该系统开展一次长交互对话，20天+，对话内容至少需要覆盖LANTERN和ARENA，使用前先观看视频教程https://youtu.be/r4j3l3IxFHE）



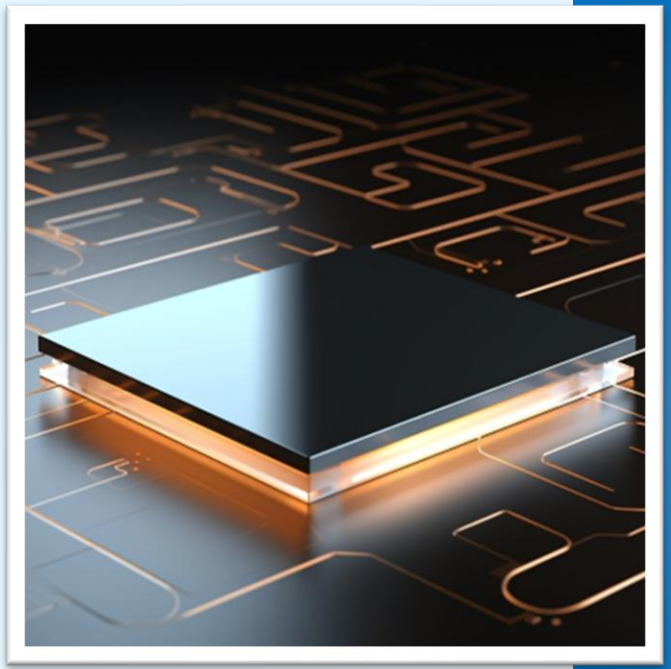
- 实训平台网址：https://dbedu.xidian.edu.cn，上述各子系统都有对应的使用说明视频

Ø 使用“学号@DBkernel”作为邮箱(如 1010230412@DBkernel)，“Xidian University”作为单位名登录使用

Ø 系统会记录所有上述信息登录的用户的完整使用记录

Ø 大家的使用记录会作为实验成绩的一部分，占最终总评成绩的20%

本章内容



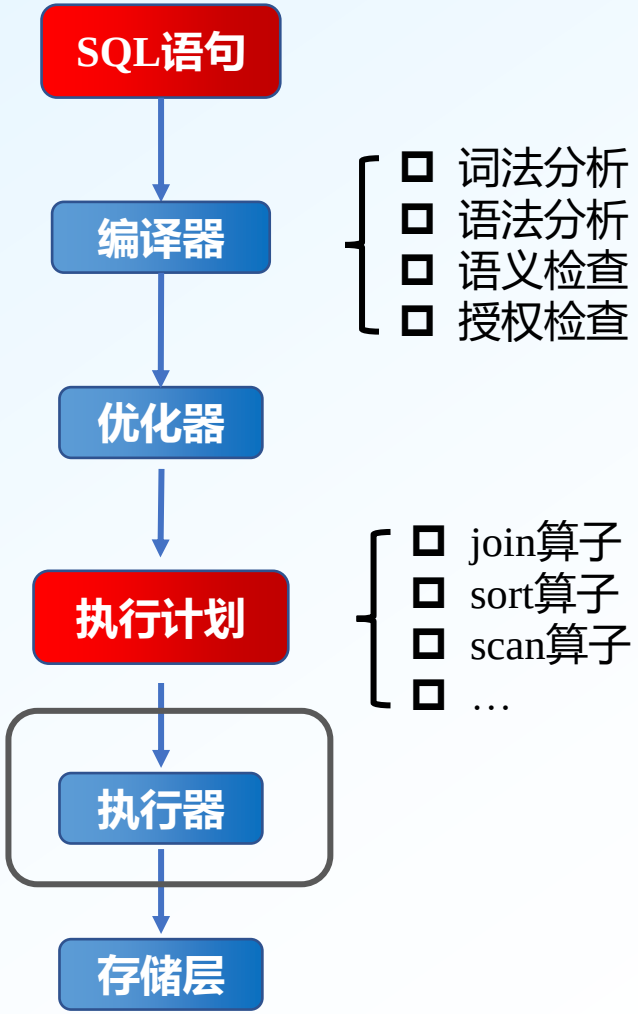
一、查询解析

二、逻辑重写

三、物理算子

四、查询优化

五、执行引擎



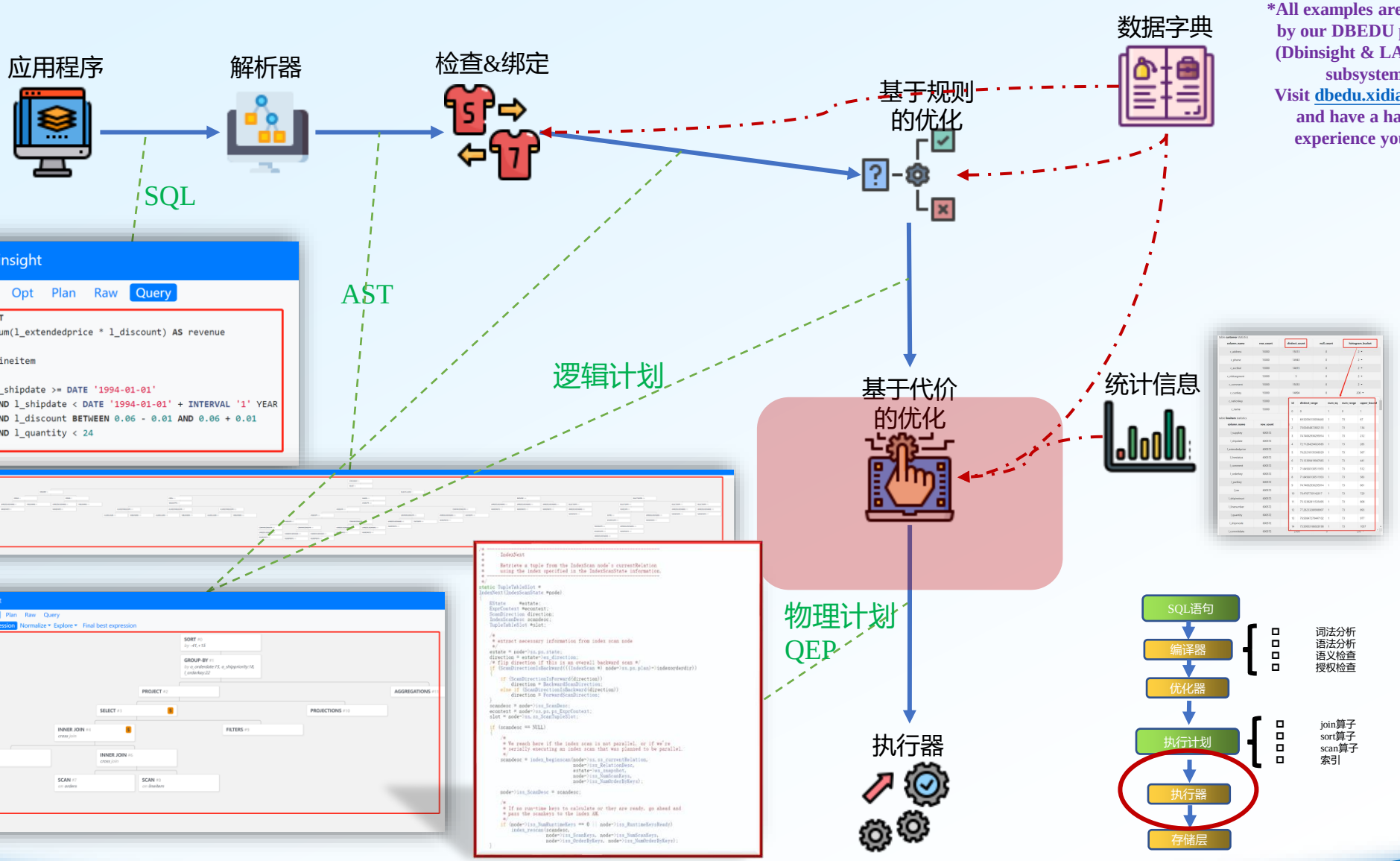
查询优化CBO

SQL引擎pipeline

西安电子科技大学 XIDIAN UNIVERSITY

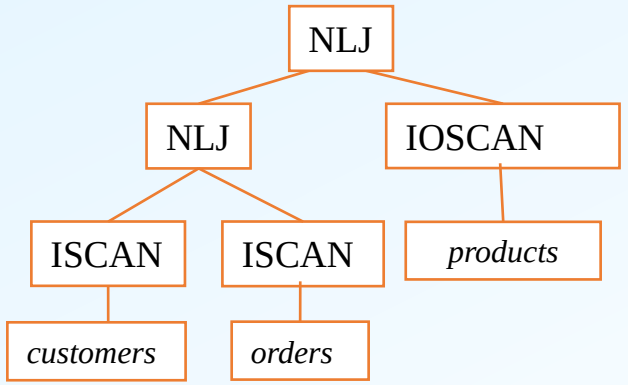
*All examples are provided by our DBEDU platform (Dbinsight & LANTERN subsystems) Visit dbedu.xidian.edu.cn and have a hand-on experience yourself!

DBEDU



执行计划

- **CBO输出**
  - “最优”的物理计划
    - 执行计划QEP（Query Execution Plan）
    - 算法区别 -> 执行代价（cost）
- **执行模型**
  - 给定QEP，DBMS执行的流程
    - QEP中每个物理操作符
      - 对应一段具体代码实现
  - 执行模型需要考虑如何从输入关系实例，结合QEP，串联各操作符的代码实现，以输出最终结果
    - 物化模型：每一步物理操作符的中间结果物化（甚至落盘）后，用于下一步输入
    - 火山模型/迭代模型：物理操作符的结果元组直接发送给下一个操作符使用（每个操作符需要实现一个Next ( )方法），中间不需物化
      - 向量化模型：不是每次传递一个元组，而是每次一批
        - 以更好的利用现代CPU的SIMD指令



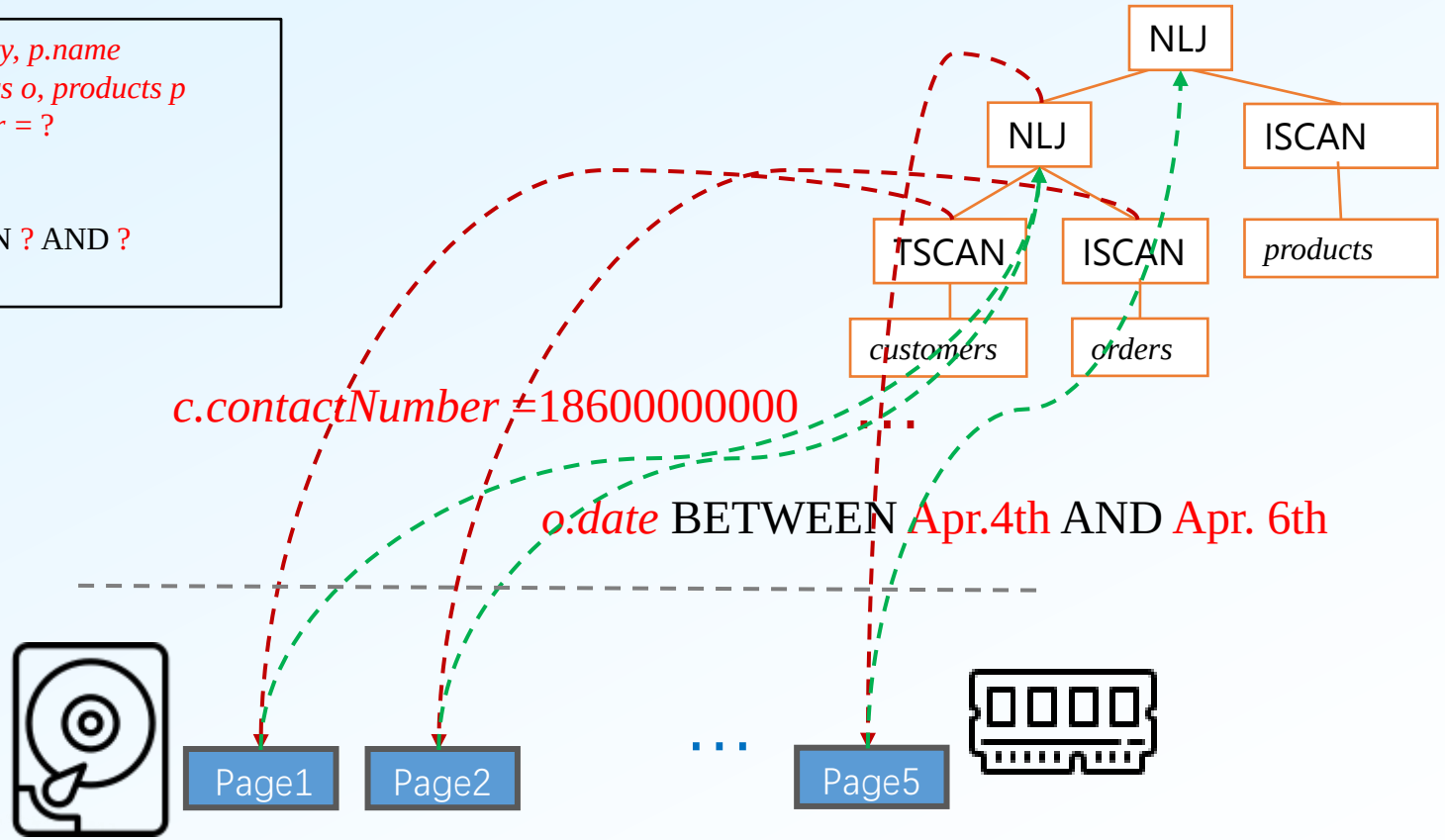

```
SELECT o.date, o.quantity, p.name
FROM customers c, orders o, products p
WHERE c.contactNumber = ?
   AND c.id = o.cid
   AND p.id = o.pid
   AND o.date BETWEEN ? AND ?
ORDER BY o.date
```

物化模型

■ 物化模型

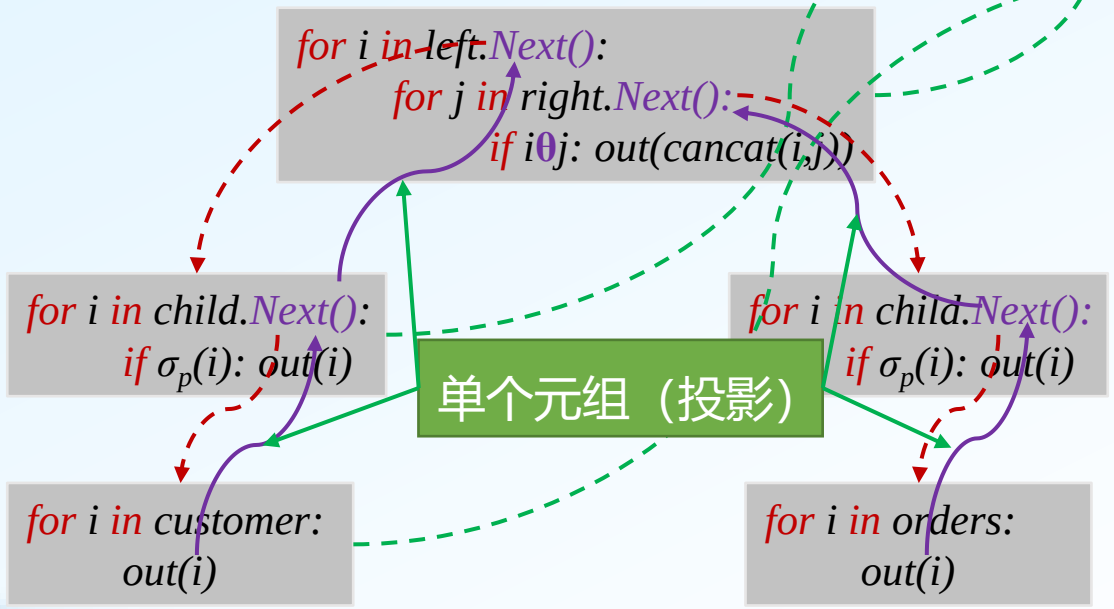
- ➢ 每个操作符一次性读入所有输入（元组）
- ➢ 一次性输出所有（元组）

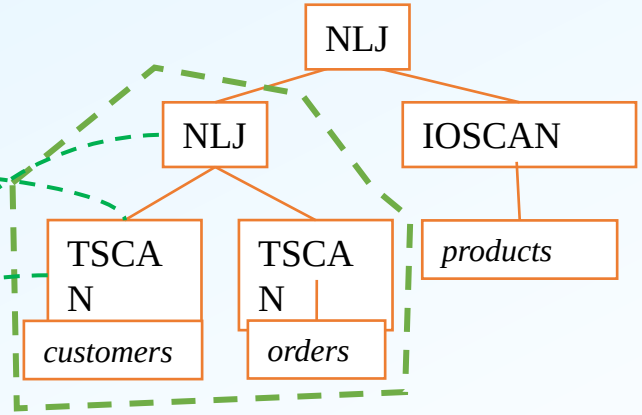
```
SELECT o.date, o.quantity, p.name
FROM customers c, orders o, products p
WHERE c.contactNumber = ?
   AND c.id = o.cid
   AND p.id = o.pid
   AND o.date BETWEEN ? AND ?
ORDER BY o.date
```



火山模型

- **火山模型/管线化模型**
  - 每个操作符看成是迭代器
    - Open(), Next(), Close()
    - 父节点->外层迭代
    - 子节点->内层迭代
    - 操作符（迭代层）间传递的是单个元组（投影）
      - 随时产生随时迭代处理

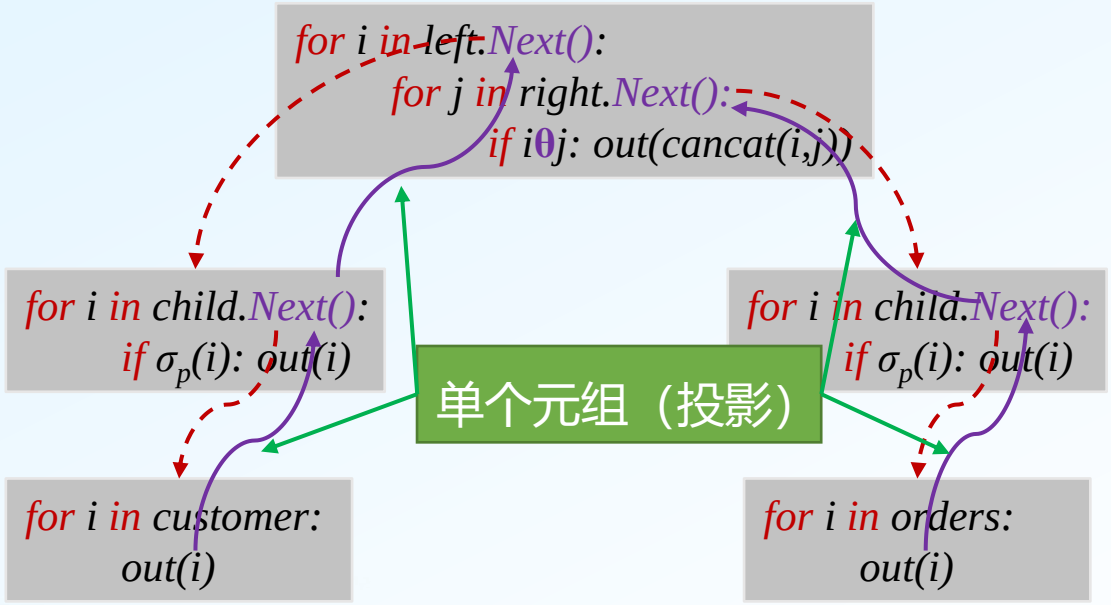


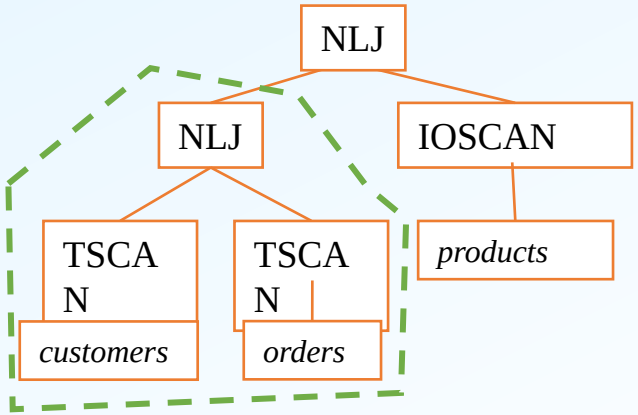



```
SELECT o.date, o.quantity, p.name
FROM customers c, orders o, products p
WHERE c.contactNumber = ?
   AND c.id = o.cid
   AND p.id = o.pid
   AND o.date BETWEEN ? AND ?
ORDER BY o.date
```

火山模型

- **向量化模型**
  - 同火山模型，每个操作符看成是迭代器
  - 异：操作符（迭代层）间传递的
    - 不是单个元组
    - 而是一批元组
      - 攒够一批进行传递
      - 最大化利用SIMD指令特性

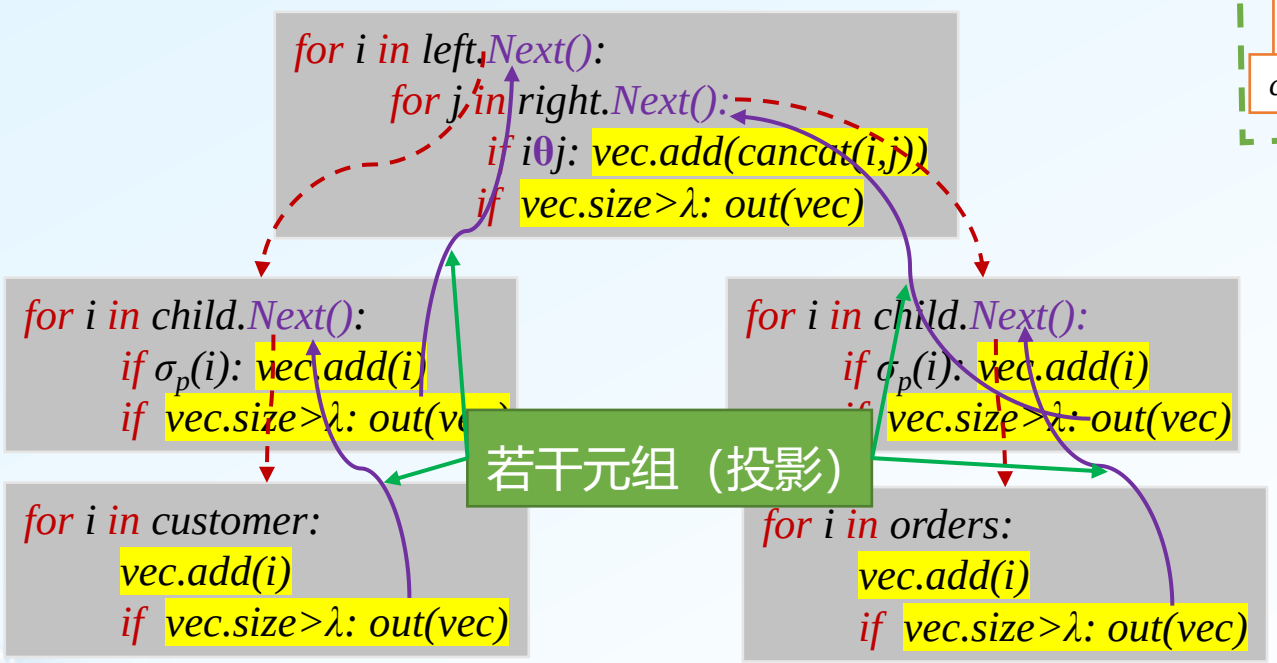


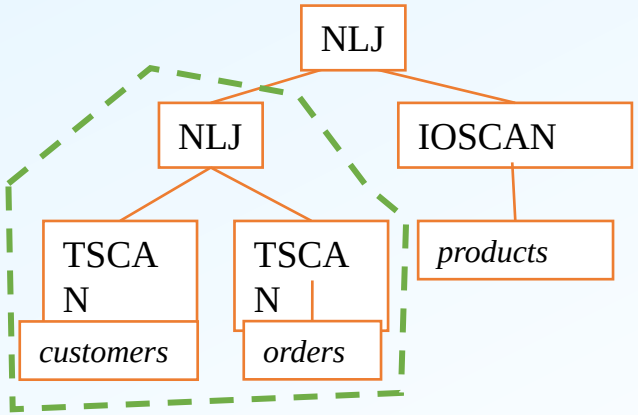



```
SELECT o.date, o.quantity, p.name
FROM customers c, orders o, products p
WHERE c.contactNumber = ?
   AND c.id = o.cid
   AND p.id = o.pid
   AND o.date BETWEEN ? AND ?
ORDER BY o.date
```

火山模型

- **向量化模型**
  - 同火山模型，每个操作符看成是迭代器
  - 异：操作符（迭代层）间传递的
    - 不是单个元组
    - 而是一批元组
      - 攒够一批进行传递
      - 最大化利用SIMD指令特性

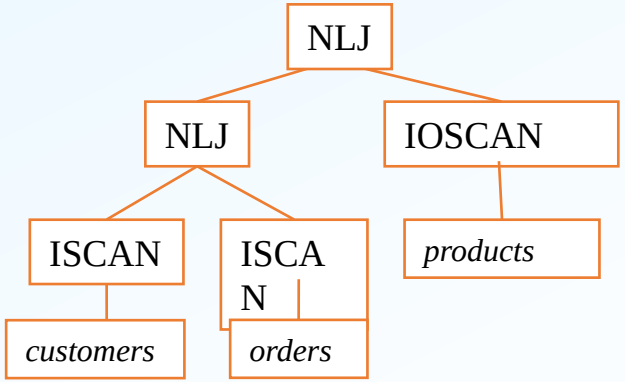





```
SELECT o.date, o.quantity, p.name
FROM customers c, orders o, products p
WHERE c.contactNumber = ?
   AND c.id = o.cid
   AND p.id = o.pid
   AND o.date BETWEEN ? AND ?
ORDER BY o.date
```

编译型执行器

- **解释型执行器**
  - **物化模型**：每一步物理操作符的中间结果物化（甚至落盘）后，用于下一步输入
  - **火山模型/迭代模型**：物理操作符的结果元组直接发送给下一个操作符使用（每个操作符需要实现一个Next()方法），中间不需物化
  - **向量化模型**：类比于火山模型，但不是每次一个元组，而是每次一批
    - 以更好的利用现代CPU的SIMD指令
- **编译型执行器**
  - **解释型执行器的局限性**
    - 需要反复查找元数据，以在元组中找到目标列
      - 反复嵌套调用函数find(tuple, attributeID)
    - （预处理PREPARE后）表达式计算需要很多额外开销
      - E.g.
  - 即时编译技术JIT（Just-in-time Compilation）
    - 即时生成字节码（in JVM）/IR（in LLVM）
    - **编译器带来的优势**
      - **变量常量化**：E.g., 分量在元组的位置偏移常量化
      - **最小化函数调用**：E.g., 解释型固守的规则化的函数调用可在编译中最小化
      - LLVM的支持下，还可轻松支持向量化的指令集




```
SELECT o.date, o.quantity, p.name
FROM customers c, orders o, products p
WHERE c.contactNumber = ?
  AND c.id = o.cid
  AND p.id = o.pid
  AND o.date BETWEEN ? AND ?
ORDER BY o.date
```

参考资料

参考书 & DBEDU（西电库道）

西安电子科技大学 XIDIAN UNIVERSITY

参考教材

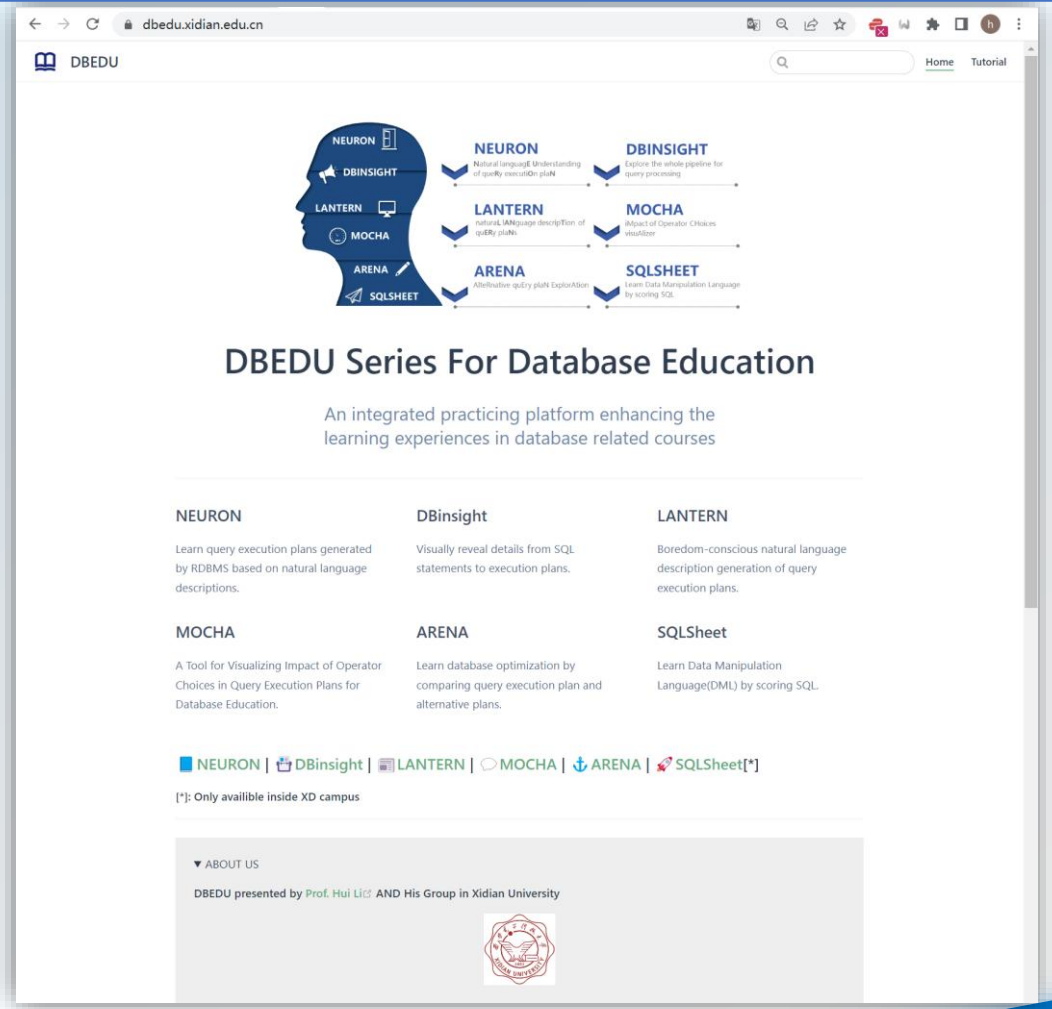


- 高等教育出版社
- ISBN：9787040605266

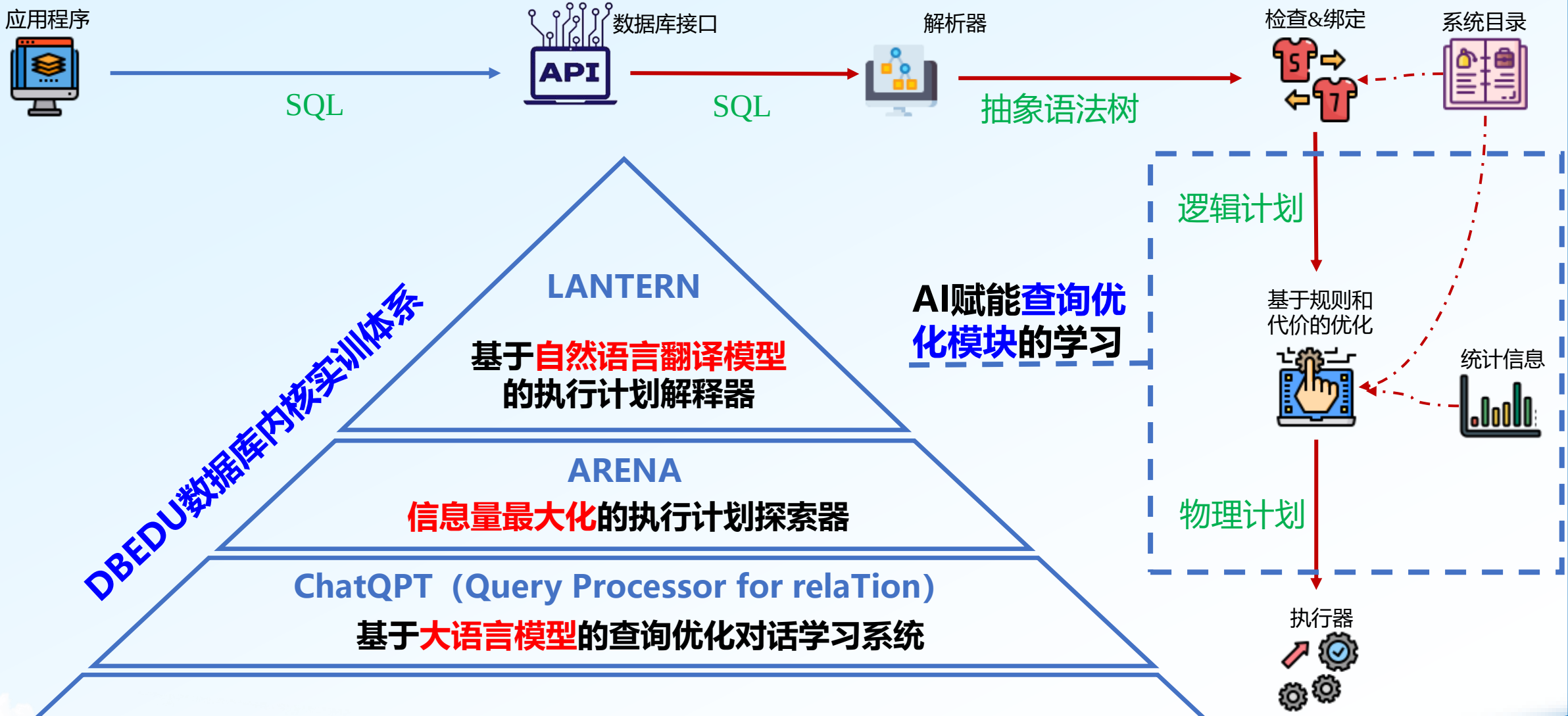
“101计划”教材试点应用教材清单

	教材名称	主编	学校	出版社
1	计算导论——计算+、互联网+与人工智能+	战德臣	哈尔滨工业大学	高等教育出版社
2	计算机科学导论	徐志伟	中国科学院大学	清华大学出版社
3	数据结构	俞勇	上海交通大学	高等教育出版社
4	计算机系统（基于RISC-V+Linux平台）	袁春风	南京大学	高等教育出版社
5	计算机系统	刘宏伟	哈尔滨工业大学	高等教育出版社
6	计算机组成与实现	高小鹏	北京航空航天大学	高等教育出版社
7	编译原理与技术	张莉	北京航空航天大学	高等教育出版社
8	计算机网络	胡事	吉林大学	高等教育出版社
9	数据库管理系统——从基本原理到系统构建	李国良	清华大学	高等教育出版社
10	软件工程——从理论到实践	毛新军	国防科技大学	高等教育出版社
11	软件工程——经典、现代和前沿	孙艳春	北京大学	北京大学出版社
12	人工智能引论	吴飞	浙江大学	高等教育出版社

查询处理实训平台（https://dbedu.xidian.edu.cn）



DBEDU（西电库道）平台子系统功能说明





感谢观看

李辉　2025.6

西安电子科技大学计算机科学与技术学院

hli@xidian.edu.cn

https://lihuixidian.github.io

https://dbedu.xidian.edu.cn